

**CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION
TECHNOLOGY**



**UNDERGRADUATE THESIS MAJOR IN
INFORMATION TECHNOLOGY
(HIGH-QUALITY PROGRAM)**

TOPIC:

**AI-BASED NPC INTERACTION SYSTEM FOR
VIDEO GAMES**

STUDENT: TRAN MINH QUANG

STUDENT ID: B2111946

COURSE: K47

Can Tho 12/2025

**CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION
TECHNOLOGY**



**UNDERGRADUATE THESIS MAJOR IN
INFORMATION TECHNOLOGY
(HIGH-QUALITY PROGRAM)**

TOPIC:

**AI-BASED NPC INTERACTION SYSTEM FOR
VIDEO GAMES**

Advisor:
Dr. THAI MINH TUAN

Student:
**TRAN MINH
QUANG
ID: B2111946
Course: 47**

Can Tho, 12/2025

This image shows a full page of a document template designed for handwritten notes or essays. It features approximately 28 evenly spaced, thin grey horizontal lines extending across the entire width of the page. The margins are consistent on all sides, providing ample space for writing. There are no pre-printed questions, headings, or other markings on the page.

Advisor

Dr. Thai Minh Tuan

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to Dr. Thai Minh Tuan, my thesis advisor. His guidance has been invaluable throughout this entire journey. Her profound knowledge and extensive experience in the field provided me with constant encouragement and direction, not only during my research but also in navigating the challenges of academic life.

To my family, I offer my heartfelt appreciation for their unwavering support. Their financial backing and constant encouragement allowed me to dedicate myself fully to this research and persevere through the demanding process.

Finally, I extend my sincere thanks to my friends and classmates. Their insightful comments, thoughtful discussions, and unwavering support throughout this endeavor have been instrumental in refining my work.

Thank you sincerely!

Can Tho, day ... month ... year 2025

Student

Trần Minh Quang

CONTENTS

Contents

INSTRUCTOR'S COMMENT	1
ACKNOWLEDGEMENT	2
CONTENTS	3
PICTURE LIST	4
TABLE LIST.....	9
LIST OF ABBREVIATED TERMS.....	10
ABSTRACT	11
PART I: INTRODUCTION	12
I. Problem.....	13
II. Previous Solution	14
III. Project Purpose.....	15
1. NPC AI and Interaction Systems:.....	15
2. Supporting Game Systems:	17
IV. Target and scope of research.....	17
1. Target.....	17
2. Scope of Research	18
V. Research Content.....	18
PART 2: CONTENTS OF THE PROJECT	20
CHAPTER 1: PROJECT OVERVIEW	21
I. PROJECT DESCRIPTION	21
II. SOLUTION FOR THE PROJECT.....	29
CHAPTER 2: DESIGN AND PROGRAMMING OF GAME FUNCTIONS	32
I. THEORETICAL BASIS	32
a. Unity	32
b. Basic Components in Unity2D	34
c. Supporting Website and App.....	47
II. GAME DESIGN.....	50
III. GAME BUILD	67
IV. PROGRAMMING GAME COMPONENT FUNCTIONALITY	69

CHAPTER 3: NPC AI SYSTEM	96
I. NPC AI Overview	96
II. Routine System Concept	98
III. Routine System – Detailed Activity Breakdown.....	100
IV. AI-Driven Dialogue System	102
CHAPTER 4: TESTING AND EVALUATION.....	105
I. TESTING OBJECTIVES	105
II. TESTING PLAN	105
III. TESTING RESULT EVALUATION	107
PART 3: CONCLUSION.....	132
I. CONCLUSION	133
II. DEVELOPMENT DIRECTION.....	134
REFERENCES	135

Picture 1.1 The Elder Scrolls V: Skyrim - NPC following autonomous daily schedule.....	22
Picture 1.2 Voice Assistant Technology – Architecture of natural language interaction.....	22
Picture 1.3. AI Dungeon - AI-powered contextual dialogue generation	23
Picture 1.4. AI Dungeon – Charactor created	23
Picture 1.5. Game Flowchart.....	31
Picture 2.1. Unity 6	32
Picture 2.2. Unity Asset Store in Unity	34
Picture 2.3. Unity Asset Store filters	35
Picture 2.4. Assets in Project folder	35
Picture 2.5. Scene component in Unity	36
Picture 2.6. Hierarchy	37
Picture 2.7. Scenes in Asset folder of the Project	38
Picture 2.8. Game Object in Scene.....	38
Picture 2.9. Move tool in Scene (key W).....	39
Picture 2.10. Rotate tool in Scene (key E)	39
Picture 2.11. Scale tool in Scene (key R).....	40
Picture 2.12. Transform field	40
Picture 2.13.Components of a Game Object in Unity	41
Picture 2.14. Add Component for Game Object	42
Picture 2.15. Create new Component	42
Picture 2.16. Collider shape	43
Picture 2.17. Rigidbody2D of Game Object.....	44
Picture 2.18. Script for NPCs	45
Picture 2.19. A Prefab folder save in Asset folder	46
Picture 2.20. Saved prefab use in a scene	47
Picture 2.21. itch.to website	47
Picture 2.22. ChatGPT AI website.....	48
Picture 2.23. Aseprite Interface.....	49
Picture 2.24. LM Studio Interface.....	49
Picture 2.25. MySQL Workbench Interface	50
Picture 2.26. Pixelarium - Player asset	50
Picture 2.27. Pixelarium - Slime monster asset.....	51

Picture 2.28. Pixelarium - Barrels and wooden crates asset.....	51
Picture 2.29. Mystic Woods - Environment asset	52
Picture 2.30. Red Hood NPC asset.....	52
Picture 2.31. Red Hood NPC asset.....	53
Picture 2.32. FlowerMarket asset.....	54
Picture 2.33. Inventory asset.....	54
Picture 2.34. Spring Day and Night asset.....	55
Picture 2.35. Coins asset 1.....	55
Picture 2.36. Coins asset 2.....	56
Picture 2.37. Prefab asset.....	56
Picture 2.38. Level Interface.....	57
Picture 2.39. Mainmenu Interface	57
Picture 2.40. Pause game Interface	58
Picture 2.41. Gameplay Interface.....	58
Picture 2.42. Gameplay Dialog NPCs	59
Picture 2.43. Gameplay Inventory	59
Picture 2.44. Gameplay Items Descriptions	60
Picture 2.45. Gameplay CoinsInventory	60
Picture 2.46. Gameplay QuestPanel.....	61
Picture 2.47. Gameplay Trade Interface	61
Picture 2.48. Environment Layout 1.....	67
Picture 2.49. Environment Layout 2.....	67
Picture 2.50. Slime Enemy	68
Picture 2.51. Grape Shotter Enemy	68
Picture 2.52. Main character spawn	69
Picture 2.53. Player Movement Flowchart.....	70
Picture 2.54. Sword attack flowchart	71
Picture 2.55. Bow attack flowchart.....	72
Picture 2.56. Staff attack flowchart	73
Picture 2.57. Knockback flowchart.....	74
Picture 2.58. Player health flowchart	75
Picture 2.59. Item Spawn and Pickup flowchart	76
Picture 2.60. Main NPC Routine Loop.....	77

Picture 2.61. Market Trading Routine	78
Picture 2.62. Flowers Hunting Routine	79
Picture 2.63. Dialogue Pause & Player Override System	80
Picture 2.64. NPC Voice-to-AI Dialogue System	81
Picture 2.65. NPC Voice-to-AI Dialogue System	82
Picture 2.66. Quest Progress Tracking & Completion.....	83
Picture 2.67. NPC Pathfinding (A* Algorithm).....	84
Picture 2.68. Shop Trading Transaction	85
Picture 2.69. Shopping Cart System	86
Picture 2.70. Inventory Add/Remove Item	87
Picture 2.71. View Item Detail.....	88
Picture 2.72. Time Manager	89
Picture 2.73. Player Stamina	90
Picture 2.74. Flower Respawn	91
Picture 2.75. Player Move	93
Picture 2.76. Player Dash.....	93
Picture 2.77. Player Sword Attack.....	94
Picture 2.78. Player Staff Attack.....	94
Picture 2.79. Player Bow Attack	95
Picture 2.80. NPC interaction.....	95
Picture 3.1. NPC AI Component Diagram.....	96
Picture 3.2. HFSM Structure for NPC Behavior.....	98
Picture 3.3. Trading Time from 8:00 to 12:00	99
Picture 3.4. Flower Gathering Time from 14:00 to 16:00.....	99
Picture 3.5. Market Trading Interaction.....	102
Picture 3.6. LLM Response.....	103
Picture 3.7. NPC Dialog Conversiation and Intent	104
Picture 4.1. Build Game window and setting of Unity	107
Picture 4.2. Begin to build game.....	108
Picture 4.3. The files and directories of the game after packaged	108
Picture 4.4. Game Menu Interface	109
Picture 4.5. Pause Game Interface	109
Picture 4.6. Game Scene starting the game.....	110

Picture 4.7. Change Weapon	110
Picture 4.8. Sword Slash Effect	111
Picture 4.9. Staff Laser Effect	111
Picture 4.10. Arrow Effect	112
Picture 4.11. Knockback and Flash Effect	112
Picture 4.12. Dash Effect and Lose Stamina	113
Picture 4.13. Structure Destruction	113
Picture 4.14. Enemy Death	114
Picture 4.15. Dropped Items	114
Picture 4.16. Picking up Coins.....	115
Picture 4.17. Picking up EXP and Health	115
Picture 4.18. Attacked Effect.....	116
Picture 4.19. Death Effects.....	116
Picture 4.20. Player Inventory.....	117
Picture 4.21. Items Detail.....	117
Picture 4.22. Coins Bag	118
Picture 4.23. Time Moring Light.....	118
Picture 4.24. Time Midday Light	119
Picture 4.25. Time Night Light	119
Picture 4.26. NPC Dialogue	120
Picture 4.27. Players Speak.....	120
Picture 4.28. NPC Reply	121
Picture 4.29. Ask NPC for Quest.....	121
Picture 4.30. Reply with Quest content	122
Picture 4.31. Player accept the Quest	122
Picture 4.31. Quest Panel List	123
Picture 4.33. Quest Detail Panel.....	123
Picture 4.34. Quest Prograss Updated.....	124
Picture 4.36. Quest Report.....	125
Picture 4.37. Quest Reward	125
Picture 4.38. Shop Opened.....	126
Picture 4.39. Shop UI.....	126
Picture 4.40. Buy some Flowers.....	127

Picture 4.41. Pay Coins	127
Picture 4.42. Not Enough Money Error	128
Picture 4.43 Valid Payment	128
Picture 4.44. Items Updated	129
Picture 4.45. NPC Flower Gathering Routine	129
Picture 4.46. Asking for Village Position.....	130
Picture 4.47. NPC show the way.....	130
Picture 4.48. Firefly pathfinding	131

TABLE LIST

Table 2.1. Table of interactive components	63
Table 2.2. Table of scene components.....	66
Table 3.1. Table of the testing results according to the scenarios.....	106

LIST OF ABBREVIATED TERMS

NPC	Non-Playable Character	A character that is controlled by the game's programming rather than by the player
RPG	Role-Playing Games	A genre of video games where players assume the roles of characters in a fictional setting
IDE	Integrated Development Environment	A software application that provides comprehensive tools and features to facilitate software development
NLP	Natural Language Processing	Technology that enables computers to understand and process human language.
LLM	Large Language Model	An AI model trained on massive text data to understand and generate natural language
API	Application Programming Interface	A communication interface that allows different systems or applications to exchange data.
LM	Local Model (LM Studio)	A language model running locally on the user's computer through LM Studio
TTS	Text-to-Speech	Technology that converts written text into audible speech
HFSM	Hierarchical Finite State Machine	A layered state-machine system used to manage NPC behaviors in games

ABSTRACT

In the era of the fourth industrial revolution, artificial intelligence technology is rapidly transforming the gaming industry worldwide. Game development has evolved beyond traditional interaction methods, with AI and voice recognition becoming key drivers of innovation in creating immersive player experiences. This graduation thesis explores the development of an AI-powered voice interaction system for NPCs in video games, implemented using Unity Engine and OpenAI technology. The system, built for Windows platforms and programmed in C# with Python backend services, features natural voice-driven dialogue, context-aware AI responses, and dynamic NPC behavior routines. The game incorporates advanced features such as speech recognition, intelligent quest management, time-based NPC activities, and a comprehensive economy system. The design aims to demonstrate how AI integration can create more natural and engaging interactions between players and game characters, providing a foundation for next-generation game development approaches.

Keywords: Unity, Game 2D, Fantasy Game, AI, NPC.

PART I: INTRODUCTION

I. Problem

Gaming has become an indispensable form of entertainment in the modern era. Throughout various periods, game genres have continuously evolved to meet the diverse needs of players. Contemporary games integrate numerous features and modern technologies, notably: Artificial Intelligence (AI), Natural Language

Processing (NLP), and Real-time Interactive Systems. 2D RPG games – games designed with character sprites and environments in a two-dimensional space, are built on various platforms to create diversity and novelty to attract players. Among these, Unity is one of the top choices for game design and development, especially for students and individual developers during their research and learning period, as it is free to use (Personal Edition) and has a large supportive community.

Traditional RPG games typically rely on pre-scripted dialogue systems with limited interaction options. Players are confined to clicking through predetermined dialogue trees or selecting from a fixed set of responses. This approach significantly restricts player freedom and immersion, as NPCs cannot understand or respond to natural language input, making interactions feel mechanical and repetitive.

Furthermore, most existing games feature static NPCs with no autonomous behavior or daily routines. NPCs stand idle in fixed locations, waiting for player interaction, which breaks immersion and makes the game world feel artificial and lifeless. There is a lack of dynamic, time-based NPC behaviors that simulate realistic daily activities such as working, trading, and resting.

Modern AI technologies, particularly Large Language Models (LLMs) and Speech Recognition systems, present an opportunity to revolutionize NPC interactions in games. By integrating these technologies, we can create NPCs that understand natural language, respond contextually to player queries, and exhibit autonomous behaviors based on in-game time and environmental factors.

Recognizing these development trends and opportunities, during the period of learning and researching game design and development knowledge, I developed the idea and proposed the project: "**AI- Based NPC INTERACTION SYSTEMS FOR VIDEO GAMES**". This document presents a comprehensive overview of the research and development of an interactive video game environment on Unity, integrating AI-powered voice interaction, context-aware dialogue systems, NPC routine AI with time-based behaviors, and database-driven quest management systems.

For an information technology student, this is essential to equip oneself with development experience to keep pace with the times and become a contributing factor in the era of the 4.0 technological revolution. This project demonstrates the practical application of modern AI technologies in game development, bridging the gap between

traditional game mechanics and next-generation interactive experiences.

II. Previous Solution

Today, NPC AI systems in video games have evolved significantly, with various approaches being developed to create more realistic and engaging character interactions. Developers have experimented with different technologies ranging from simple scripted behaviors to advanced machine learning models.

Traditional NPC systems rely heavily on finite state machines and behavior trees. Games like The Elder Scrolls series and Red Dead Redemption 2 showcase impressive NPC routines where characters follow daily schedules, go to work, eat meals, and sleep. However, their dialogue remains entirely pre-scripted. Players can only select from predetermined conversation options, and NPCs cannot understand or respond to anything outside their scripted lines.

The introduction of voice assistants opened new possibilities. Bethesda's Skyrim Alexa integration allowed players to use voice commands for game actions, but this was limited to simple commands like "use potion" or "save game" rather than actual conversations with NPCs. The interaction remained mechanical and couldn't support natural dialogue.

AI Dungeon and NovelAI demonstrated that modern language models could generate dynamic, contextual narratives. These text-based games use GPT models to create unlimited story possibilities based on player input. However, they lack structured gameplay systems—no quest tracking, no economy management, no visual representation. The experience is purely narrative-driven without the mechanics that define traditional RPGs. Commercial AI platforms have emerged to bridge this gap. Inworld AI and Convai offer sophisticated conversational AI specifically designed for game NPCs. These platforms provide emotion modeling, memory systems, and voice synthesis. However, they come with significant costs—subscription fees, API usage charges, and licensing requirements that make them inaccessible for students and indie developers.

Additionally, they often require substantial integration work to connect with existing game systems like quests and inventory.

Research institutions have also explored this space. MIT's Media Lab and Stanford's AI Lab have published papers on "believable agents" and "socially intelligent NPCs." These academic projects demonstrate impressive AI capabilities in controlled environments but rarely produce that game developers can actually use. The gap between research prototypes and production-ready tools remains wide.

Some indie developers have attempted custom solutions. Games like *Faade* (2005) featured NPCs that could understand typed natural language input, while more recent projects experiment with integrating ChatGPT into Unity games. However, these

implementations typically handle only dialogue without considering the broader context of NPC autonomy, quest management, or time-based behaviors.

The fundamental problem is that existing solutions are either too simplistic (pre-scripted dialogue), too expensive (commercial AI platforms), too narrow in scope (voice commands only), or too disconnected from actual gameplay systems (pure narrative AI). There's a lack of comprehensive, accessible systems that combine natural language understanding, autonomous NPC behaviors, and integration with core RPG mechanics.

With my interest in both AI technology and game development, I recognized an opportunity to create something different. My goal was to develop a complete NPC AI system that brings together multiple technologies: speech recognition for voice input, large language models for intelligent dialogue, time-based routine AI for autonomous behaviors, and seamless integration with quest and economy systems—all using free or low-cost tools accessible to students and indie developers.

This project draws from various successful implementations: the conversational capabilities of modern chatbots, the autonomous scheduling seen in simulation games like *The Sims*, the quest systems from traditional RPGs, and the voice interaction concepts pioneered by virtual assistants. By combining these proven technologies in a novel way and making them work together cohesively, the project aims to demonstrate that sophisticated NPC AI is achievable without enterprise-level budgets or proprietary platforms.

III. Project Purpose

The project aims to design and develop an advanced NPC AI interaction system for a video game using Unity, focusing on the following core features:

1. NPC AI and Interaction Systems:

- **Voice Recognition and Natural Language Processing:**

- **Speech-to-Text Integration:** Capture player voice input through microphone and convert to text in real-time.
- **Context-Aware Dialogue System:** NPCs understand player intent and respond intelligently based on current game context(quest, time, location)
- **AI-Powered Chatbot Backend:** Integration with Large Language Models (LLM) via Flask API server to generate natural, dynamic NPC responses.
- **Intent Detection System:** Automatic classification of player requests (quest inquiries, trading, general conversation) with fallback keyword detection.

- **Autonomous NPC Behavior System:**

- **Time-Based Activity Scheduling:** NPCs follow realistic daily

routines with 8+ distinct activities (sleeping, working, trading, socializing, main activity).

- **Dynamic Pathfinding:** A* algorithm implementation for NPC navigation with obstacle avoidance and map boundary detection.
- **State Machine Logic:** Comprehensive state management (Idle, Moving, Gathering, Trading, Resting).
- **Player Override System:** NPCs respond to player requests and temporarily pause their routines during conversations.
- **Quest Management Integration:**
 - **Database-Driven Quest System:** SQLite database storing quest data, objectives, progress, and NPC-quest relationships.
 - **Voice-Activated Quest Flow:** Players can accept, check status, and complete quests through natural.
 - **Dynamic Progress Tracking:** Automatic quest progress updates when players collect items or complete objectives.
 - **Multi-Objective Support:** Handle complex quests with multiple steps and conditional requirements.
- **Economy and Trading System:**
 - **Multi-Currency System:** Six-tier currency hierarchy (Obal, Varos, Sylv, Feron, Astryl, Aurum) with automatic conversion.
 - **Time-Based Trading:** NPCs operate shops during specific hours (e.g., 8:00 AM - 12:00 PM) and must be at designated market locations.
 - **Role-Based Inventory:** Shop inventory loaded from database based on NPC role (flower_merchant, hunter, blacksmith).
 - **Dynamic Pricing:** Quest rewards, shop prices, and currency transactions integrated with the dialogue system.
- **Time and Environment Management:**
 - **Day/Night Cycle System:** 24-hour in-game time with configurable real-time duration (e.g., 30 real minutes = 1 game day).
 - **Dynamic Lighting:** Five-phase lighting transitions (Morning, Noon, Afternoon, Evening, Night) with color and intensity changes.
 - **Seasonal System:** Four seasons (Spring, Summer, Autumn, Winter) affecting NPC behaviors and environment.
 - **Event Scheduling:** Time-triggered events for NPC activities and shop availability.
- **Backend Architecture:**
 - **RESTful API Server:** Python Flask server handling chatbot requests, player require, quest data, and shop inventory.
 - **Database Integration:** MySQL/SQLite for persistent storage of quests, items, NPCs, and player progress.

- **API Communication:** Unity-Python integration via UnityWebRequest for real-time data exchange.
- **Session Management:** Track conversation context and player state across multiple interactions.

2. Supporting Game Systems:

- **Basic Gameplay Mechanics:**
 - **Character Movement:** WASD controls with dash mechanic for exploration.
 - **Combat System:** Multiple weapons with unique attack patterns for enemy encounters.
 - **Inventory System:** Item collection and management integrated with quest objectives.
 - **Health and Energy Systems:** Resource management affecting player capabilities.
- **Visual and Audio:**
 - **UI System:** Quest panels, dialogue boxes, shop interfaces, and currency displays.
 - **Particle Effects:** Visual feedback for item collection, quest completion, and NPC interactions.
- **Technical Objectives:**
 - **Demonstrate AI Integration:** Show practical implementation of modern NLP technologies in game development.
 - **Autonomous Agent Design:** Create believable NPCs with independent decision-making and scheduling.
 - **System Interoperability:** Achieve seamless integration between dialogue, quests, economy, and time systems.
 - **Accessibility Focus:** Use free/open-source tools (Unity Personal, Python, SQLite) suitable for students and indie developers.

IV. Target and scope of research

1. Target

The primary focus of this study is the design and implementation of intelligent NPC AI systems using modern natural language processing and autonomous behavior algorithms. The research encompasses integrating speech recognition, large language models, time-based scheduling systems, and database-driven content management to create believable, interactive NPCs in a 2D RPG environment.

The intended audience for this project includes AI researchers, game programmers specializing in NPC systems, indie game developers, and students interested in practical applications of conversational AI in games. The project leverages Unity as the game

engine, Python Flask for backend AI services, and OpenAI API for natural language understanding, producing a comprehensive framework that demonstrates how modern AI technologies can be integrated into game development without requiring expensive commercial platforms.

2. Scope of Research

This project focuses on developing three core systems: voice-driven NPC interactions with context-aware dialogue, autonomous time-based NPC behaviors with pathfinding and daily routines, and seamless integration between dialogue systems and game mechanics such as quests and trading. The implementation includes a Python Flask REST API backend for AI processing, SQLite database for content management, and Unity frontend with speech recognition capabilities.

V. Research Content

The project focuses on researching and completing an AI-driven NPC system with the following content:

- **itch.io - Using Free 2D Assets Available Online**
 - Search for and utilize 2D assets: Characters, backgrounds, enemies, levels, and terrain.
- **Aseprite - Visual Asset Design**
 - Design 2D character sprites: Player character, NPCs with different roles (merchant, trader).
 - Create environmental objects: Flowers, trees, market stalls, camps, barrels, crates.
 - Design UI elements: Dialogue panels, quest indicators, currency icons, status markers.
 - Animate character movements: Idle, walking, gathering, trading animations.
 - Create particle effects for interactions and visual feedback.
- **Python Flask - AI Backend Development:**
 - Research and implement ChatBot API using OpenAI GPT models for natural language understanding.
 - Develop intent classification system for player requests (quest, trading, conversation).
 - Build REST API endpoints for chatbot communication and database queries.
 - Implement quest management API: accept quests, update progress, complete quests.
 - Create shop inventory system with role-based item loading.

- Design database schema using SQLite for quests, NPCs, items, and player progress.

- **Unity - Game Design and Development**

- Research existing NPC AI systems and voice interaction implementations in games.
- Define target platform as Windows PC and use C# as the primary programming language.
- Core Unity Concepts Applied:
 - **Scene:** Game world layout with camps, villages, and procedurally generated maps.
 - **GameObject:** NPCs, player character, interactive objects (flowers, market stalls).
 - **Component:** NPC scripts (NPCRoutineAI, NPCTrader, NPCQuestGiver), TimeManager, QuestManager.
 - **Script:** AI behavior logic, pathfinding algorithms, voice recognition integration, database communication.
 - **Prefab:** Reusable NPC templates, market stalls, flowers, UI panels.
 - **C#:** Object-oriented programming for all game systems including state machines, coroutines, and API requests.

- **Development Process:**

- Understand fundamental principles and mechanisms of AI-driven NPC systems in games.
- Identify necessary components for voice interaction, autonomous behaviors, and database integration.
- Design and implement the NPC AI system based on researched materials and reference projects.
- Analyze system requirements, design architecture, develop features, and conduct testing.
- Document implementation details and create guides for replication and extension.

PART 2: CONTENTS OF THE PROJECT

CHAPTER 1: PROJECT OVERVIEW

I. PROJECT DESCRIPTION

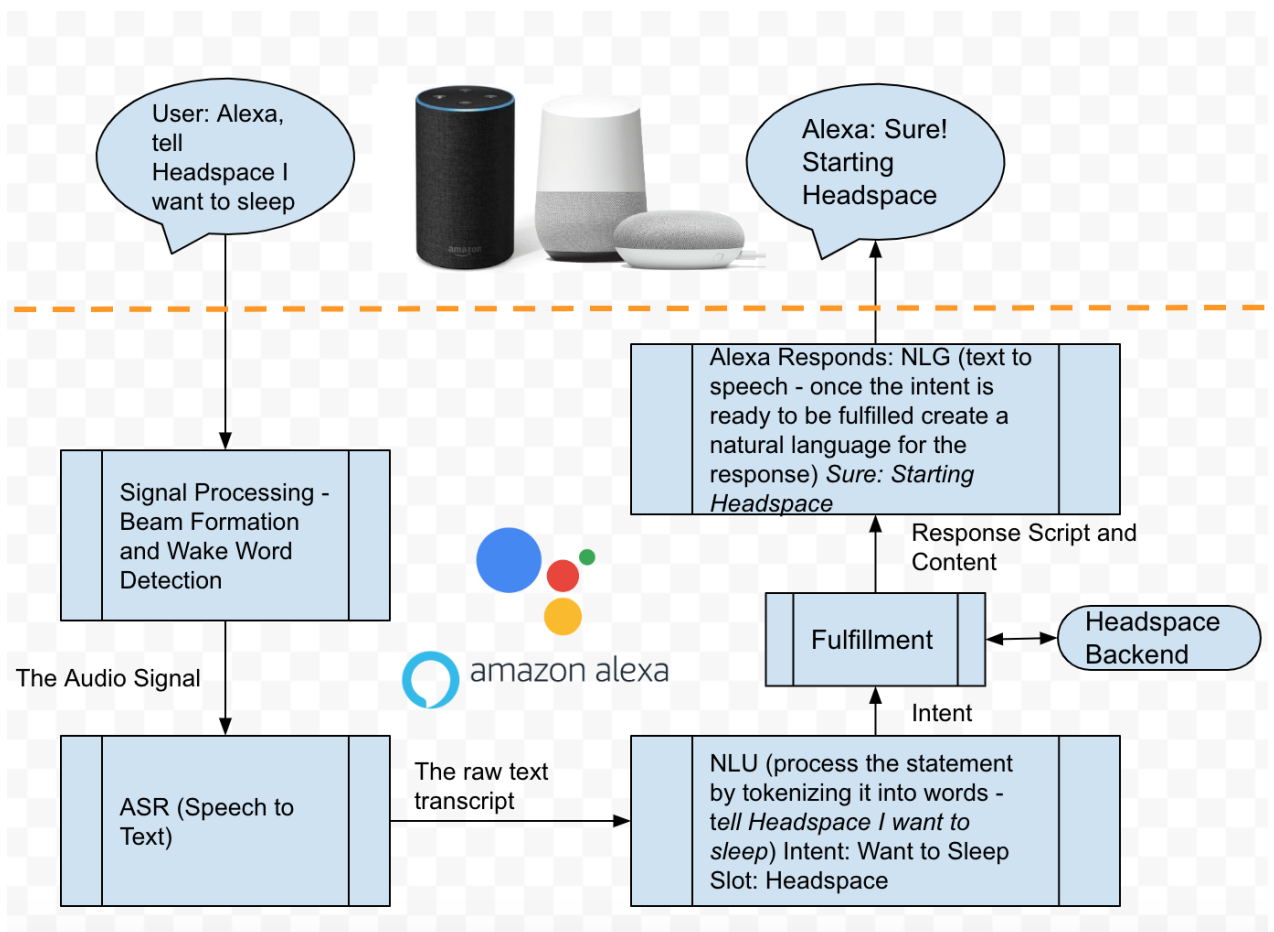
Game Concept:

The NPC AI interaction system represents a breakthrough in creating immersive and dynamic character experiences in RPG video games. Drawing inspiration from the sophisticated NPC routines seen in *The Elder Scrolls V: Skyrim* (Bethesda, 2011), where characters follow realistic daily schedules including sleeping, eating, and working, the system elevates this concept by integrating modern AI-powered natural language understanding. Additionally, the voice recognition mechanics are influenced by virtual assistant technologies such as Amazon Alexa and Google Assistant, enabling players to communicate naturally with

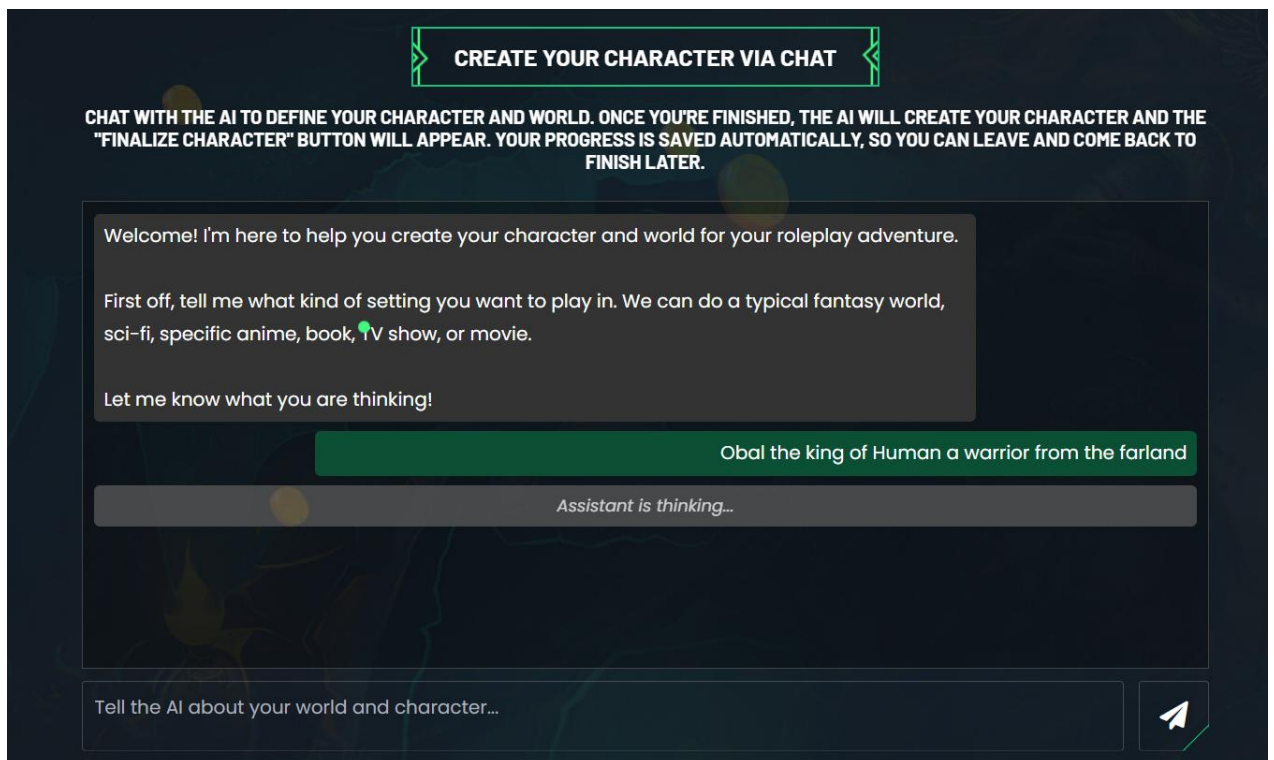
NPCs through speech rather than clicking pre-defined dialogue options. The conversational AI capabilities take inspiration from *AI Dungeon* (Latitude, 2019), which demonstrated that large language models could generate contextually relevant, dynamic narratives. By combining these proven concepts—autonomous NPC behaviors from simulation games, voice interaction from virtual assistants, and intelligent dialogue from conversational AI—this system creates a living game world where NPCs are not merely static quest-givers but autonomous agents with their own schedules, goals, and the ability to understand and respond to players in natural language. The result is a RPG environment where interactions feel organic, responsive, and truly driven by artificial intelligence rather than rigid scripts.



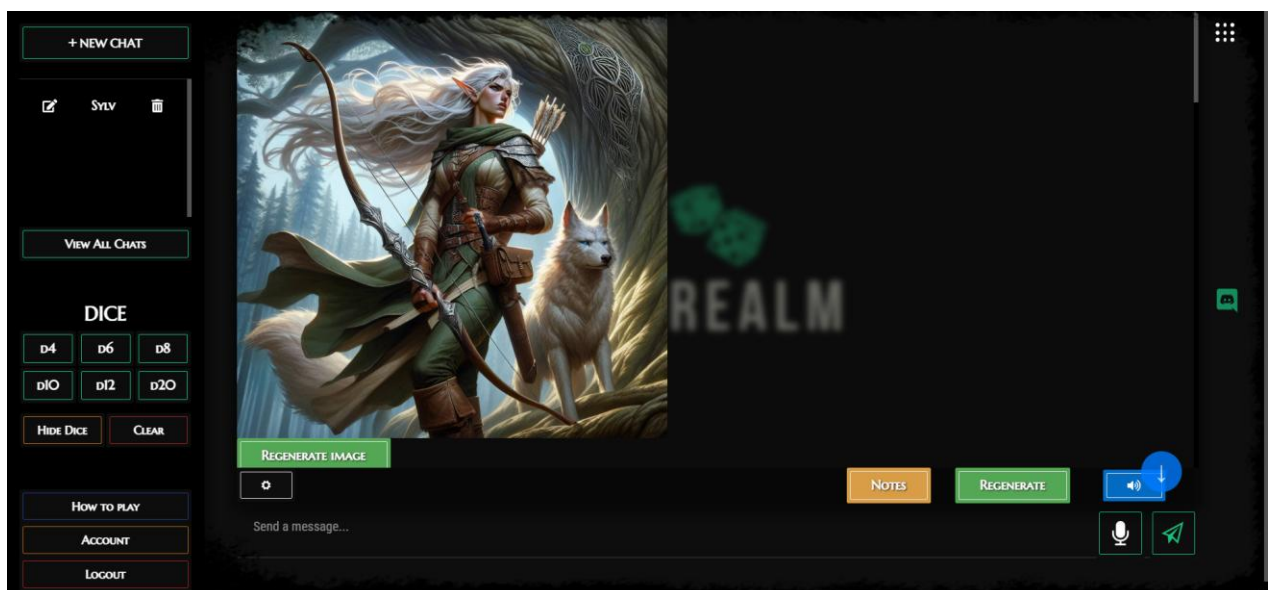
Picture 1.1 The Elder Scrolls V: Skyrim - NPC following autonomous daily schedule



Picture 1.2 Voice Assistant Technology – Architecture of natural language interaction



Picture 1.3. AI Dungeon - AI-powered contextual dialogue generation



Picture 1.4. AI Dungeon – Charactor created

Game Story:

In the ancient realm of **Eclyria**, a land shaped by towering forests, crystalline lakes, wandering spirits, and remnants of long-forgotten magic, countless races live upon the foundations left behind by legendary heroes of the past. Though the great war that once shook the world ended long ago, its echoes still linger in abandoned ruins, corrupted creatures, and scattered traces of ancient power. Between peaceful villages and dangerous wilds, Eclyria remains a world of mystery—beautiful yet unpredictable—where forgotten legends sleep beneath moss and stone, waiting for those brave enough to uncover them.

Among the people of Eclyria lives **Sylv**, a young elf who once dreamed only of adventure. Her life changed when her family was killed by lingering **shadowborn**, twisted remnants of an old darkness that continue to haunt the land despite the war's end. Left as the sole survivor of her household, Sylv chooses not to live in fear but to step beyond her village in search of purpose, strength, and answers. What begins as a personal journey—to understand why the shadowborn still roam and to forge her own path—slowly leads her toward a destiny far greater than she could ever imagine.

Game Content:

Unlike traditional scene-based progression, the game features a procedurally generated open world where Sylv explores a living, breathing environment. The gameplay is divided into interconnected areas within a dynamically created fantasy world:

The Starting Region: Sylv awakens in a procedurally generated wilderness featuring winding roads, flowing rivers, and scattered vegetation. The world layout changes with each playthrough, ensuring unique exploration experiences. The player's initial objective is to familiarize themselves with the environment, gather basic resources such as flowers and health orbs, and locate the first camp settlement where NPCs reside.

The Camp: At the heart of the world lies a central camp where NPCs live and work according to their daily routines. Here, Sylv encounters Snow a flower merchants. Players engage with NPCs through natural voice conversations rather than pre-scripted dialogue options. By speaking with NPCs, players can discover available quests, learn about the world's lore, and understand each character's daily schedule and needs.

Quest-Driven Exploration: Through voice interactions, NPCs assign quests that send players into the surrounding wilderness. For example, a flower merchant might request 10 red flowers, prompting the player to explore the world, locate flowers, and return for rewards. Quest objectives update automatically as players collect items, and completion is acknowledged through natural dialogue.

The Market District: During specific working time windows (8:00 AM - 12:00 PM), trader NPCs operate market stalls within the camp. Players can purchase equipment, trade resources, and sell gathered materials using the game's multi-tier currency system. The market comes alive during business hours as NPCs autonomously navigate to their designated stalls, creating a dynamic trading environment.

Dynamic World Cycles: The game world transitions through a 24-hour day-night cycle with changing lighting conditions and weather effects. NPCs adapt their behaviors based on the time of day—sleeping at night, working during their scheduled hours, and socializing in the evenings. Players must learn these patterns to optimize interactions and quest completion timing.

Gameplay:

The player controls Sylv freely throughout the procedurally generated open world, with objectives centered around NPC interaction, resource gathering, and environmental exploration. The player can click the left mouse button to attack enemies, break wooden crates, and cut through bushes and trees to clear paths or claim items.

The player can dash to avoid attacks and navigate quickly through the terrain, collect resources such as flowers, currency coins, and experience orbs when defeating creatures or destroying environmental objects.

Players can switch between different weapons—such as a magical staff for ranged attacks or a sword for close combat—to adapt to various situations. Health regenerates naturally over time or can be restored immediately by collecting health orbs scattered throughout the world.

Players discover quests and gather information by speaking directly to NPCs using their microphone. Instead of clicking through dialogue menus, players ask questions naturally—"What do you need help with?" or "Where can I find flowers?"—and NPCs respond conversationally based on their current situation. Quest acceptance happens through agreeing verbally ("Sure, I can help!"), and completion is confirmed by telling the NPC you've finished ("I collected all the flowers you needed"). This creates immersive interactions where conversations flow naturally rather than feeling scripted.

NPCs are not static quest-givers waiting in fixed locations. Players must learn each NPC's daily routine to find them at the right time and place. A flower merchant might be gathering resources in the forest at 2:00 PM but sleeping at home by 11:00 PM. Traders only operate their market stalls during morning hours (8:00 AM - 12:00 PM), so players arriving in the afternoon find closed shops.

Core Functional Components:

- **Weapon Selection:** Allows players to change between different weapons for combat.
- **Dash Mechanism:** Enables quick movement to avoid enemy attacks or obstacles.
- **Combat System:** Features attack buttons and abilities to fight monsters, break objects, and progress through levels.
- **Health System:** Track the character's health, with healing mechanisms through drops or regeneration.
- **Stamina System:** Track the character's stamina, regeneration after 5 seconds if players losing it.
- **Enemy Behavior:** AI-driven behaviors for monsters and bosses, with unique patterns and challenges for each encounter.
- **Voice Recognition System:** Captures player speech through microphone and converts to text in real-time using HuggingFace API with OpenAI Whisper (whisper-tiny model) for natural conversation input with NPCs.
- **AI-Powered Chatbot Integration:** Processes natural language through Llama 3.2 3B Instruct model running locally via LM Studio to understand player intent, generate contextual responses, and determine appropriate actions. Uses Sentence Transformers (all-MiniLM-L6-v2) for semantic intent detection with cosine similarity matching.
- **Context-Aware Dialogue System:** Provides NPCs with real-time game information (quest status, current time, NPCs activity) enabling intelligent responses that adapt to circumstances. System prompt dynamically includes quest context and NPC status to generate relevant natural language responses.
- **NPC Routine AI System:** Manages autonomous NPCs behaviors with 24-hour daily schedules including sleeping (23:00-6:00), working (flower hunting 14:00-16:00, market trading 8:00-12:00), socializing, and traveling. NPCs navigate independently using **A pathfinding algorithm** with obstacle detection and map boundary clamping.
- **Quest Management System:** MySQL database-driven quest tracking with Flask REST API backend for data synchronization. Features automatic quest discovery through NPC conversations, real-time progress updates as players collect items via API calls to /update_progress endpoint, and voice-driven acceptance and completion workflows.
- **Time Management System:** 24-hour day-night cycle controlling NPC schedules, market operating hours, and dynamic lighting transitions. Game time progresses at configurable rates (default: 30 real minutes = 1 game day).
- **Dynamic NPC Pathfinding:** A* algorithm implementation enabling NPCs to navigate the procedurally generated world with tile-based grid system, avoid obstacles using Physics2D.OverlapCircle on "Obstacle" and "Water" layers, clamp movement within map bounds (0 to width/height), and reach destinations autonomously during scheduled activities.

- **Player Request Override System:** Allows NPCs to temporarily set `isPaused` flag and halt scheduled routine coroutines when players initiate voice interactions, execute player-commanded actions through chatbot action system (`GATHER_FLOWER`, `QUEST_DIALOGUE`, etc.), then resume normal activities by restarting coroutines after dialogue ends.
- **Economy and Currency System:** Multi-tier currency management with six denominations (`OBAL` base unit, `VAROS` $\times 10$, `SYLV` $\times 100$, `FERON` $\times 1000$, `AURUM` $\times 10,000$, `ASTRYL` wizard currency tracked separately) with automatic conversion calculations, quest reward distribution via `EconomyManagement` singleton, and NPCs trading integration during market hours using `DatabaseShopLoader` for role-based inventory.
- **Database Integration System:** MySQL backend (host: localhost, database: `GameRPG2d`) with Flask REST API server (Python) managing quest definitions, NPC data, item catalogs, shop inventories, and player progress. Real-time synchronization between Unity client and server via `UnityWebRequest` POST/GET requests to endpoints including `/quests`, `/quest_objectives`, `/player_quests`, `/quest_progress`, `/update_progress`, `/items`, `/inventory`, and `/shop_items`
- **Procedural World Generation:** Algorithm-based map creation using Perlin noise and recursive branching for road networks. Generates unique layouts each playthrough with guaranteed 3-tile-wide main road, configurable branch depth (max 2 levels), connected river systems with meandering controlled by noise (amplitude 6f, frequency 0.03f), camp settlement with 18×14 tile footprint positioned using intelligent spacing algorithms (minimum 3 tiles from roads, 2 tiles padding), and resource distribution ensuring varied exploration experiences.
- **Resource Spawning System:** Dynamically places collectible items (flowers with `FlowerObject` wrapper, currency coins using `PickUpSpawner` with tier-based drop rates, health orbs, experience orbs with 6-tier system) throughout the world. Respawn mechanics tied to game time for renewable resources and `PickUpSpawner.dropValue` calculations for currency conversion based on configurable thresholds (`allowFeronDrop` if value ≥ 1000 , `allowSylvDrop` if value ≥ 100).

Non-Core Functional Components:

- **User Interface:**
 - Simple, intuitive, and easy-to-understand design.
 - Voice interaction panel with microphone status indicator and real-time speech-to-text display.
 - Quest management UI: quest list panel, detail panel showing objectives with progress bars, multi-tier reward display (`AURUM`/`FERON`/`ASTRYL`/`SYLV`/`VAROS`/`OBAL` coin icons, EXP).
 - Inventory system: grid-based slots item management using `UIInventoryController`.
 - Shop interface: item display panels, shopping cart system, price calculator with multi-currency breakdown, payment handler with audio feedback.
 - NPC dialogue panel with character portraits and subtitle synchronization

season indicator displaying 24-hour clock and seasonal icons.

- Simple, intuitive design using TextMeshPro for text rendering.
- Responsive layout supporting minimum resolution 1280×720.
- Fantasy theme with warm color schemes and smooth UI transitionsSuggested.

● **Audio Systems:**

- NPC Text-to-Speech: Microsoft Edge TTS (edge-tts library) with en-US-JennyNeural voice generating MP3 audio files.
- Player sounds: footsteps (interval-based), dash effects, weapon sounds (sword swing, staff cast, bow shoot)
- Environmental audio: destructible objects with tag-based sounds, item pickup confirmation.
- UI feedback: coin collection sounds, payment success/fail audio, shopping bag interactions.

● **Backend Infrastructure:**

- MySQL Server (localhost, database: GameRPG2d) for quest, NPC, item, and inventory data
- LM Studio inference server (port 1234) running Llama 3.2 3B Instruct model for NPC dialogue
- Flask REST API servers: ChatBox.py (AI chatbot + TTS), database.py (quest/inventory endpoints)
- HuggingFace API integration for Whisper speech recognition (requires API token)

● **System Requirements:**

- Operating System: Windows 7 or later (64-bit)
- Processor: Intel Core i3 or AMD equivalent
- RAM: 8GB minimum (12GB recommended for LM Studio AI model)
- Storage: 500MB free space
- Minimum Resolution: 1280x720
- Graphics Card: Intel HD Graphics 4000, NVIDIA GeForce GTX 750 Ti, or AMD Radeon RX 550.
- Microphone: Required for voice input to NPC system.
- Internet Connection: Required for HuggingFace API and Edge TTS services.

II. SOLUTION FOR THE PROJECT

To address the NPC AI interaction challenges, I have chosen a practical and systematic approach that combines existing resources, AI-generated assets, and custom AI integration to create an innovative and technically feasible solution.

The project utilizes a hybrid approach for 2D asset creation:

Itch.io and Unity Asset Store will be applied for:

- Providing foundational 2D sprite assets for environmental objects (trees, flowers, buildings), UI components, and tilemap resources.
- Acquiring pre-made scripts and prefabs to accelerate non-AI development tasks.

AI-Generated Asset Creation:

- ChatGPT is used to generate initial 2D pixel art concepts and sprite designs for NPCs, player characters, and unique game objects.
- Aseprite (professional pixel art editor) is employed to refine, animate, and optimize the AI-generated sprites, ensuring consistent art style and proper sprite sheet formatting for Unity integration.
- Shaker AI is utilized for generating promotional images and game menu backgrounds. Creating conceptual art references for scene design

Unity Engine serves as the core development platform with the following advantages:

- Supported by a large community of game developers providing extensive NPC AI implementation examples and troubleshooting resources.
- Real-time preview of NPC behaviors, pathfinding, and dialogue systems during development, enabling rapid iteration.
- Component-based architecture ideal for modular NPC AI systems (NPCRoutineAI, ChatbotClient, QuestManager as separate components).
- Lightweight software capable of building games for various configurations while supporting complex AI backend communication.
- Seamless integration with C# scripting for implementing sophisticated NPC logic including state machines, A* pathfinding, and asynchronous API calls.

The NPC AI system requires specialized backend technologies:

- Python Flask servers provide REST API endpoints for AI dialogue processing (ChatBox.py) and database operations (database.py).
- MySQL Database stores quest definitions, NPC data, shop inventories, and player progress with relational schema.

- LM Studio runs Llama 3.2 3B Instruct (GGUF) language model locally for generating contextual NPC dialogue without external API costs.
- HuggingFace API with OpenAI Whisper model for accurate speech-to-text conversion from player microphone input.
- Microsoft Edge TTS (edge-tts library) synthesizes natural NPC voices (en-US-JennyNeural) with customizable rate and pitch.
- Sentence Transformers (all-MiniLM-L6-v2) performs semantic intent detection using cosine similarity to understand player voice commands.

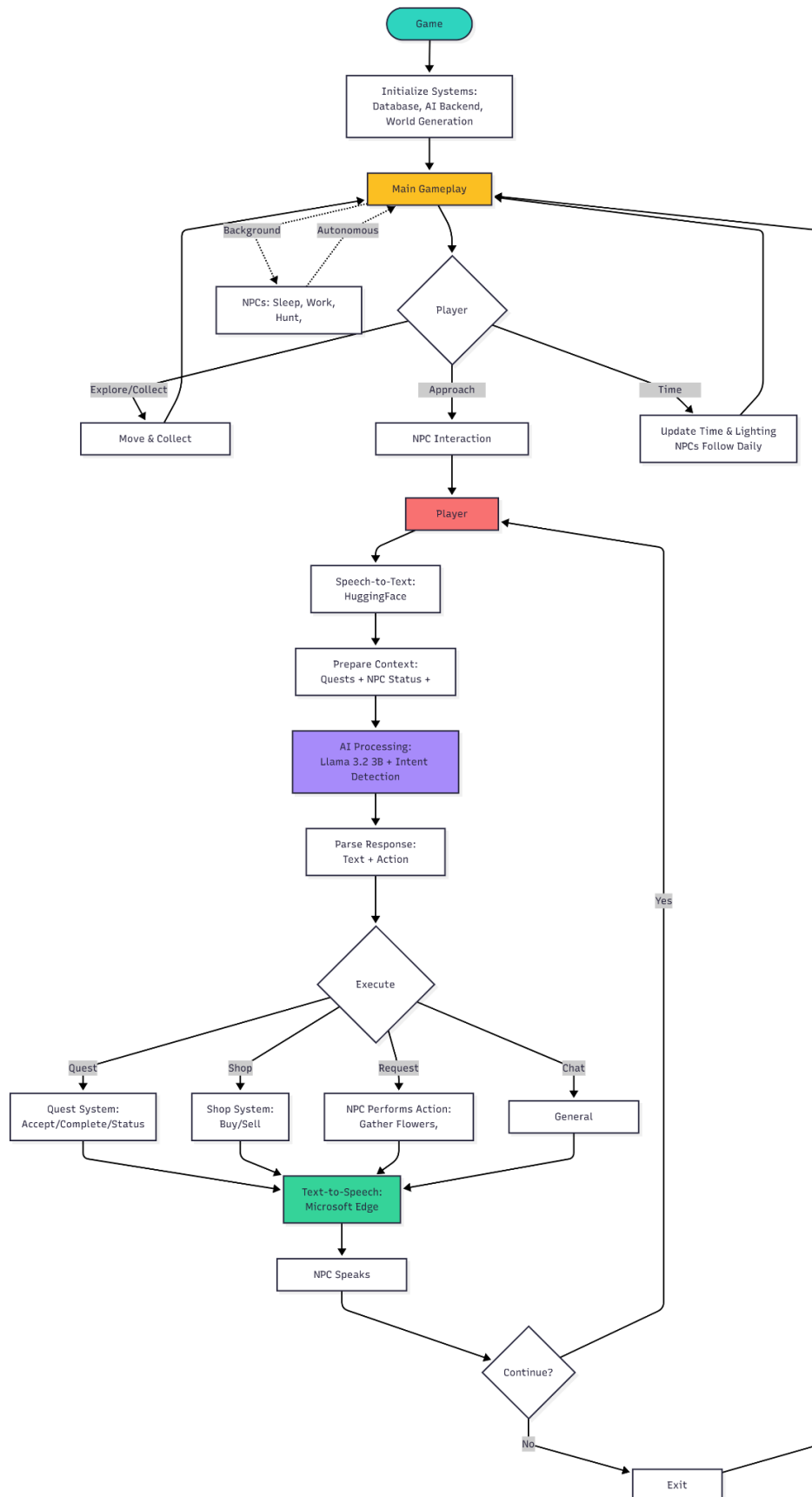
Microsoft Visual Studio is used to write C# scripts implementing NPC AI functionalities. C# is an object-oriented programming language combining the best features of languages such as C++ and Java. It is a modern, type-safe language highly compatible with Unity, providing:

- Strong integration with Unity's API for NPC control, coroutine-based behavior scheduling, and UI management
- UnityWebRequest library for HTTP communication with Flask backend servers
- Asynchronous programming support essential for non-blocking API calls and NPC routine management

Python 3.8+ is used for backend AI services with libraries: Flask, flask-cors, mysql-connector-python, sentence-transformers, edge-tts, and requests.

By combining these asset creation workflows, AI infrastructure, and development tools, the result is the 2D game "Shadows of Eclryia" featuring intelligent NPCs with voice-driven interactions, autonomous daily routines, and context-aware dialogue systems.

Game flowchart:

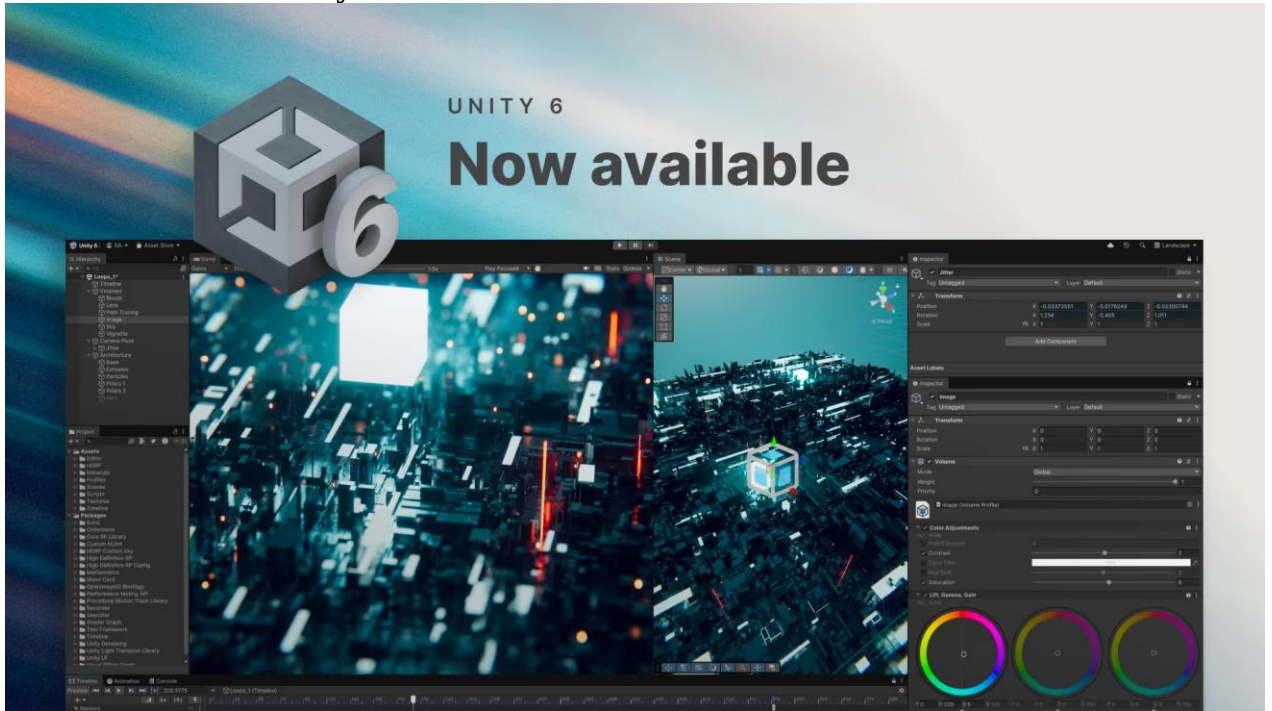


Picture 1.5. Game Flowchart

CHAPTER 2: DESIGN AND PROGRAMMING OF GAME FUNCTIONS

I. THEORETICAL BASIS

a. Unity



Picture 2.1. Unity 6

Unity is a software that allows direct game development on computers in real-time, enabling game developers to design graphics, maps, and characters in an intuitive environment. Developed by Unity Technologies, Unity offers the capability to develop games across various platforms and supports the transition between 2D and 3D game spaces. Unity is widely popular in the game development community, from individuals and small teams to large game studios, as it does not require in-depth knowledge of game programming.

In the early stages, Unity primarily focused on non-professional game developers with simple capabilities. Over time, after numerous improvements and the integration of many powerful technological components, Unity has become capable of developing a series of games. Today, although the community largely consists of individuals and small groups, Unity continues to grow rapidly.

What is a Game Engine?

A game engine is a software that supports the development of the environment in which a game operates. Game developers use a game engine to build games for various platforms such as consoles, PCs, and mobile devices.

The key features of a game engine include:

- **Rendering:** Creating images in 2D or 3D graphics.
- **Physics engine:** Calculating and handling collisions.
- **Game audio.**
- **Human-machine interaction handling.**
- **Animation,** creating movement for objects.
- **Artificial intelligence.**
- **Data management.**
- **Multithreading.**
- **Network connections and online interaction.**
- **Video playback in-game.**

In summary, a game engine supports the construction of the essential components for a complete game environment, from creating basic objects to the interaction between those objects through advanced calculations and processing.

Game development can be carried out on different game engines. However, Unity remains the optimal choice and is widely recognized as one of the best game engines currently available.

Advantages of Unity:

- **Ease of Use:** Unity provides a user-friendly interface that is easy to use, allowing non-professional developers to quickly access and develop 2D and 3D games. Additionally, Unity offers features that help optimize the development process, such as drag-and-drop tools and an integrated IDE.
- **Cross-Platform Support:** Unity allows games to be exported to various platforms, from mobile devices and computers to web and console systems. This helps developers save time and effort when releasing games on multiple devices.
- **Flexibility and Extensibility:** Unity supports multiple programming languages, including C#, JavaScript, and Boo, allowing developers to choose the language best suited for their project. Furthermore, Unity provides a rich asset store, making it easy for developers to incorporate objects, materials, audio, effects, and scripts into their games.
- **Cost-Effective:** Unity offers a free version for individuals and organizations with revenue below \$100,000 per year. Full features of Unity can be used without paying until the developer's revenue reaches a certain threshold.

Overall, while Unity may not be the "most powerful" game engine, it is truly one of the best cross-platform game engines available today.

b. Basic Components in Unity2D

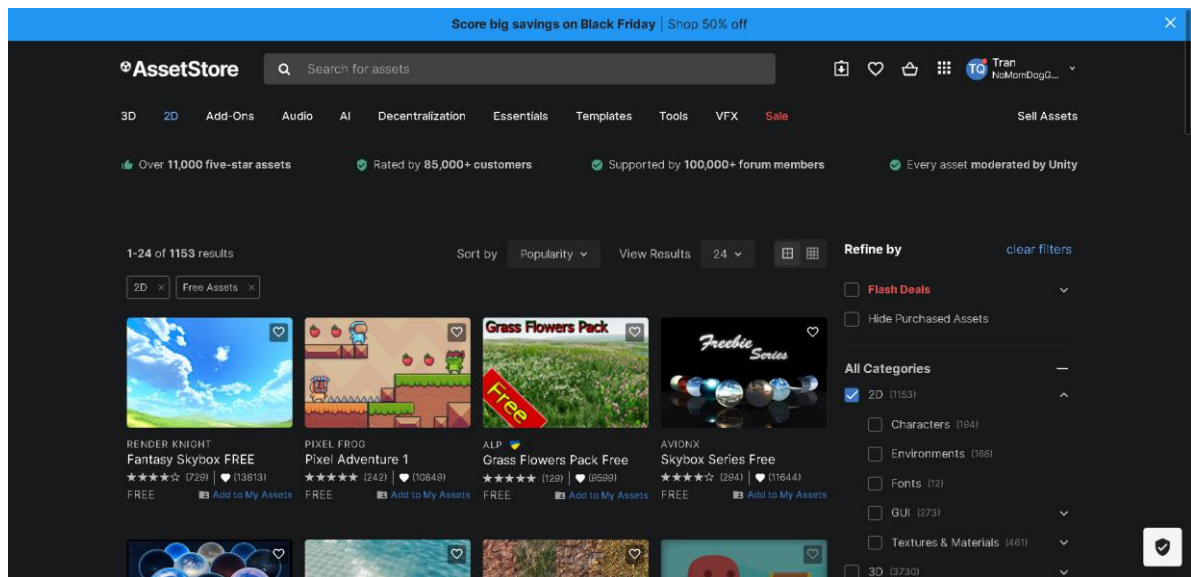
The six core components that make up a game project in Unity2D include:

- Asset
- Scene
- Game Object
- Components
- Script
- Prefab

ASSET:

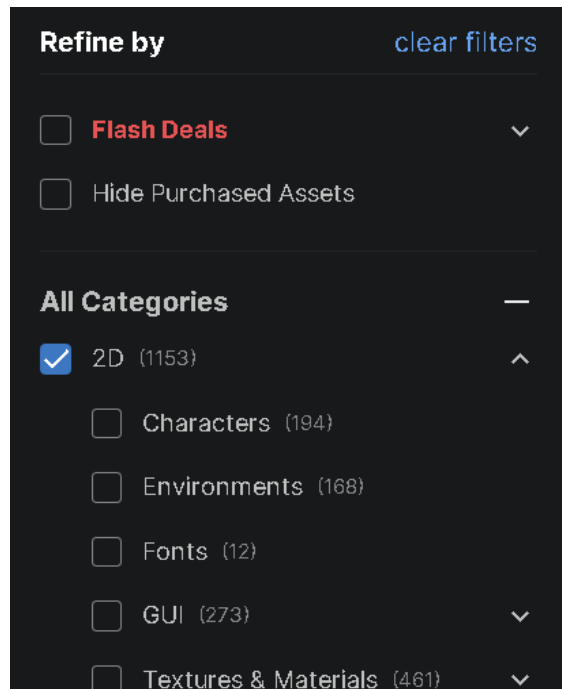
An asset is a resource package that provides the materials needed to build a complete project. These materials can include 2D models, textures, audio, effects, or even an entire completed project.

Assets themselves are created by game developers and can be downloaded for free or purchased from the Unity Asset Store. This is one of Unity's very useful features, as these assets help minimize the time spent on game design and programming.



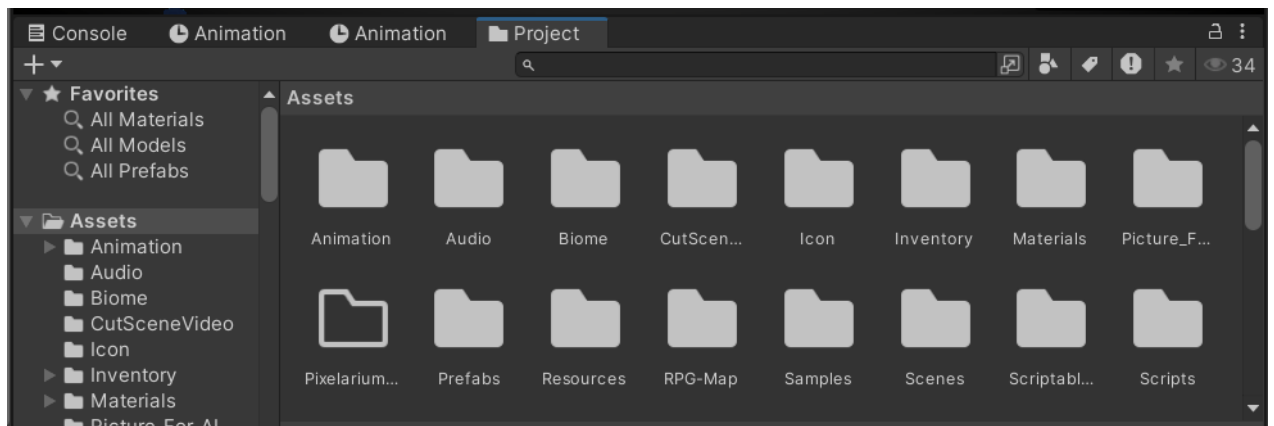
Picture 2.2. Unity Asset Store in Unity

The Asset Store also provides filters to help users easily select and search for the right materials for their project, based on price or free options.



Picture 2.3. Unity Asset Store filters

Assets can be directly imported into a project by downloading them through the Asset Store or using previously downloaded offline packages by selecting Asset > Import New Asset from the menu bar or by dragging and dropping the file directly into the Project window.



Picture 2.4. Assets in Project folder

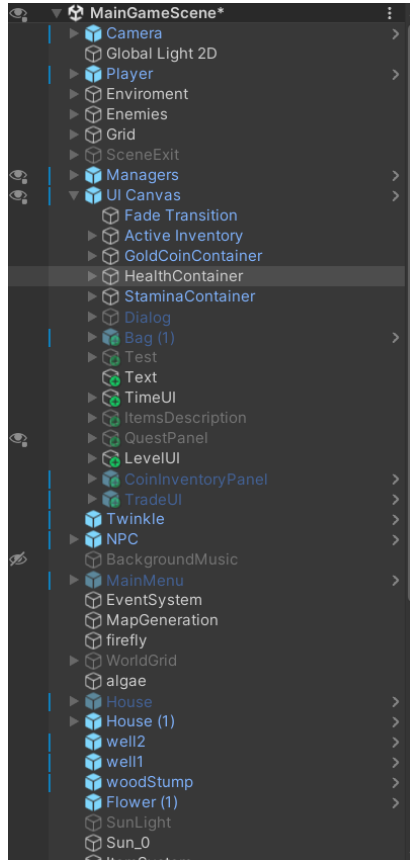
SCENE:

A scene is a game segment of the project, which can be an environment or a game menu. Scenes make it easier to manage the layout of a game. If your game has multiple levels, each scene will contain different environments, objects, and characteristics.



Picture 2.5. Scene component in Unity

The components of a scene in a project include the **Hierarchy** tab and the **Scene** view.

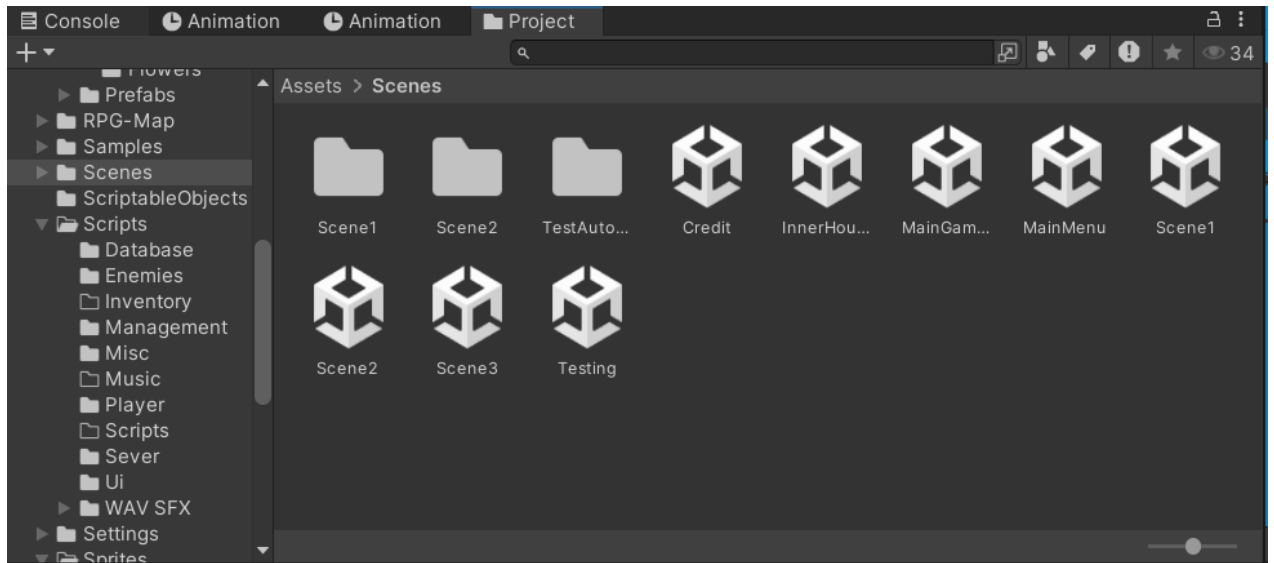


Picture 2.6. Hierarchy

The **Hierarchy** window displays all GameObjects in the active scene as a tree structure. It enables developers to organize objects by creating parent-child relationships, where child objects inherit transformations from their parents. This hierarchical nesting simplifies scene management, allowing related objects to be grouped together and moved as a unit. Developers can search, toggle visibility, and lock objects to prevent accidental changes during editing.

The **Scene View** is the primary workspace for visually constructing game environments. Developers can position, rotate, and scale GameObjects in 2D or 3D space using intuitive mouse and keyboard controls. The view displays visual aids such as lights, cameras, colliders, and gizmos to help understand spatial relationships. Multiple viewing modes including wireframe, shaded, and textured views allow developers to focus on specific aspects of scene design. Direct manipulation tools enable precise arrangement of game elements, providing immediate visual feedback without entering Play mode.

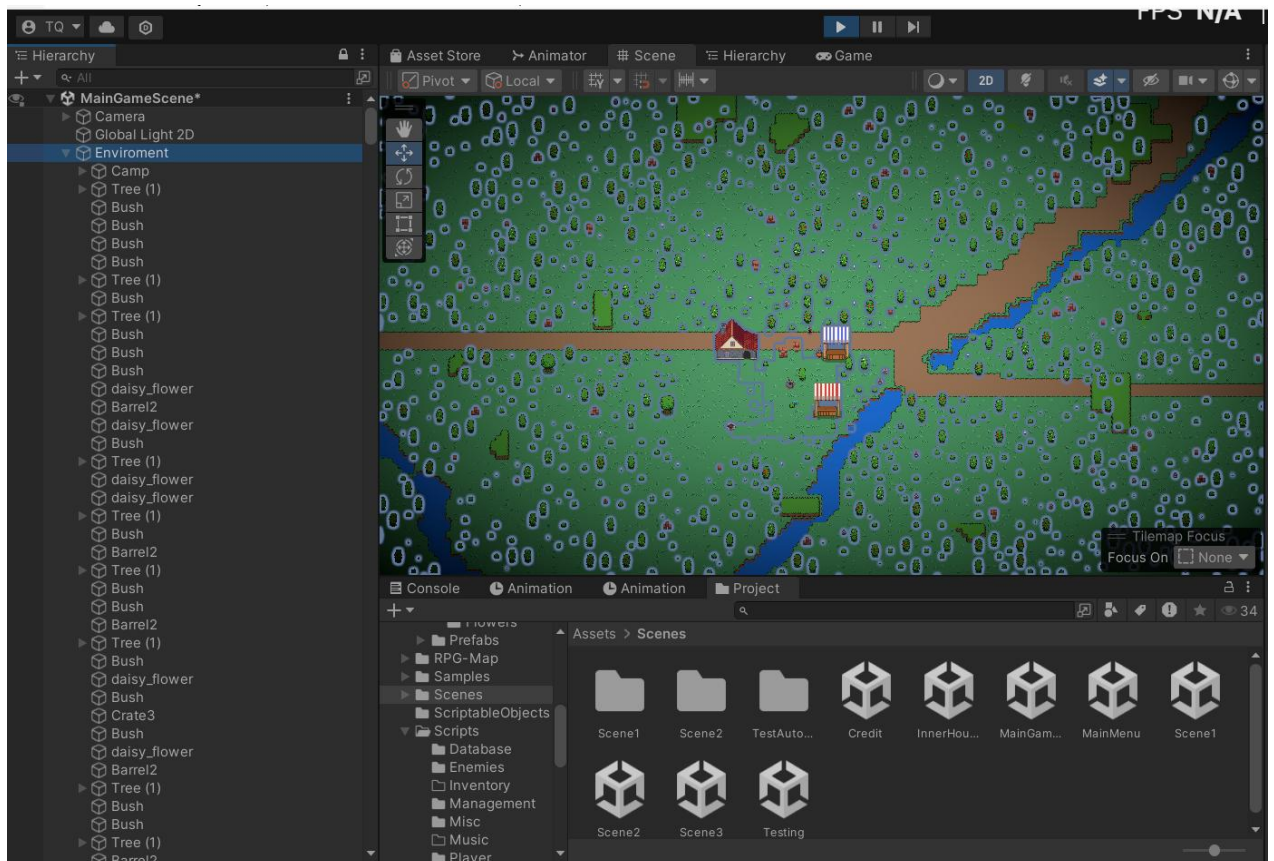
Scenes can be easily managed in the **Assets** section of the project.



Picture 2.7. Scenes in Asset folder of the Project

GAME OBJECT:

In Unity, assets added to a scene are considered Game Objects. These Game Objects added to the scene will be listed in a hierarchical format under the Hierarchy tab.



Picture 2.8. Game Object in Scene

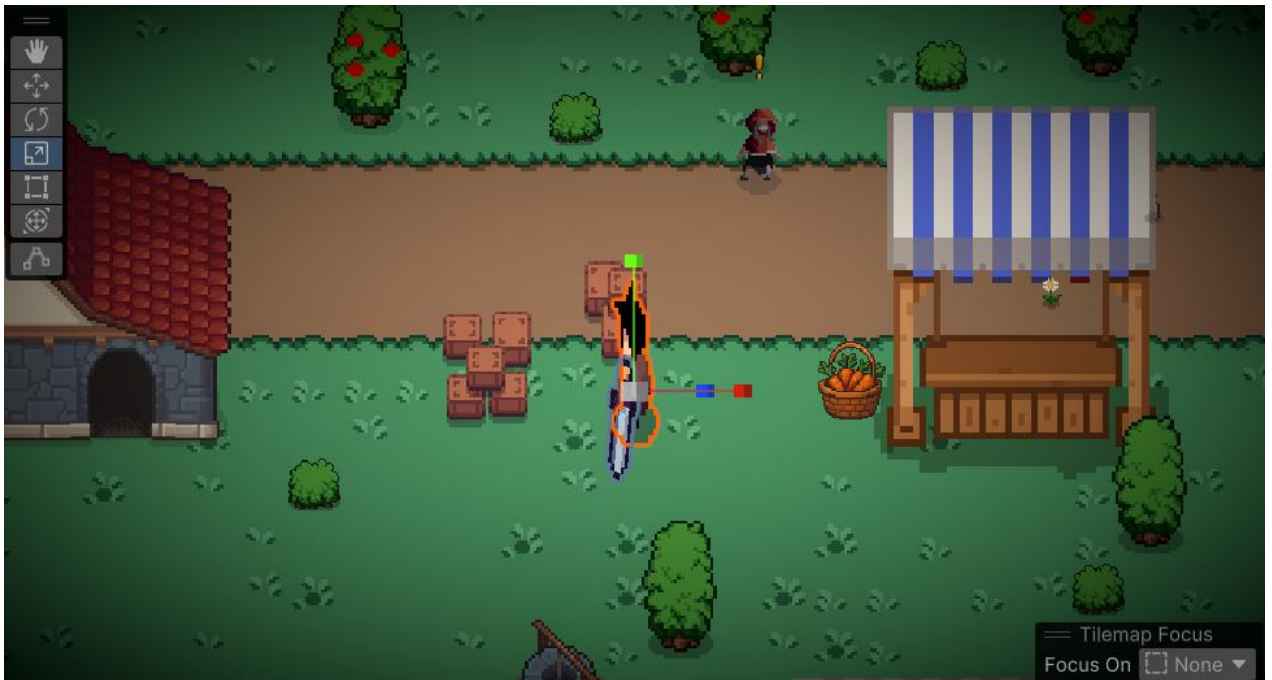
Within the scene, you can edit Game Objects using basic tools such as move, rotate, and scale. Quick access shortcuts for these tools, in order, are **W**, **E**, and **R**.



Picture 2.9. Move tool in Scene (key W)



Picture 2.10. Rotate tool in Scene (key E)

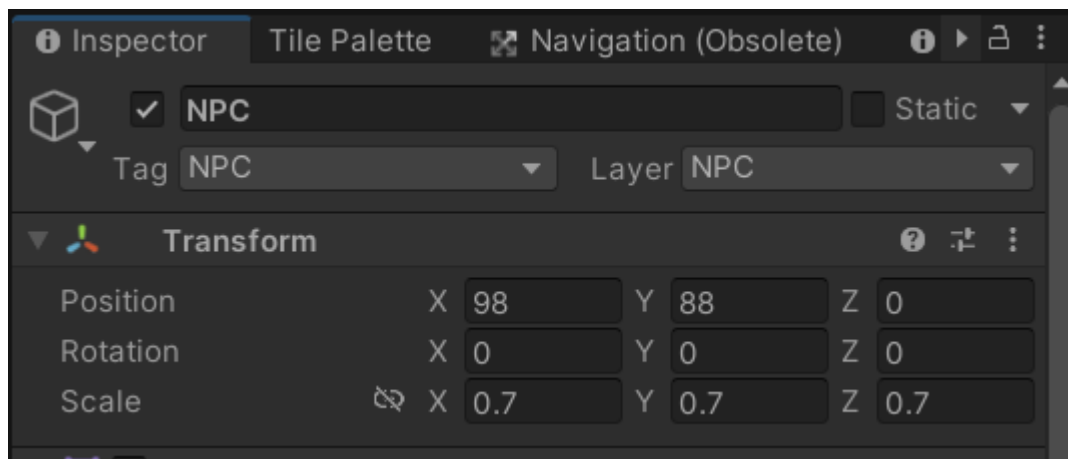


Picture 2.11. Scale tool in Scene (key R)

Additionally, instead of using the above tools by dragging with the mouse, you can adjust them using the parameters in the **Transform** section of the **Inspector** tab. Here:

- **Position** adjusts the object's position.
- **Rotation** allows you to rotate the object.
- **Scale** adjusts the object's size.

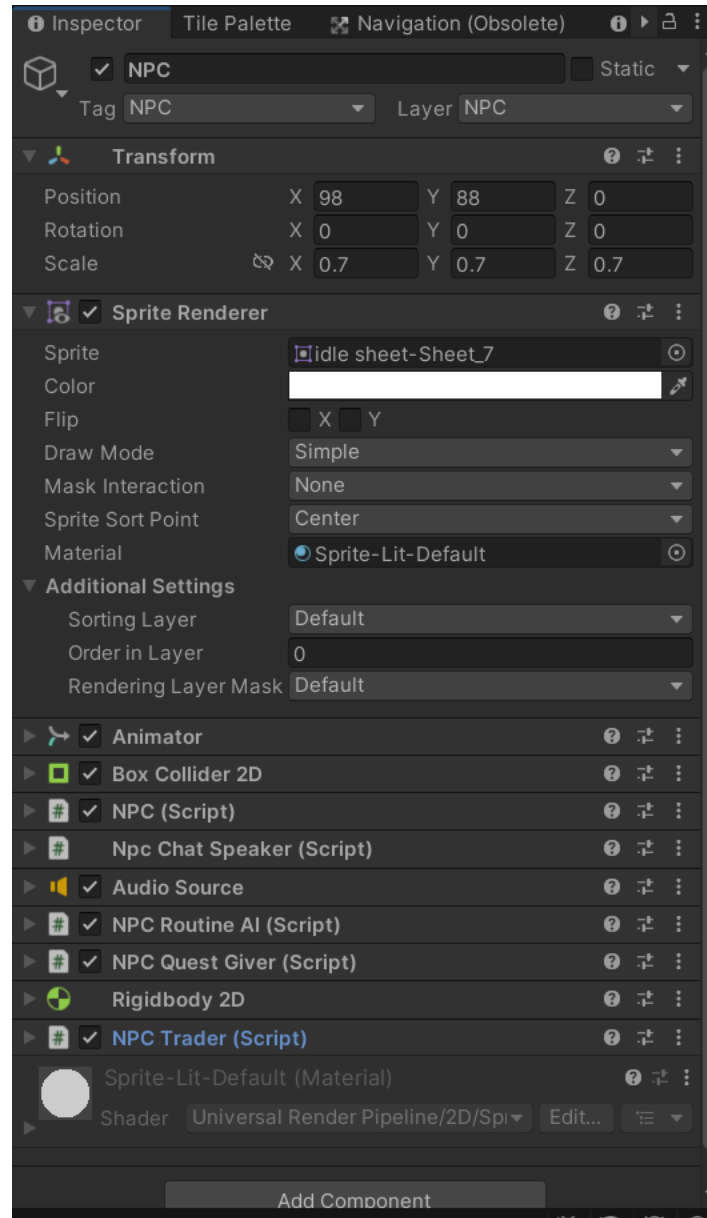
All parameters can be modified along the three axes: **X**, **Y**, and **Z**.



Picture 2.12. Transform field

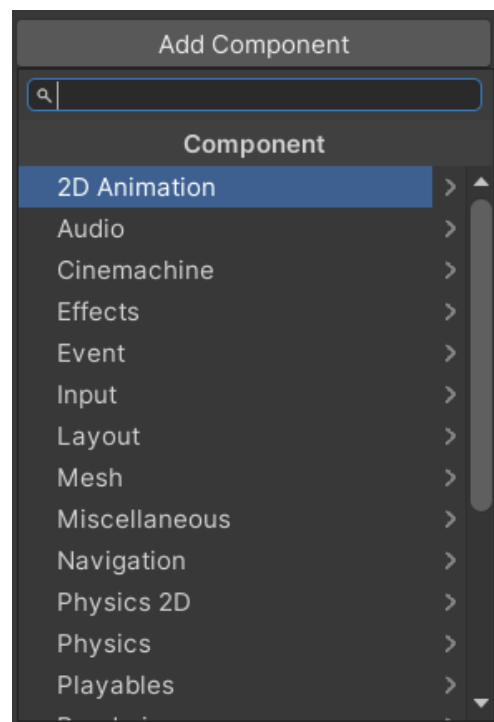
COMPONENTS

Components are elements that add functionality to Game Objects. Each component has a distinct function that defines the characteristics of a Game Object. These components can either be built-in or custom-created by developers using scripts.



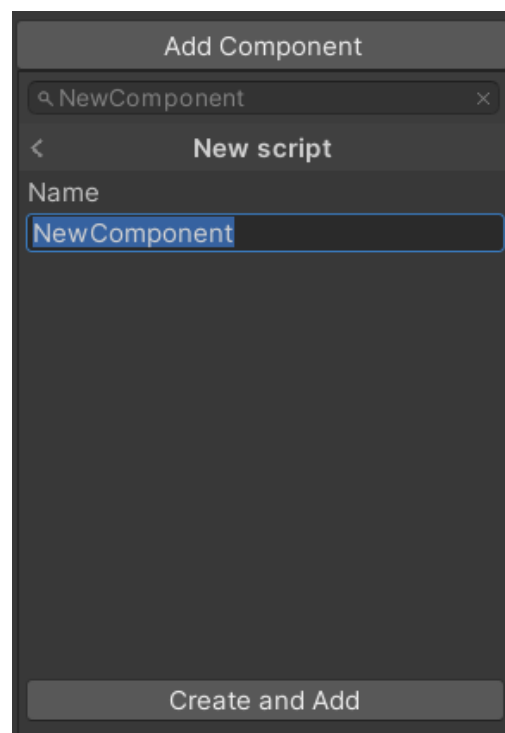
Picture 2.13.Components of a Game Object in Unity

To add a built-in component, select **Add Component** and type the name of the desired component in the search box.



Picture 2.14. Add Component for Game Object

To create a new component, simply type the name of the desired component and select **Create and Add**.

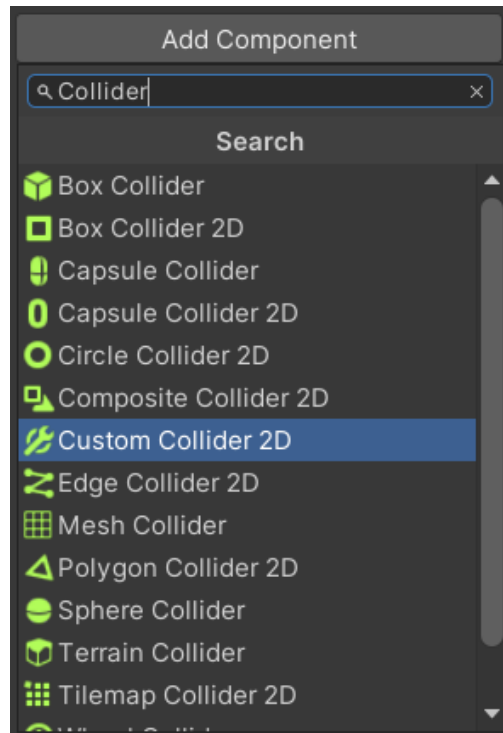


Picture 2.15. Create new Component

Some important commonly used Components in game development include:

- **Collider:**

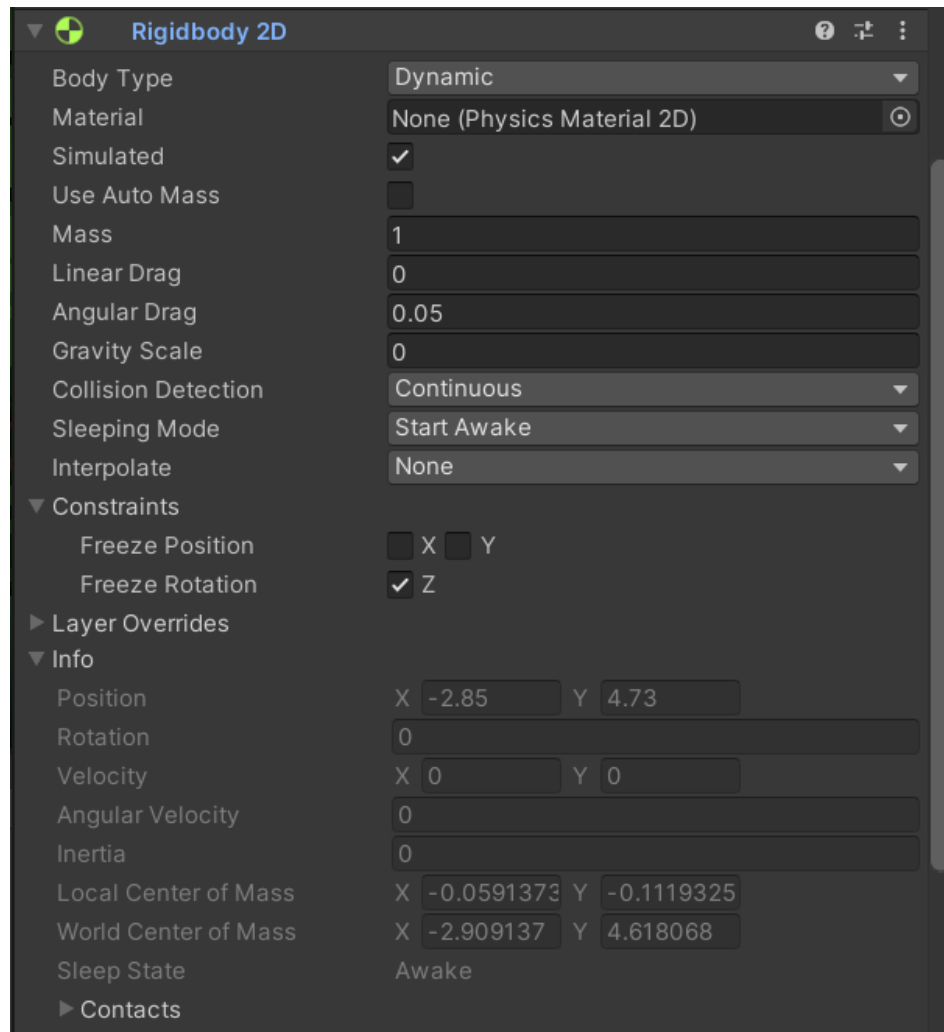
- A property used to handle collisions in the game. Colliders are often used to detect events when objects collide in the game.



Picture 2.16. Collider shape

- Common types of Colliders in games include:
 - **Box Collider:** In the shape of a box.
 - **Sphere Collider:** In the shape of a sphere.
 - **Capsule Collider:** In the shape of a capsule.
 - **Mesh Collider:** A Collider with an irregular shape, depending on the shape of the object it is attached to.

- **Rigidbody:**

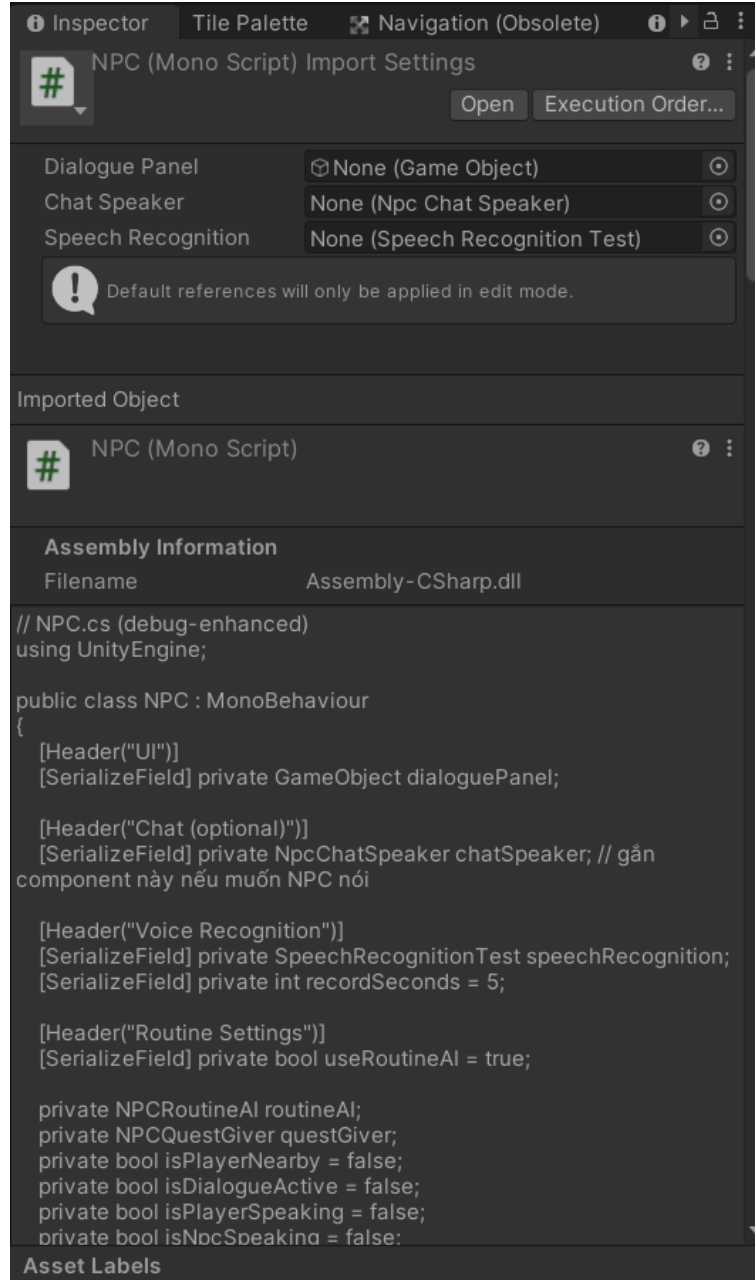


Picture 2.17. Rigidbody2D of Game Object

- One of the most prominent features that adds realistic physics and gravity simulation to the game, enhancing the realism of Unity-based games.

SCRIPT:

Scripts are components in Unity, often programmed to create interactions and control between the player and Game Objects, or among Game Objects themselves. They are also used to trigger events in the game as desired by the developer. Scripts are one of the core elements of game development.

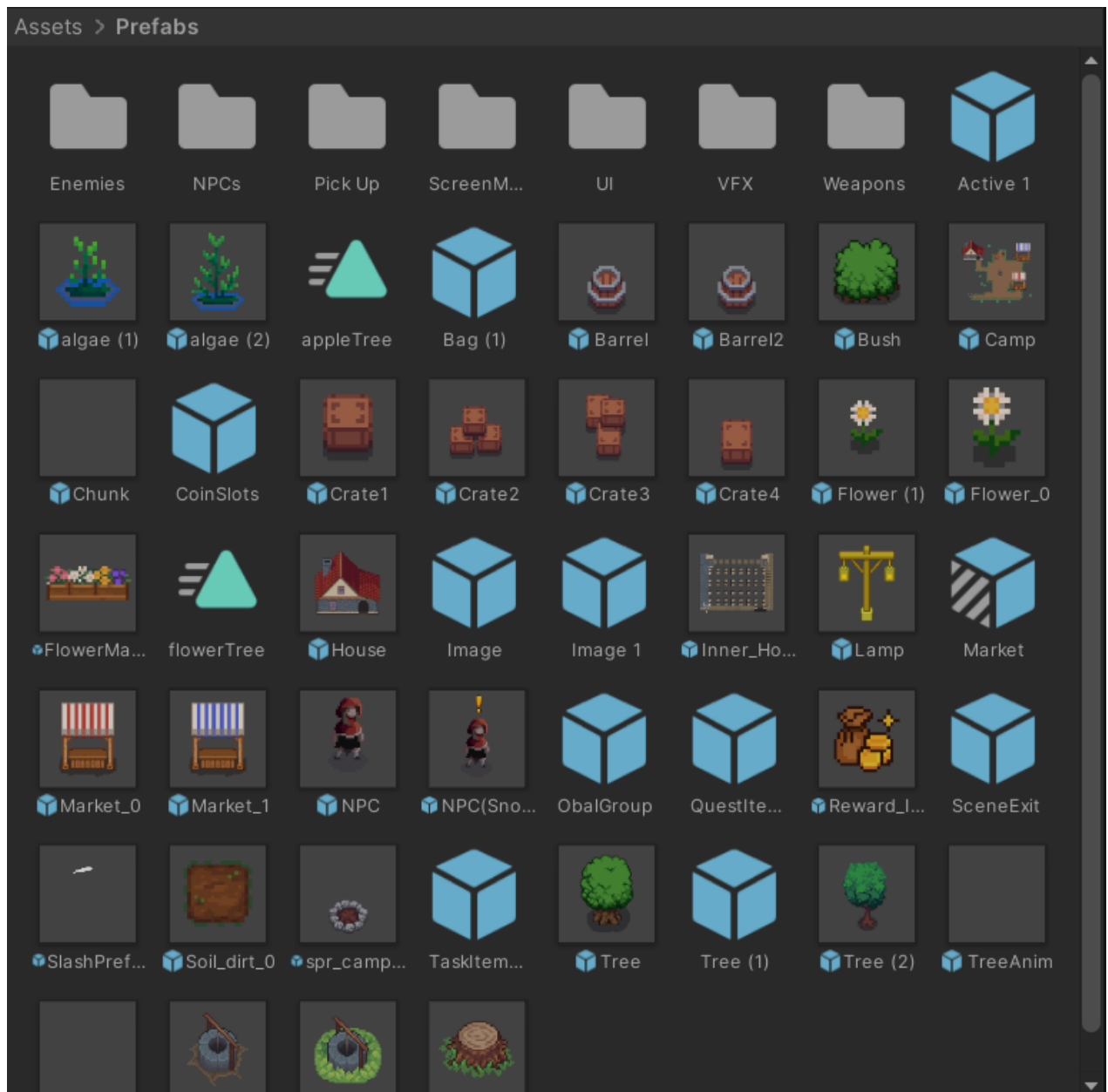


Picture 2.18. Script for NPCs

Scripts can be programmed using **C#** or **JavaScript** via an external editor. In this context, the software **Microsoft Visual Studio 2022** is used.

PREFAB:

A Prefab is essentially a Game Object used to store objects for reuse. Prefabs are used to duplicate Game Objects with identical characteristics, except for the transform properties that differentiate them



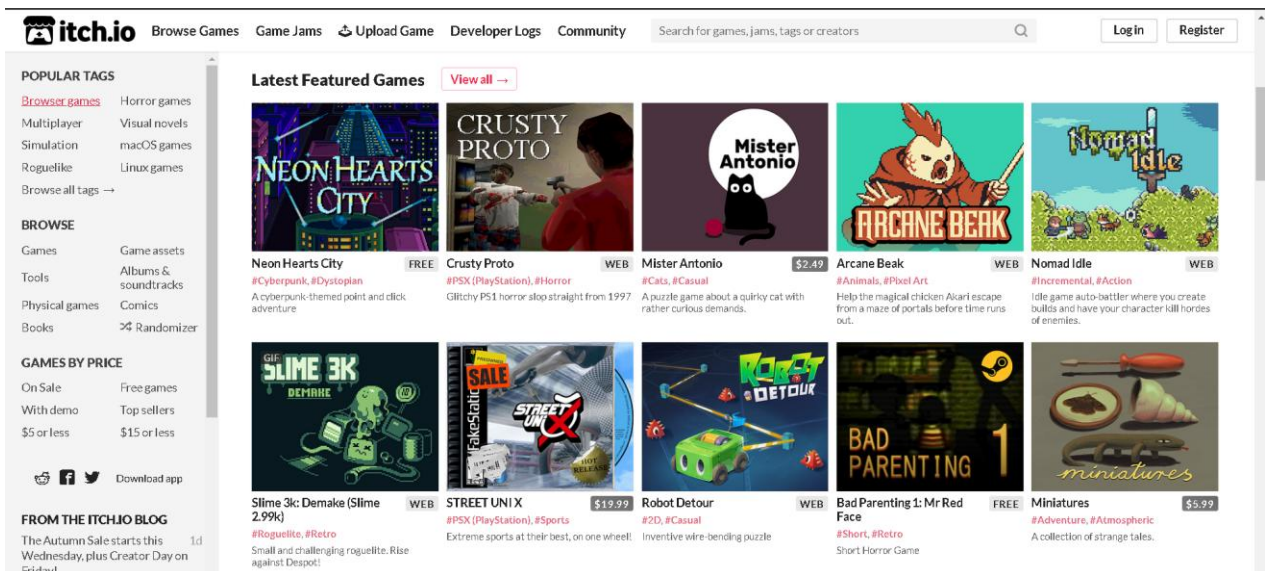
Picture 2.19. A Prefab folder save in Asset folder



Picture 2.20. Saved prefab use in a scene

c. Supporting Website and App

itch.io:



Picture 2.21. itch.to website

Itch.io is a website that provides access to free assets, similar to the Unity Asset Store, but with greater variety. It supports the search for diverse and suitable assets, saving both design time and resources.

ChatGPT:

An AI conversational assistant that supports game development through debugging and asset concept generation. For debugging, developers can describe errors or paste code snippets, and ChatGPT analyzes issues to suggest solutions, identify logical errors, and explain programming concepts. For asset creation, ChatGPT generates detailed textual descriptions of characters, items, and environments based on developer requirements. These descriptions provide blueprints that artists refine into actual game assets using tools like Aseprite. This workflow combines AI-assisted problem-solving with creative concept generation, accelerating both technical debugging and asset design processes..



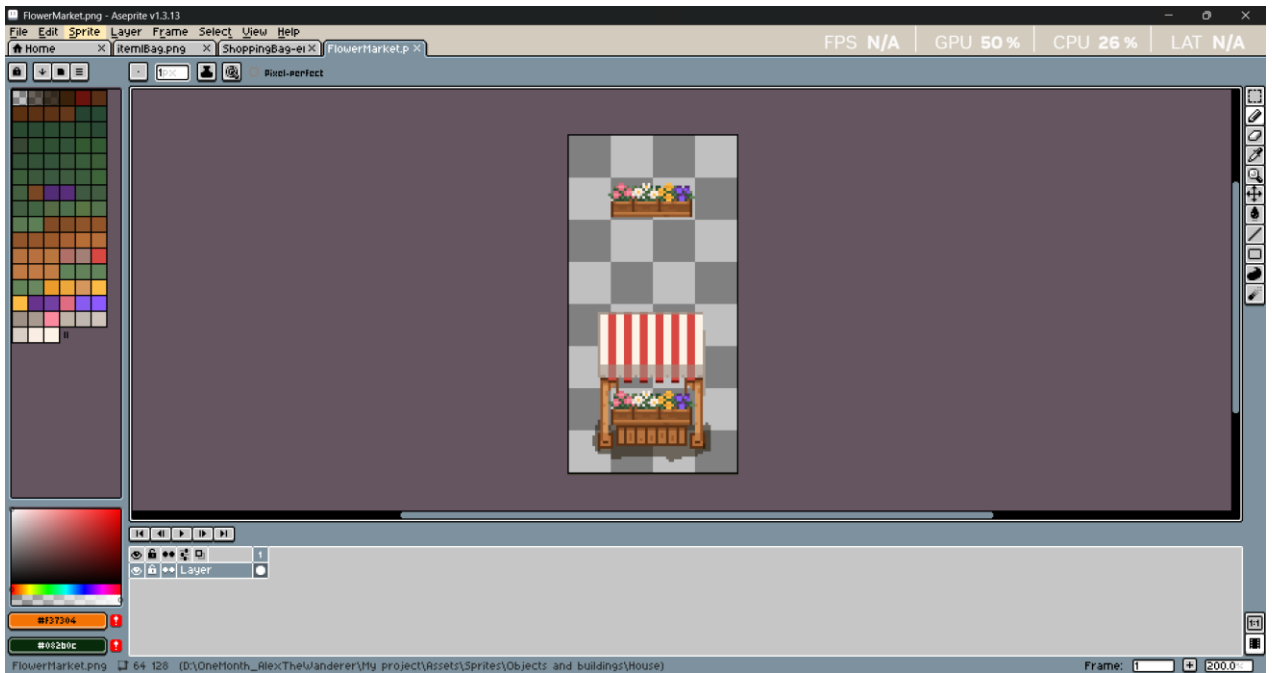
Picture 2.22. ChatGPT AI website

Microsoft Visual Studio:

A versatile programming software developed by Microsoft, supporting multiple popular programming languages. In game development, scripts that implement game functions are fully written in C# using Microsoft Visual Studio.

Aseprite:

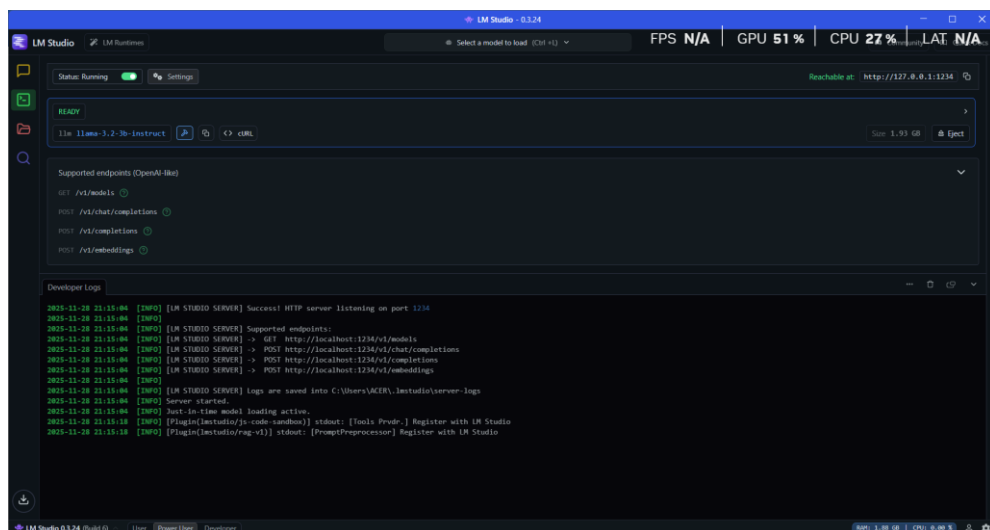
Aseprite is a specialized pixel art software designed for creating and animating 2D game sprites. It provides pixel-level editing tools for refining AI-generated asset concepts from ChatGPT and creating original pixel art from scratch.



Picture 2.23. Aseprite Interface

LM Studio

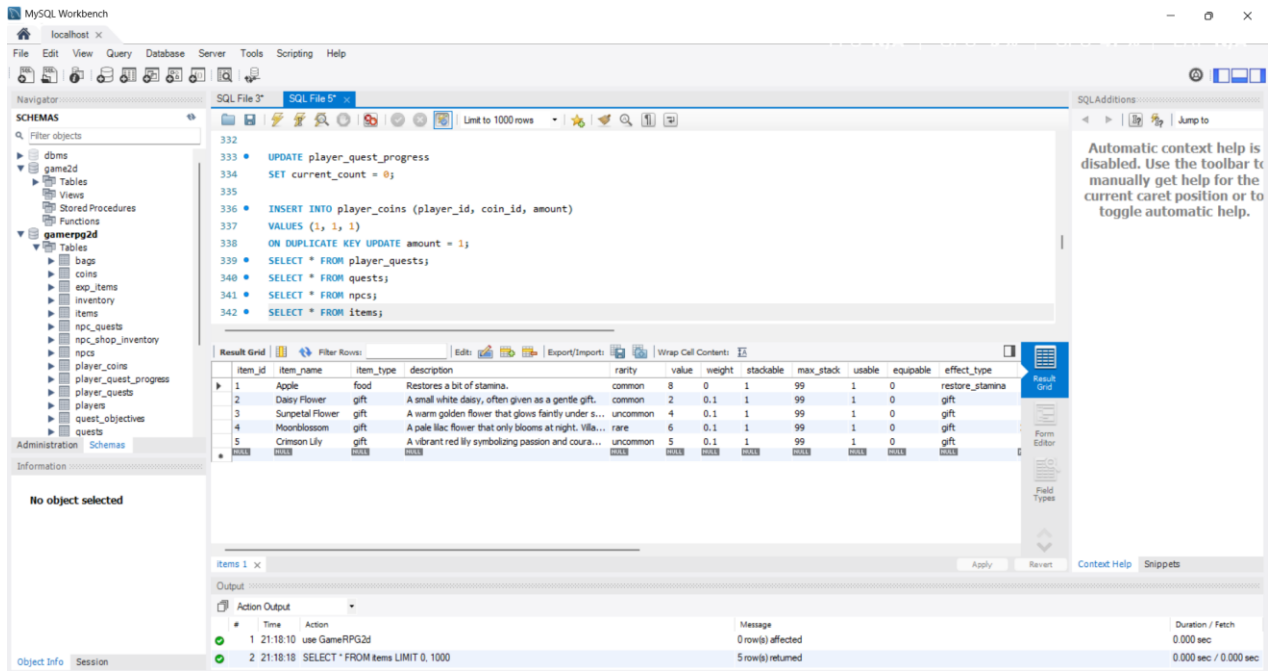
LM Studio is a desktop application for running large language models locally without cloud services. It provides an interface for hosting AI models like Llama with a local API server compatible with OpenAI's format. In this project, LM Studio runs Llama 3.2 3B Instruct on port 1234 to power NPC conversational AI and generate real-time dialogue responses.



Picture 2.24. LM Studio Interface

MySQL Workbench

MySQL Workbench is a visual database management tool that provides a graphical interface for designing database schemas, creating tables, and executing SQL queries. In this project, it manages the game database structure including quest data, items, player inventory, NPC information, and shop data through an intuitive visual interface instead of command-line operations.



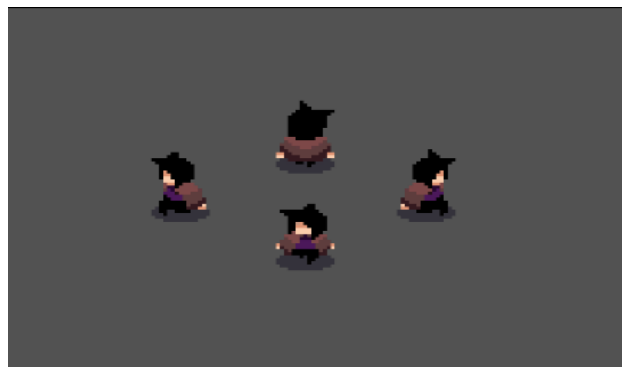
Picture 2.25. MySQL Workbench Interface

II. GAME DESIGN

a. Ideas and 2D Object Designs for the Game

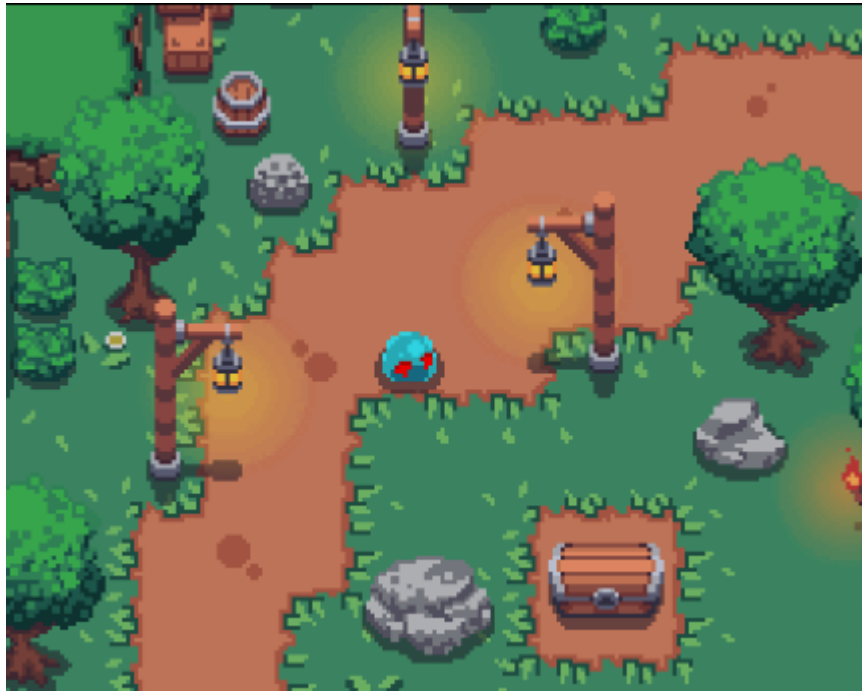
The following 2D objects are sourced from the Asset Store on itch.io:

Pixelarium is a free 2D asset pack created by LukeThePolice. This pack provides nearly all the essential elements for a game, featuring a young male protagonist with a burning desire for adventure and world exploration.



Picture 2.26. Pixelarium - Player asset

- **Monsters:** The pack includes monsters like slime, a small and weak creature that wanders around mountainous regions in search of food.



Picture 2.27. Pixelarium - Slime monster asset

- **Environment:** Pixelarium offers assets such as barrels, and wooden crates, enriching the game environment with a variety of details.



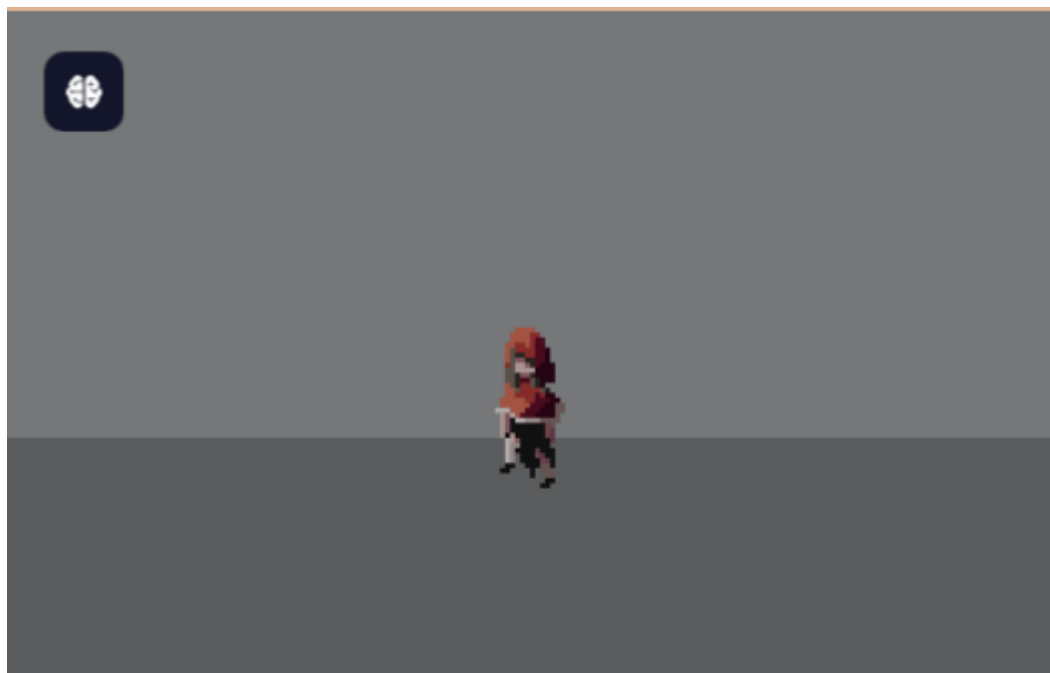
Picture 2.28. Pixelarium - Barrels and wooden crates asset

Mystic Woods is a 2D asset pack by Game Endeavor designed for creating environments. It includes trails, trees, bush, ground textures, and rivers, serving as an essential component in building a vibrant and diverse fantasy world.



Picture 2.29. Mystic Woods - Environment asset

Red Hood Pixel Character is a 2D asset, created by Legnops, depicts a girl in a red cloak. In the game, Red Hood serves as an NPC residing in the village of Virido, contributing to the story's depth and liveliness.



Picture 2.30. Red Hood NPC asset

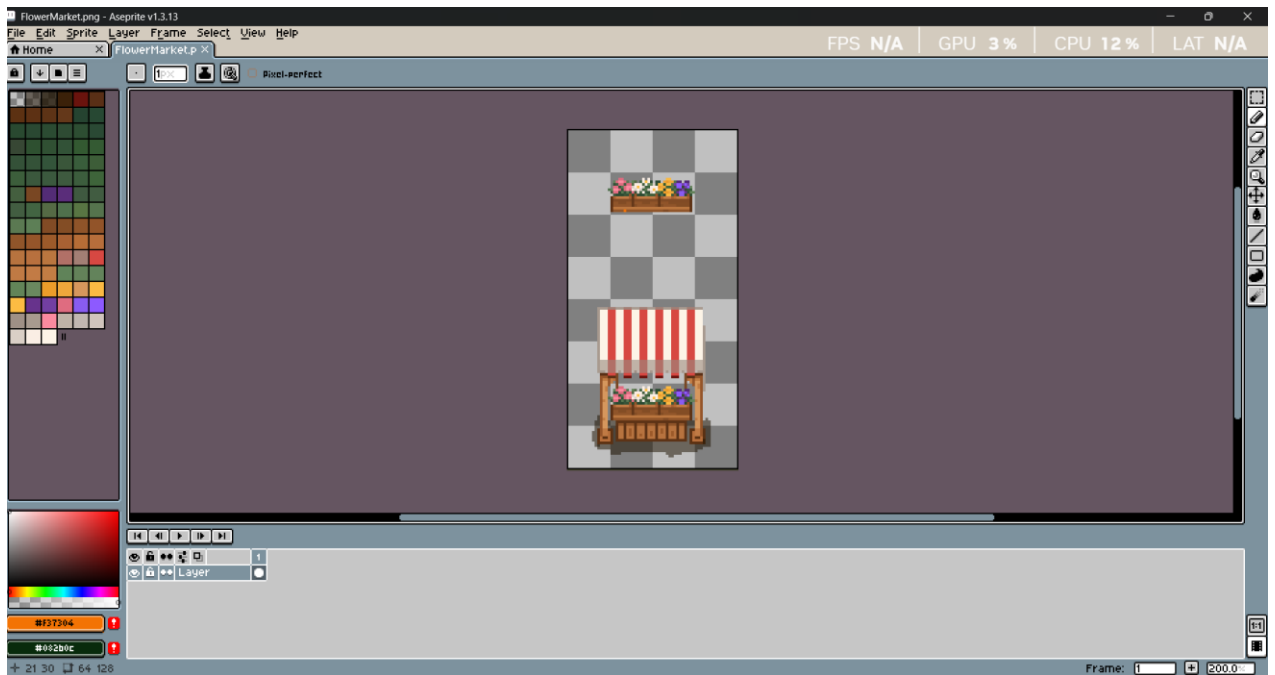
Home Top-Down Pixel Art Asset is a free top-down 2D pixel-art house asset by Free Game Assets (GUI, Sprite, Tilesets), featuring a cozy and detailed interior for RPG-style games. It can serve as an NPC home or player residence, adding warmth and life to your village environment. Its clean pixel style makes it easy to integrate into any adventure, farming, or classic 2D RPG project.



Picture 2.31. Red Hood NPC asset

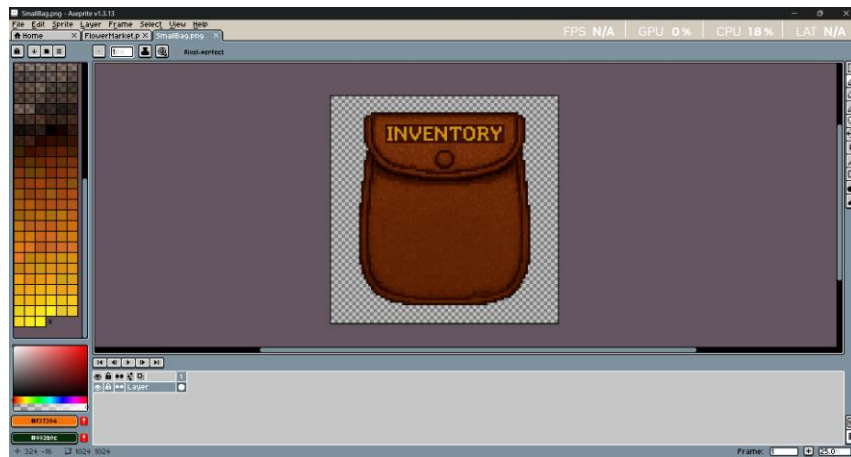
The following 2D objects are designed by Aseprite and ChatGPT:

Flower Market is a custom 2D asset combining AI-assisted design and manual artwork. The base market structure was generated through ChatGPT's image creation tool, providing the architectural framework (stall, roof, counter). I then manually drew and added various flower species, bouquets, hanging plants, and botanical details to transform it into a specialized flower trading post. This hybrid approach allowed rapid prototyping while maintaining artistic control over the final aesthetic.



Picture 2.32. FlowerMarket asset

Inventory Panel is a 12-slot inventory UI featuring a grid-based layout with item icons, quantity indicators. The panel uses a wooden frame aesthetic with parchment texture background to match the fantasy theme. Each slot displays item sprites at 32×32 resolution with stack counters in the bottom-right corner.



Picture 2.33. Inventory asset

Seasons Icons are eight circular icons representing seasonal changes with day and night: Spring (blooming flower), Summer (sun), Autumn (falling leaves), Winter (snowflake).. Icons appear in the top-left corner with subtle pulsing animation to indicate current active season.



Picture 2.34. Spring Day and Night asset

Six distinct **coins** sprite designs representing currency tiers. Each coin features rotation animation frames (8 frames per coin). Coins use a consistent 16×16 base size with slight variation in design complexity to indicate value.



Picture 2.35. Coins asset 1



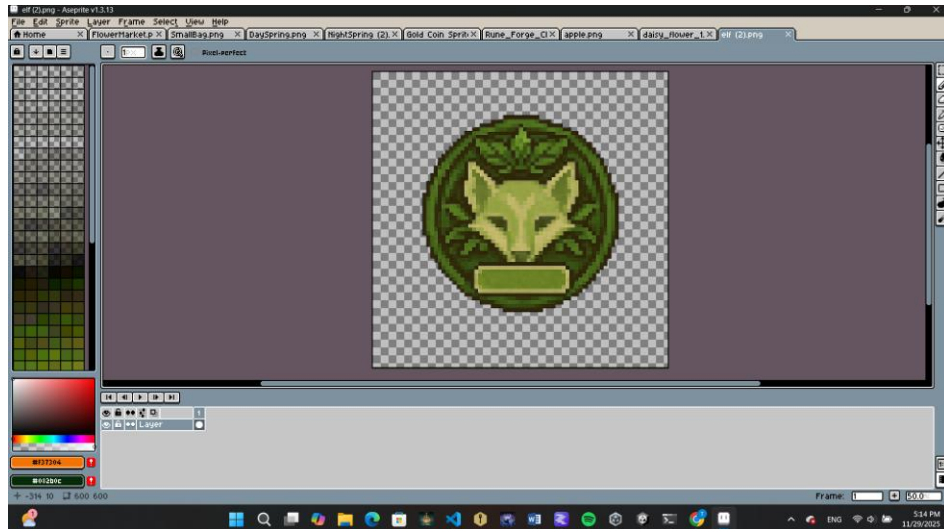
Picture 2.36. Coins asset 2

Each **prefab icon** is 32×32 pixel art prefab icons including flowers, bush and tree. Icons has transparent backgrounds for Unity integration.



Picture 2.37. Prefab asset

Level UI is a circular medallion-style badge featuring a stylized fox/wolf emblem representing the player's Elf character. The icon uses a nature-inspired fantasy design with a central canine face (cream/beige tones) set against a dark green background adorned with decorative leaf and vine motifs. The badge has a layered rim design with multiple shades of green creating depth and a coin-like appearance. A horizontal status bar placeholder sits at the bottom of the medallion for displaying level numbers or experience progress.



Picture 2.38. Level Interface

b. Interface and Component Design

Game Interfaces:

Menu Interface:



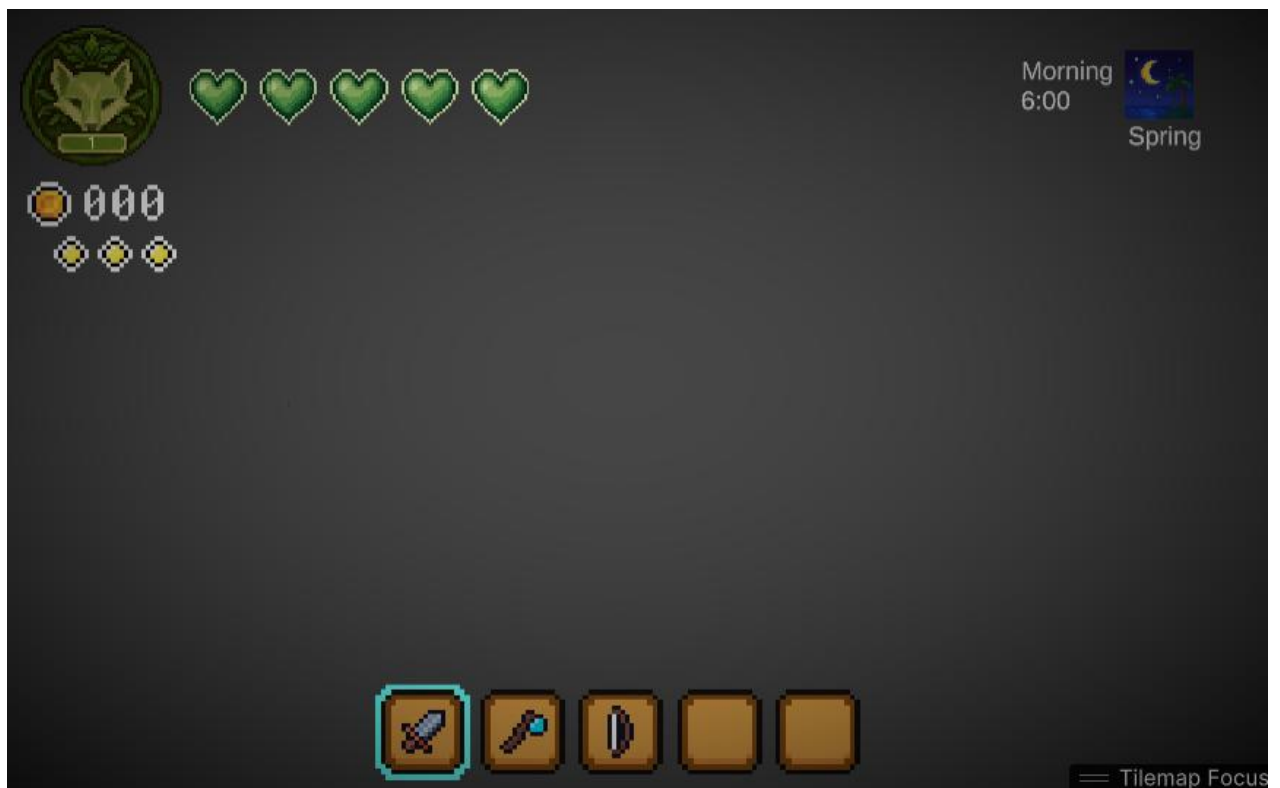
Picture 2.39. Mainmenu Interface

Pause Interface:



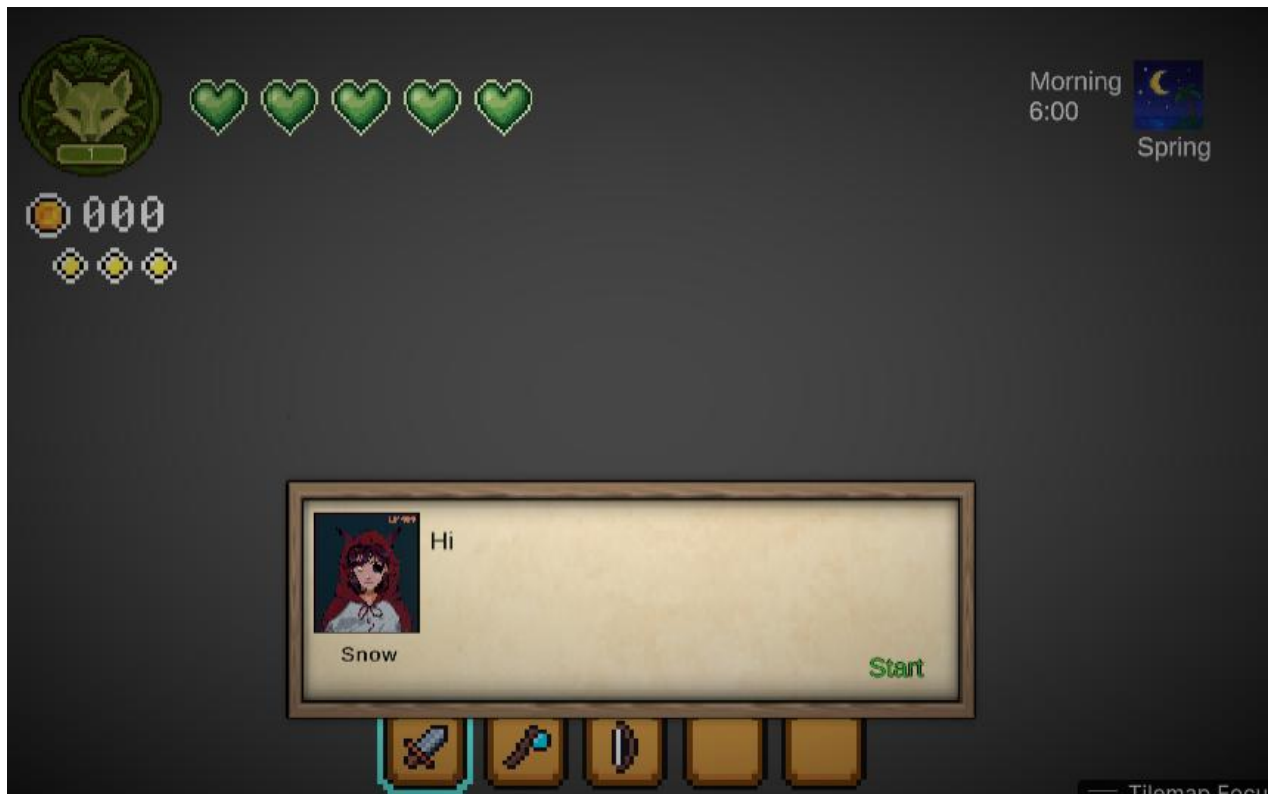
Picture 2.40. Pause game Interface

Gameplay Interface



Picture 2.41. Gameplay Interface

Gameplay Dialog NPCs



Picture 2.42. Gameplay Dialog NPCs

Gameplay Inventory Interface



Picture 2.43. Gameplay Inventory

Gameplay Items Descriptions Interface



Picture 2.44. Gameplay Items Descriptions

Gameplay CoinsInventory Interface:



Picture 2.45. Gameplay CoinsInventory

Gameplay QuestPanel Interface:



Picture 2.46. Gameplay QuestPanel

GamePlay Trade Interface:

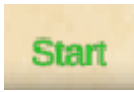


Picture 2.47. Gameplay Trade Interface

Game Components:

The game components include interactive elements and elements within a game scene.

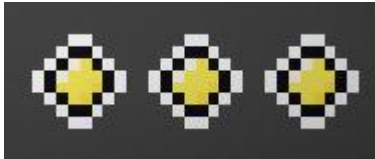
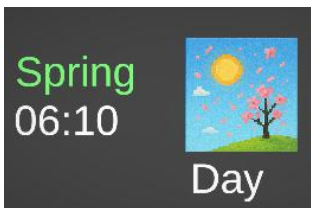
- **Interactive Components:** These are used for small interactions with NPCs or to perform actions such as starting or exiting the game.

No.	Components	Note
1		Play button in Mainmenu to enter Scence 1 of the game
2		Exit button in Mainmenu to exit the game
3		Start button when interact with NPCs to turn on player Mic
4		Stop Button when Interact with NPCs turn off player Mic
5		Coins Button appear with a slide out effect when players choose it in Inventory UI to trigger CoinsInventory
6		Items Detail Button in Inventory to open Item Detail UI
7		Quest Button to trigger the quest list if there an available quest in process
8		Quest in process Button to trigger the detail of the quest

9		Close Quest Button to close the quest panel
10		Items in shop Button to choose a item that player want to buy
11		Shopping Bag Button let player transpost the items they want or return
12		MoneyBox Button let Player Confirm their Payment
13		Coins Button let Players choose the coins they want for the payment

Table 2.1. Table of interactive components

- **Scene Components:** These elements are carefully selected to provide players with a minimalist interface that reflects the game's style while ensuring that essential information is presented clearly and effectively.

No.	Components	Note
1	 A circular green icon with a fox head and a level indicator '1' next to five green hearts.	Displays the player's current health and level.
2	 Three yellow gems with black and white checkered patterns.	Shows the remaining energy gems available for performing dashes
3	 A yellow coin icon followed by three white coin icons.	Tracks the number of coins the player has collected
4	 Five icons representing different weapons: a sword, a pickaxe, a hammer, and two empty slots.	Indicates weapon changes and the type of weapon the player is currently using
5	 A dialogue box showing an NPC's portrait, name 'Snow', and the text 'Hi'. A 'Start' button is visible in the bottom right corner.	Displays the NPC's name, portrait, and dialogue during interactions with the player.
6	 A UI element showing the season 'Spring', the time '06:10', and the day 'Day' next to a small icon of a tree and a sun.	Display seasons, time, day and Icon for time management

7		Display Player Inventory
8		Display each Item Detail
9		Display Quest Panel, Quest List and Quest detail
10		Display Coins Inventory

11		Display Shop Items Inventory that available for sell
12		Display showing cart total and coin payment

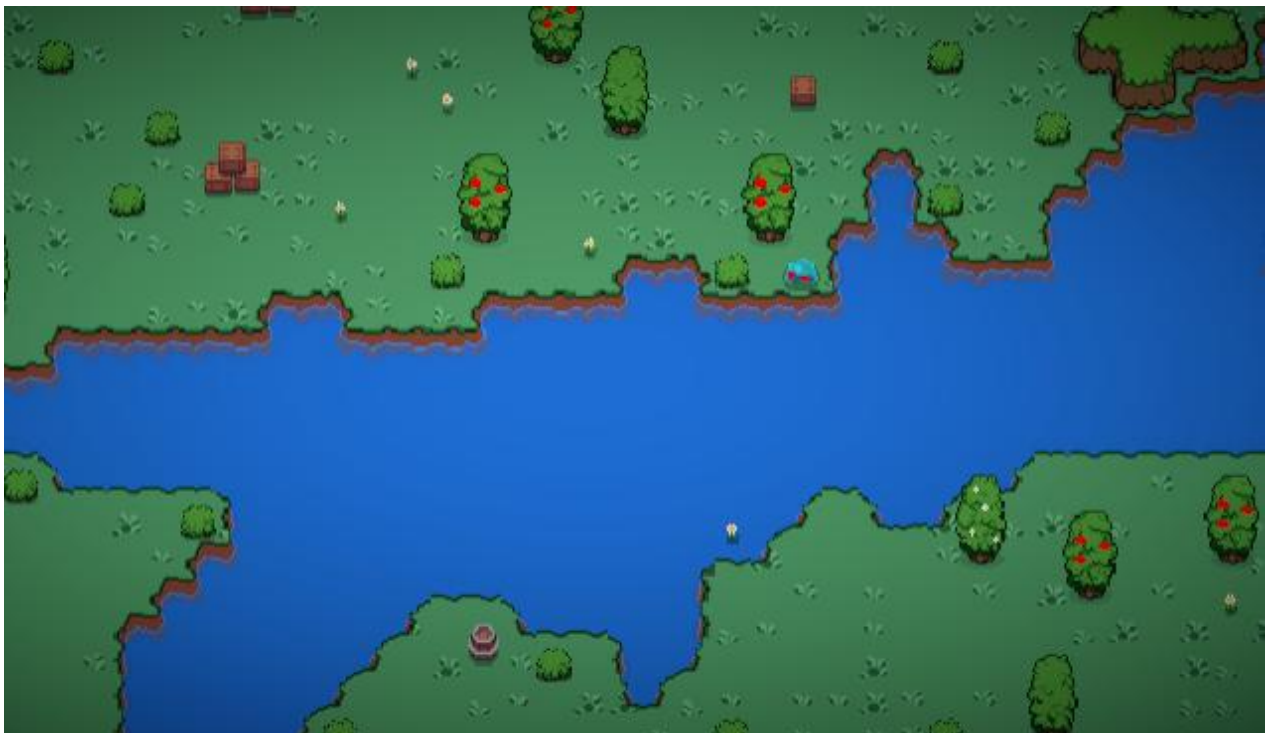
Table 2.2. Table of scene components

III. GAME BUILD

a. Game Environment



Picture 2.48. Environment Layout 1



Picture 2.49. Environment Layout 2

b. Placement of Main Objects in the Game

Enemy:

Locations of enemy objects in Game will be decided through Biome Definitions:



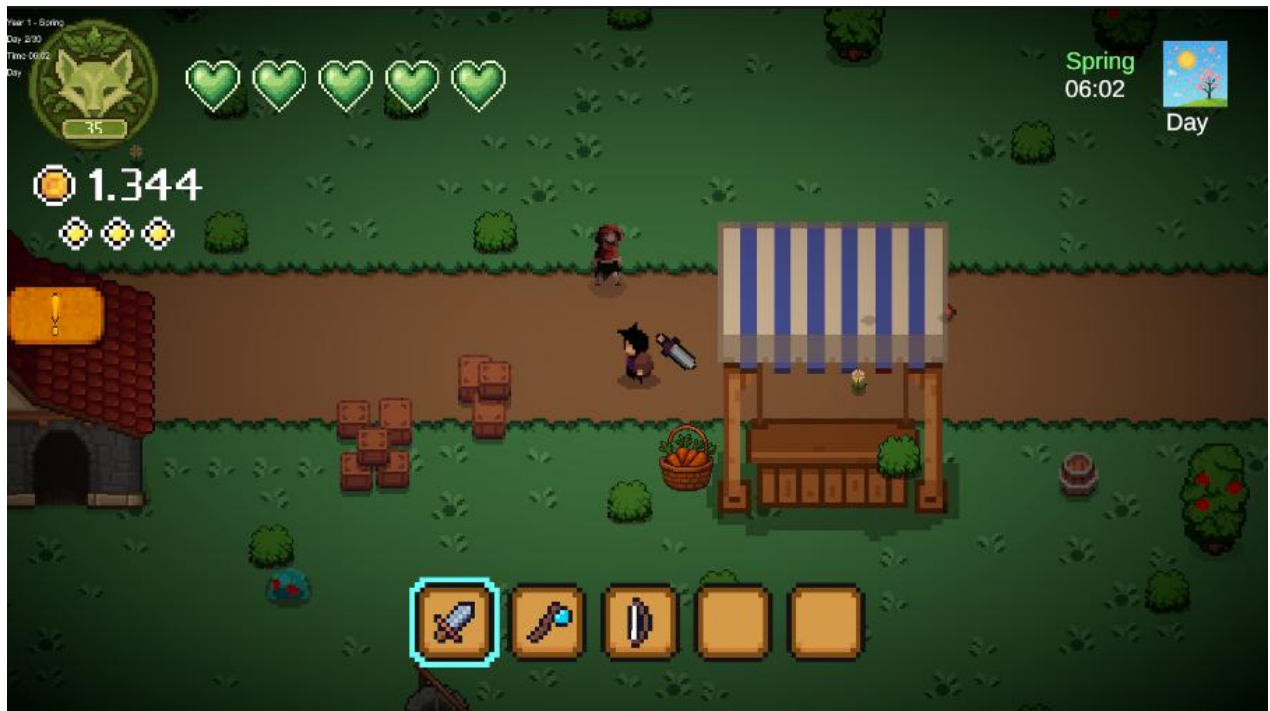
Picture 2.50. Slime Enemy



Picture 2.51. Grape Shotter Enemy

Main Character:

Main Character will spawn at the middle of the Map:



Picture 2.52. Main character spawn

IV. PROGRAMMING GAME COMPONENT FUNCTIONALITY

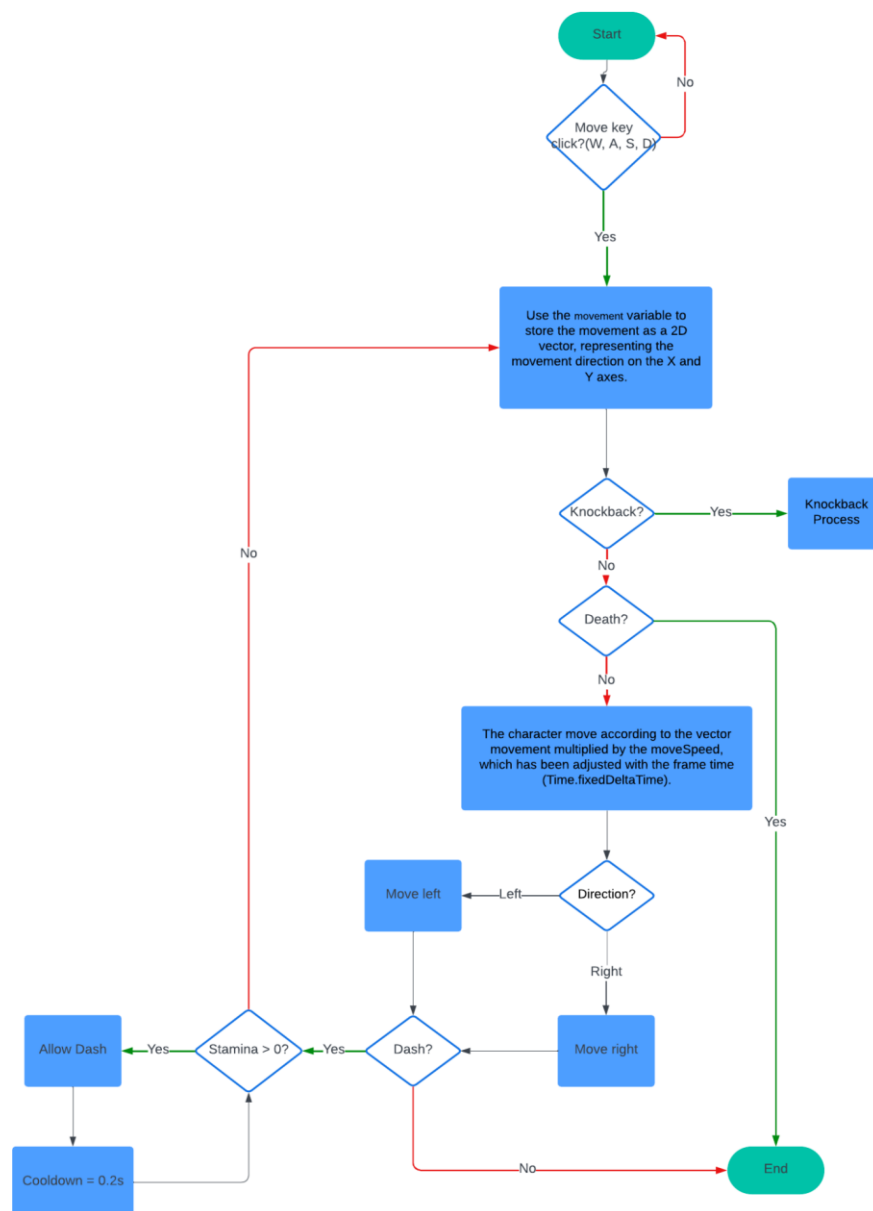
a. Main Components in the Game

- Player: The main character, Sylv
- Enemies:
 - Slime Monster
 - GrapeShooter Monster
- **Environment:** *Eclycia* includes the following objects:
 - Trees
 - Bushes
 - Ponds
 - Barrels
 - Crates
 - Small trees (with 3 conditions: Normal, Flower, and Apple)
 - Dirt walls

- Fire torches
- House
- Camp
- Market Stall
- Market Selling Stall
- Flowers (Daisy, Crimson Lily, Moonblossom, and Sunpetal Flower)

b. Flowchart

The movement mechanism of the main character in the game:



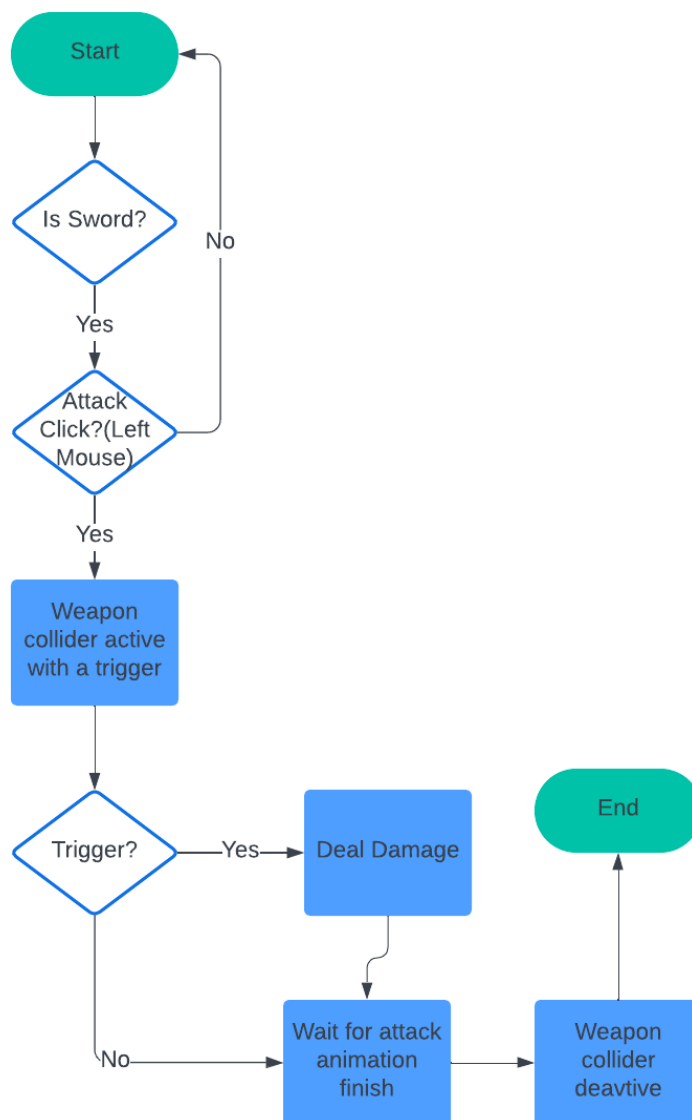
Picture 2.53. Player Movement Flowchart

The attack mechanism of the sword:

When the player performs an attack, the attack method is called. First, the animator triggers the sword's attack animation. Then, the weapon collider (the sword's collider) is activated, allowing the sword to deal damage when it collides with objects.

Next, an attack effect object is instantiated at the slash animation spawn point using instantiate, creating the visual effect for the attack. This object is parented to the sword to ensure the effect moves along with the sword during the attack and along side with the sword sound effect.

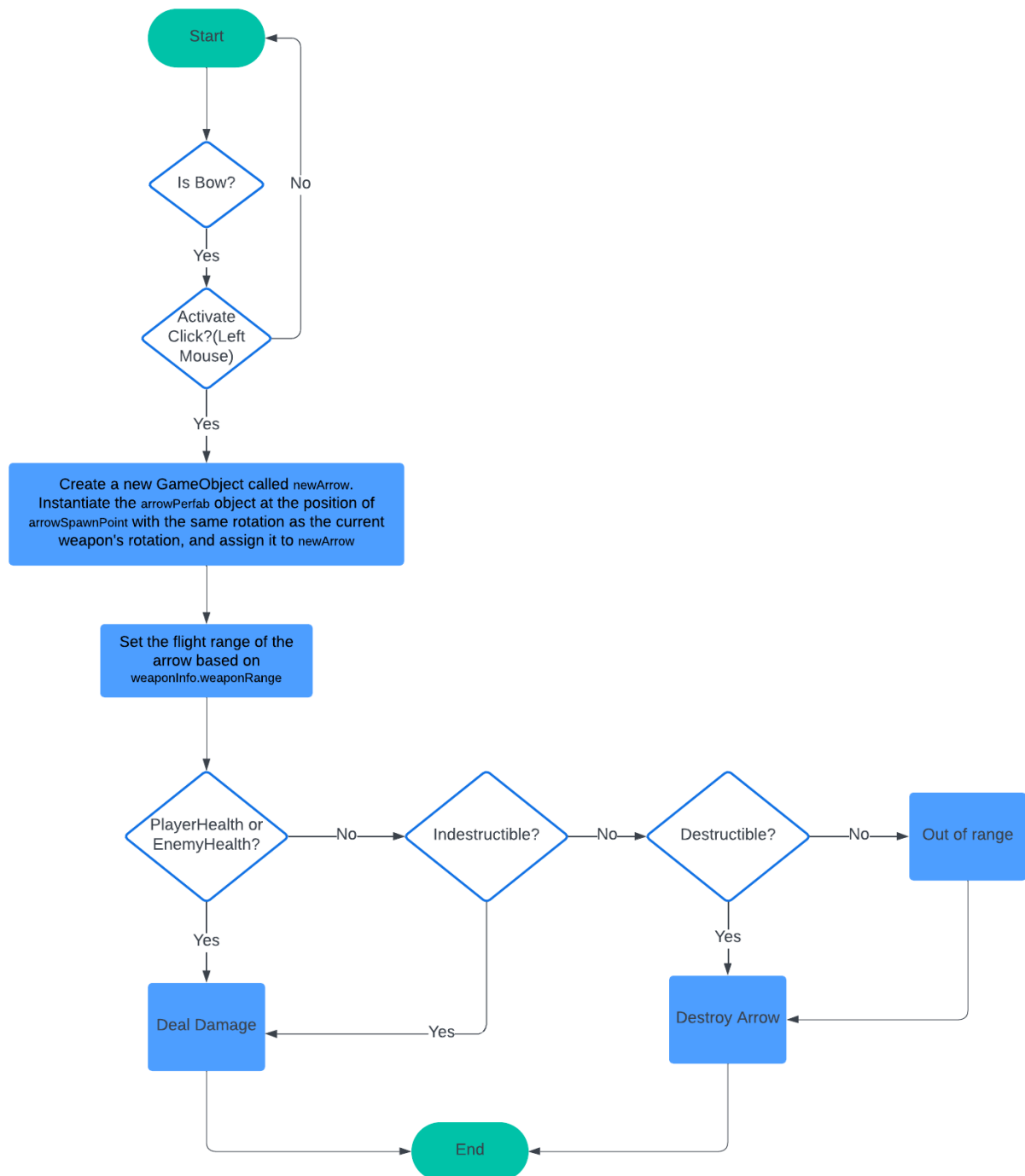
Once the attack animation is completed, deactivates the weapon collider, stopping the sword from dealing damage. This ensures that the sword can only deal damage during the attack phase and will not cause damage outside of that time.



Picture 2.54. Sword attack flowchart

The attack mechanism of the bow:

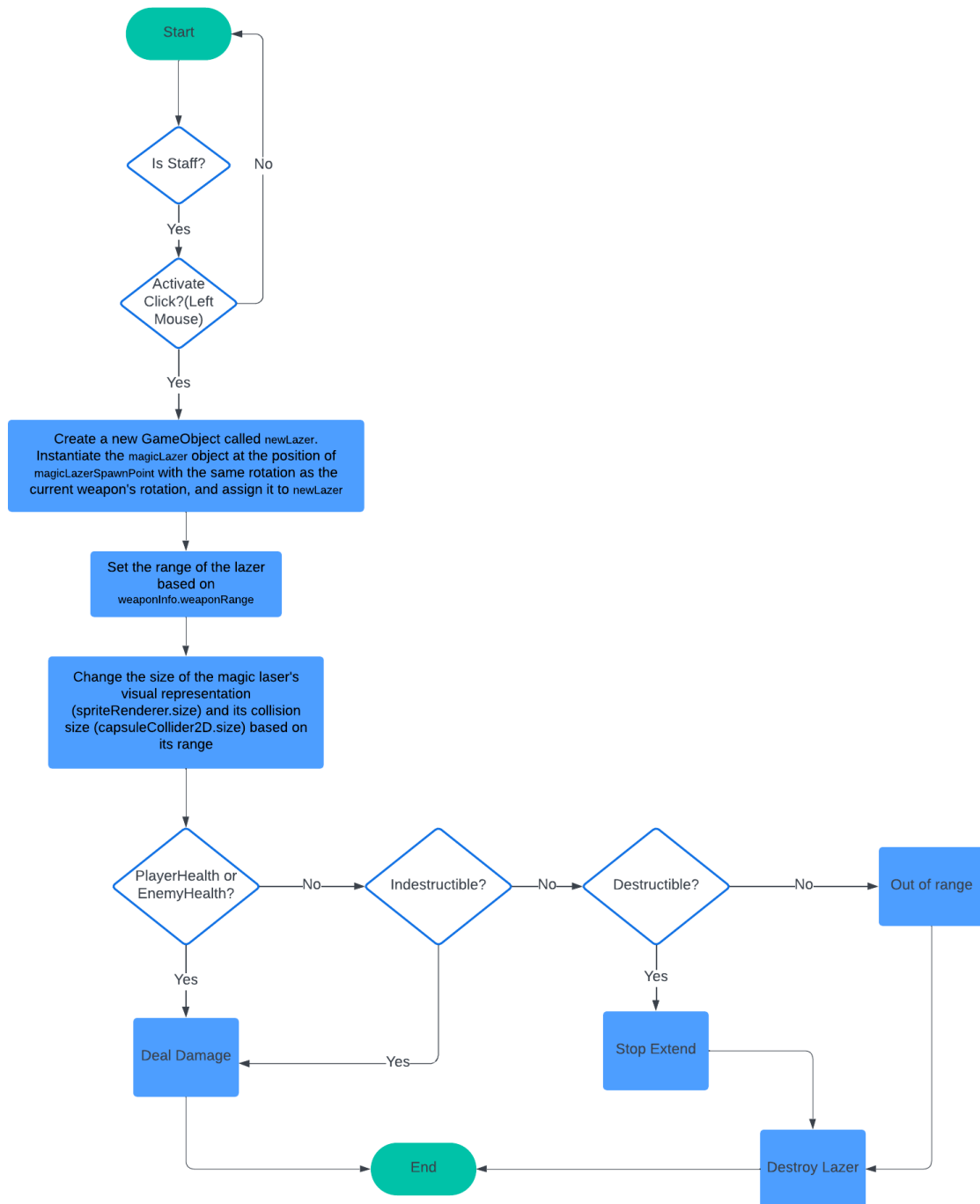
The bow attack mechanism creates a new arrow when the attack button is pressed. The arrow is spawned at the bow's spawn point and faces the same direction as the current weapon with a bow sound effect. The arrow's flight range is set using the `weaponInfo.weaponRange`. The arrow, managed by the `ProjectTile` script, will deal damage upon collision with an enemy by calling a method like `TakeDamage()`, with damage calculated based on the weapon's stats.



Picture 2.55. Bow attack flowchart

The attack mechanism of the staff:

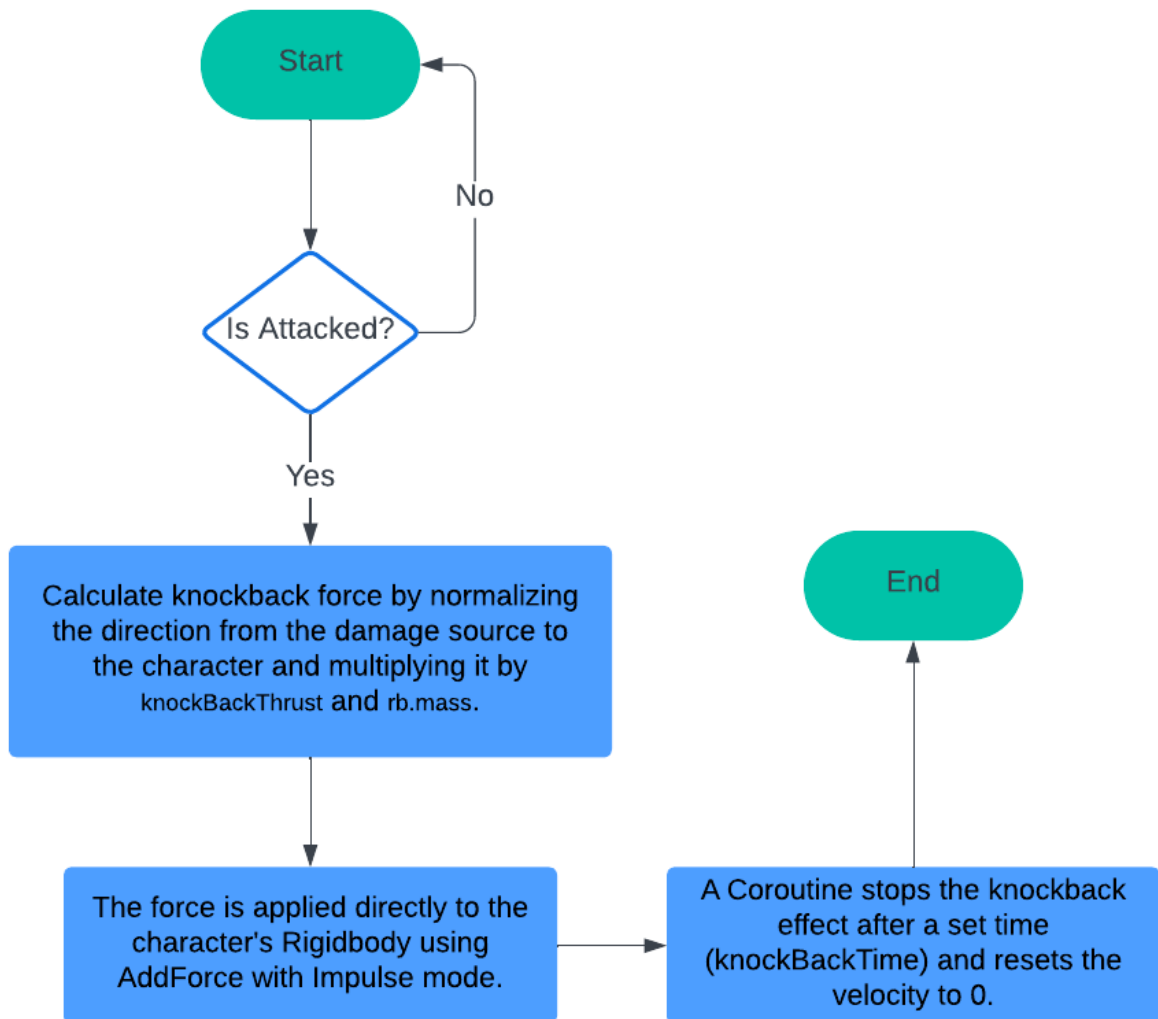
When using the staff, the attack is initiated by calling the `Attack()` method, which triggers an animation. During the attack animation, the `SpawnStaffProjectileAnimEvent()` event spawns a magic laser (`magicLazer`) at the designated spawn point with a spell sound. The laser's range is determined by the weapon's range, and its size increases over time using a coroutine (`IncreaseLazerLengthRoutine()`). The laser can stop growing if it collides with indestructible objects. The laser's direction is aimed at the mouse cursor, ensuring it shoots in the correct direction.



Picture 2.56. Staff attack flowchart

Knockback:

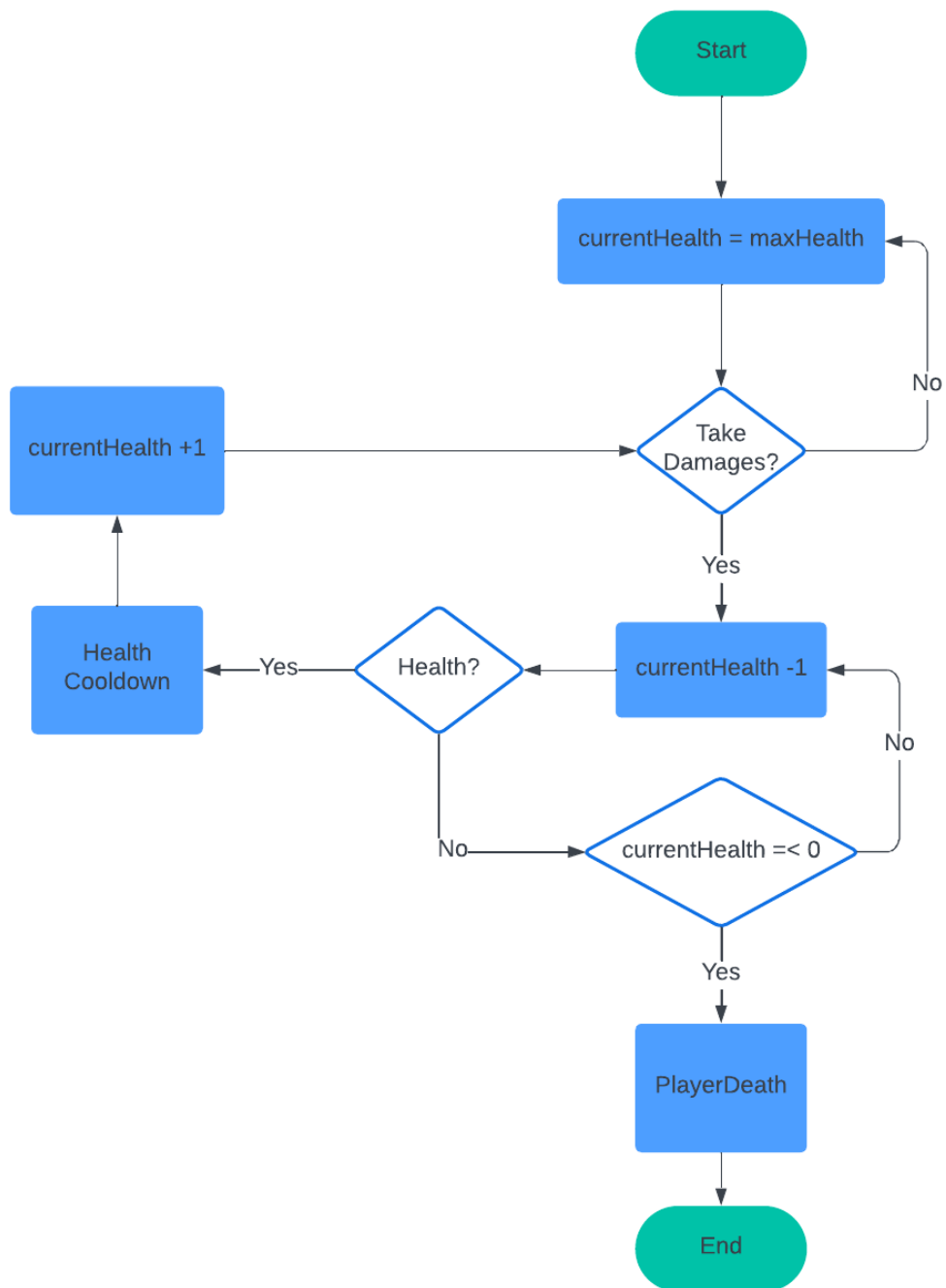
The `GetKnockBack` method activates the knockback effect by setting `GettingKnockBack = true`. It calculates the force based on the direction from the damage source to the character, normalizes it, and multiplies by the knockback thrust and mass with a hit sound. The force is applied to the `Rigidbody` using `AddForce` with `Impulse`. A `Coroutine` stops the knockback after a set time, resets velocity to 0, and deactivates the knockback state.



Picture 2.57. Knockback flowchart

Player Health:

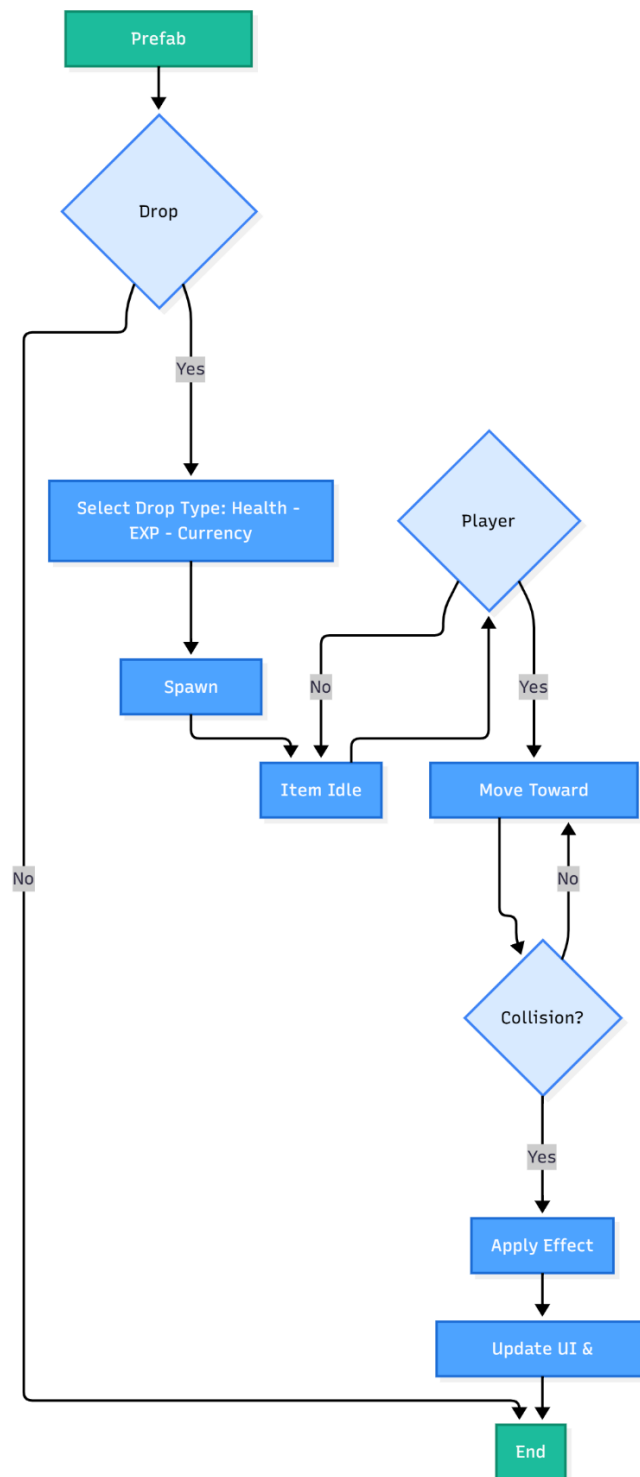
The player's health system is managed through the `currentHealth` variable, which is initialized to `maxHealth` at the start of the game. The health slider is updated by the `UpdateHealthSlider()` method to reflect the current health. When the player takes damage, the `TakeDamage()` method decreases the `currentHealth` and updates the slider. If the player's health reaches zero, the `CheckPlayerDeath()` method triggers the death sequence. Additionally, the `HealhPlayer()` method allows for health recovery, updating the slider accordingly.



Picture 2.58. Player health flowchart

Item Spawn and Pickup:

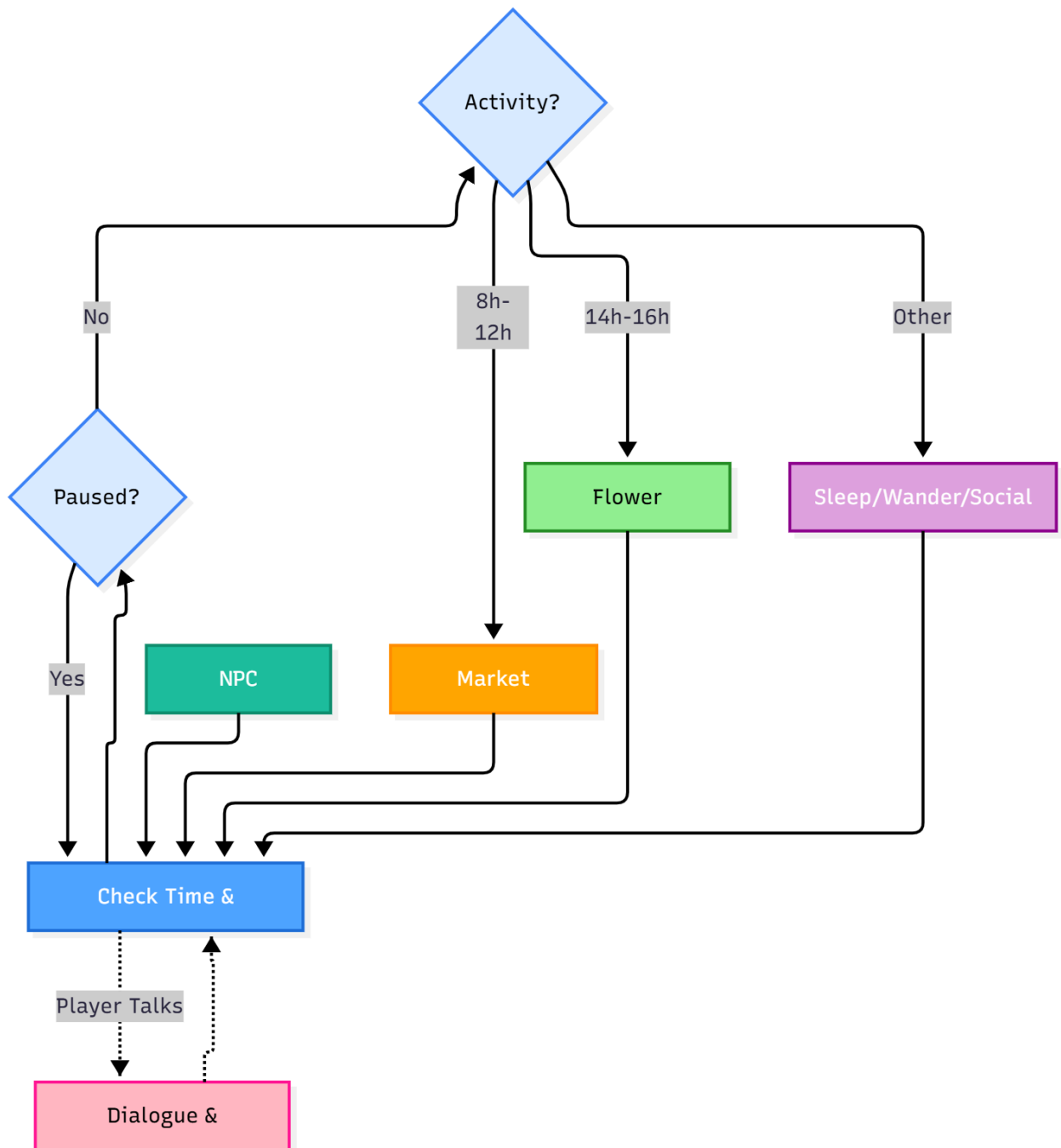
When an enemy or destructible object is destroyed or hited, it may spawn a random drop such as gold, health, experience, or collectible items (e.g., apples from trees). The item remains on the ground until the player comes close, then it moves toward the player and accelerates. When collected, its type is checked and the corresponding effect is applied. After updating the player's gold, health, EXP, or inventory, the item is destroyed.



Picture 2.59. Item Spawn and Pickup flowchart

Main NPC Routine Loop - Core System:

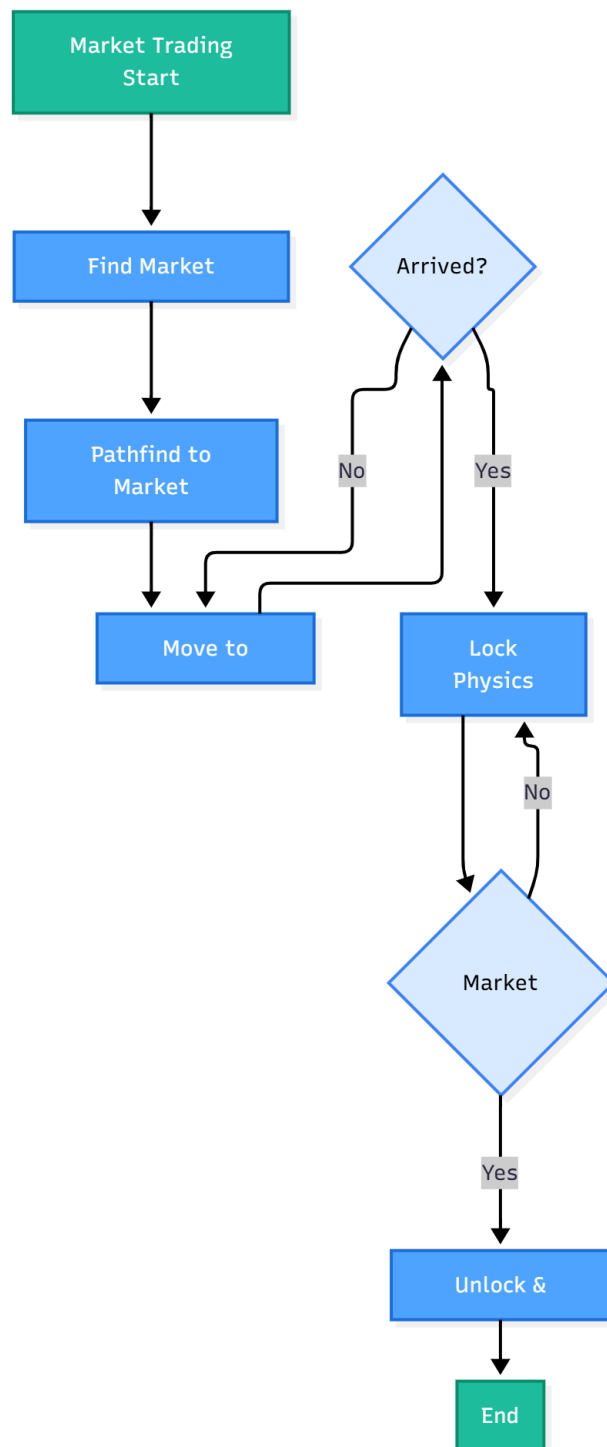
The routine system continuously checks the current time from TimeManager to determine the appropriate activity for the NPC. When the player talks to the NPC, the entire routine pauses and waits for the dialogue to end before resuming. Based on the time of day, the NPC automatically switches between activities such as Market Trading (8h-12h for traders), Flower Hunting (14h-16h), or other activities like sleeping, wandering, and socializing. This loop runs infinitely and continuously updates the NPC's state according to the game's 24-hour cycle.



Picture 2.60. Main NPC Routine Loop

Market Trading Routine:

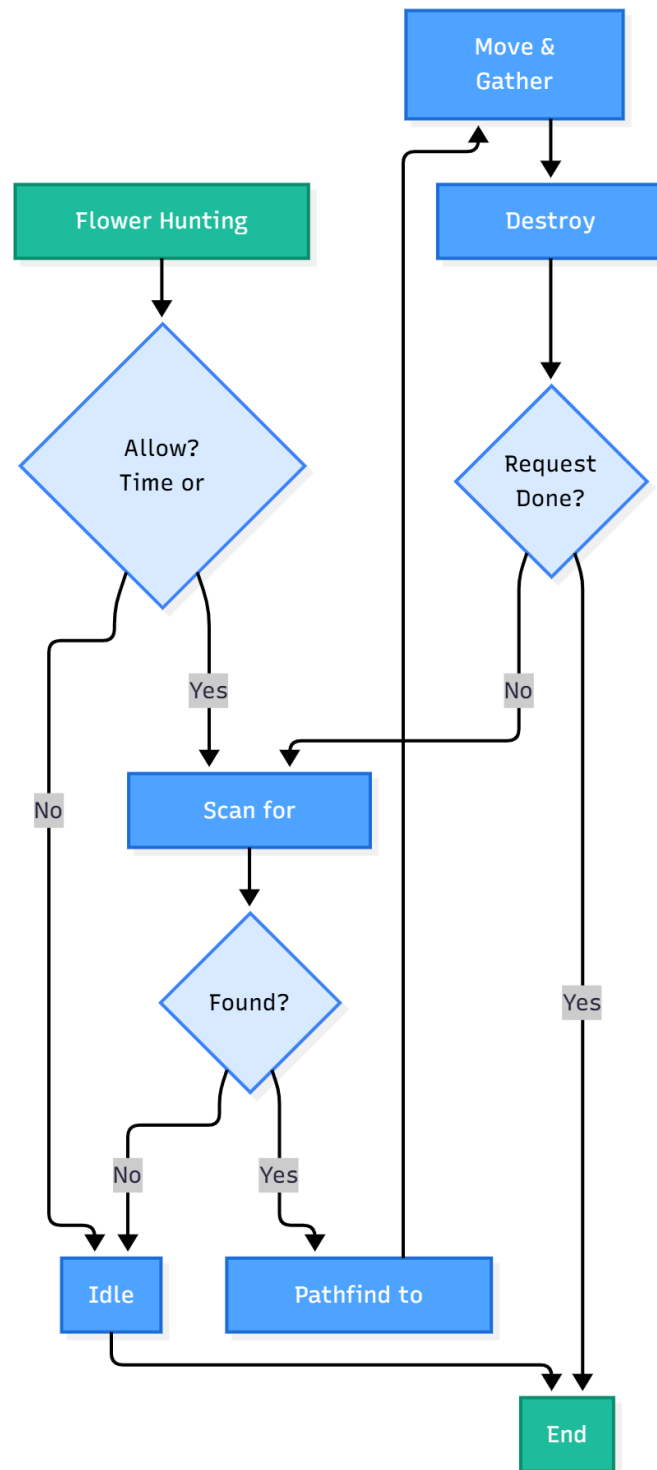
When market hours begin (8h-12h), the trader NPC automatically searches for the market location in the scene through the Camp and StandPoint search system. After determining the position, the NPC uses the A* pathfinding algorithm to move to the market and locks physics to stand still at the stall. The shop indicator is displayed and the shop UI becomes accessible to the player throughout the market's opening hours. When time expires (12h), the NPC automatically closes the shop, unlocks physics, and returns to the main routine loop to perform the next activity.



Picture 2.61. Market Trading Routine

Flower Hunting Routine:

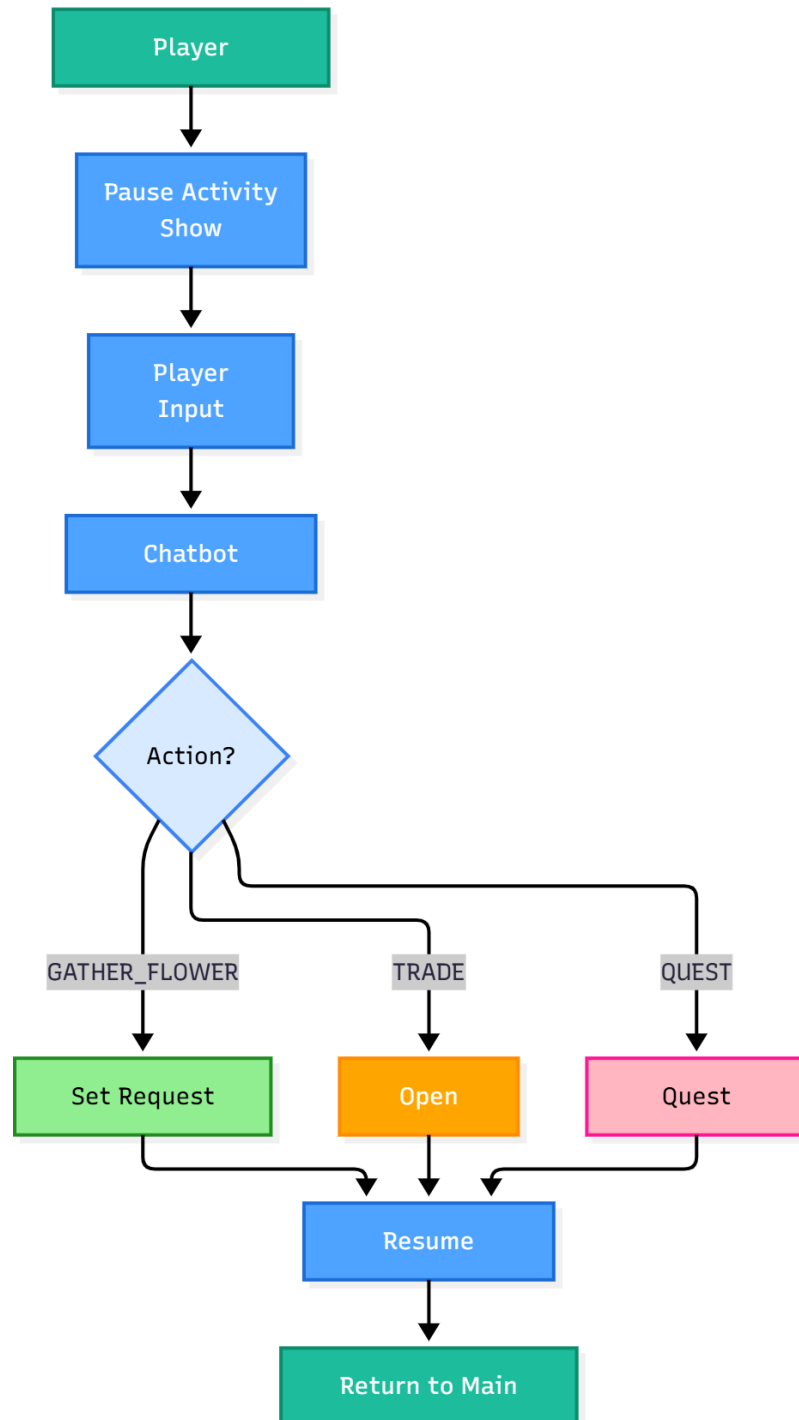
The NPC is only allowed to gather flowers when within the 14h-16h time window or when requested by the player through the dialogue system. The system scans all flowers within the detection radius and selects the nearest flower to move toward using A* pathfinding. When close enough to the flower, the NPC stops and performs a 3-second gathering animation, then notifies the FlowerManager to respawn the flower after a period of time. If it's a player request, the flag is reset after completing the task, while if it's a normal routine, the NPC continues to scan and gather flowers until the time window ends.



Picture 2.62. Flowers Hunting Routine

Dialogue Pause & Player Override System:

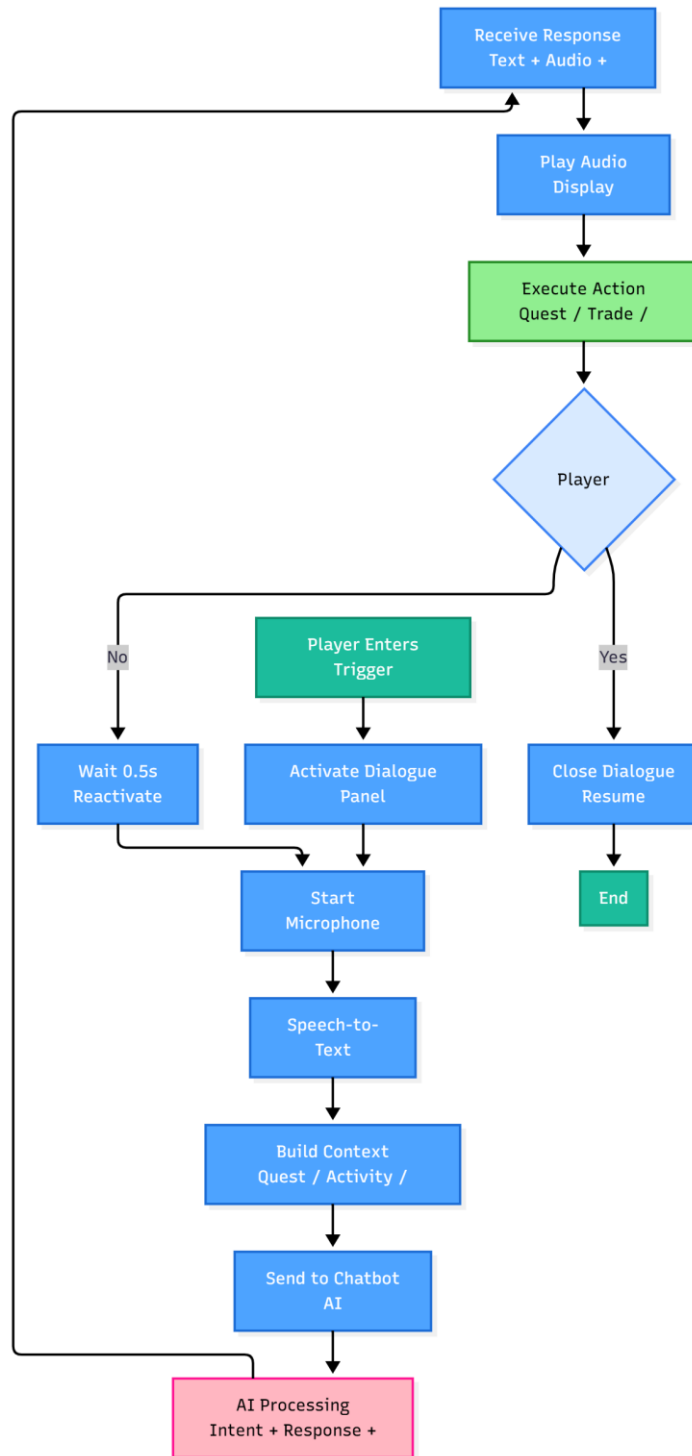
When the player enters the NPC's trigger radius, the current activity is paused and the dialogue panel appears to begin interaction. The player can communicate through voice or text input, then the chatbot AI processes and returns corresponding actions such as GATHER_FLOWER, OPEN_SHOP, or QUEST. If the action is GATHER_FLOWER, the system sets the playerMadeRequest flag to true to override time restrictions and allow the NPC to gather flowers immediately regardless of the time. After the player exits the dialogue, the NPC resumes activity and checks the override flag to execute the request before returning to the normal routine.



Picture 2.63. Dialogue Pause & Player Override System

NPC Voice-to-AI Dialogue System:

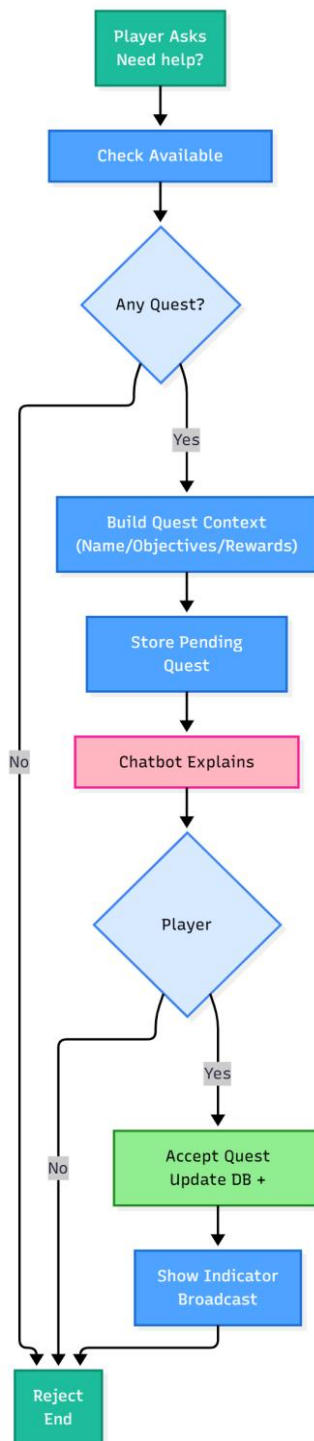
The voice-to-AI dialogue system starts when the player enters the NPC's trigger radius, pausing the NPC's routine and activating the dialogue interface. The system records 5 seconds of voice input, converts it to text, builds context, and sends it to the chatbot for intent detection, response generation, and TTS audio. The NPC then speaks the response while showing subtitles and performing the requested action. After each reply, the system waits briefly before listening again, continuing the loop until the player leaves, allowing the NPC to resume its normal routine.



Picture 2.64. NPC Voice-to-AI Dialogue System

NPC Quest Assignment (Multi-Turn Dialogue):

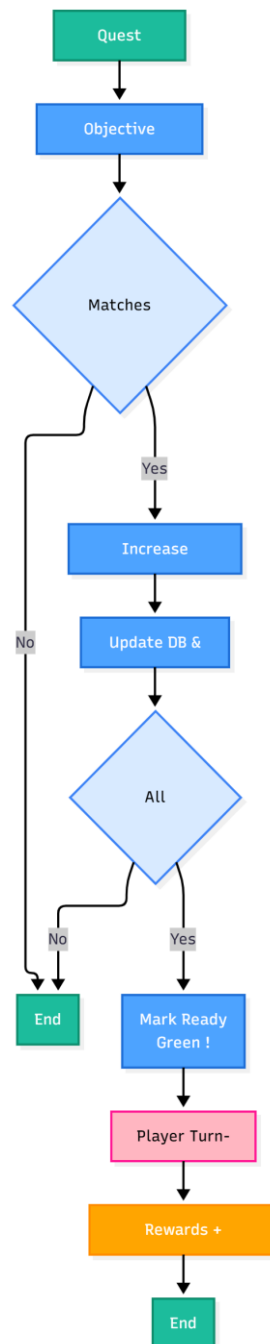
The multi-turn quest assignment system enables natural dialogue by storing quest state between player responses. When the player asks for work, the system checks available quests, builds a context with the quest's details, and has the chatbot explain it without auto-accepting. The player can confirm or decline, and if confirmed, the system officially accepts the quest, updates the database, initializes progress, and notifies the player with the yellow quest indicator. This two-step flow prevents accidental acceptance and makes quest conversations feel more natural and interactive.



Picture 2.65. NPC Voice-to-AI Dialogue System

Quest Progress Tracking & Completion:

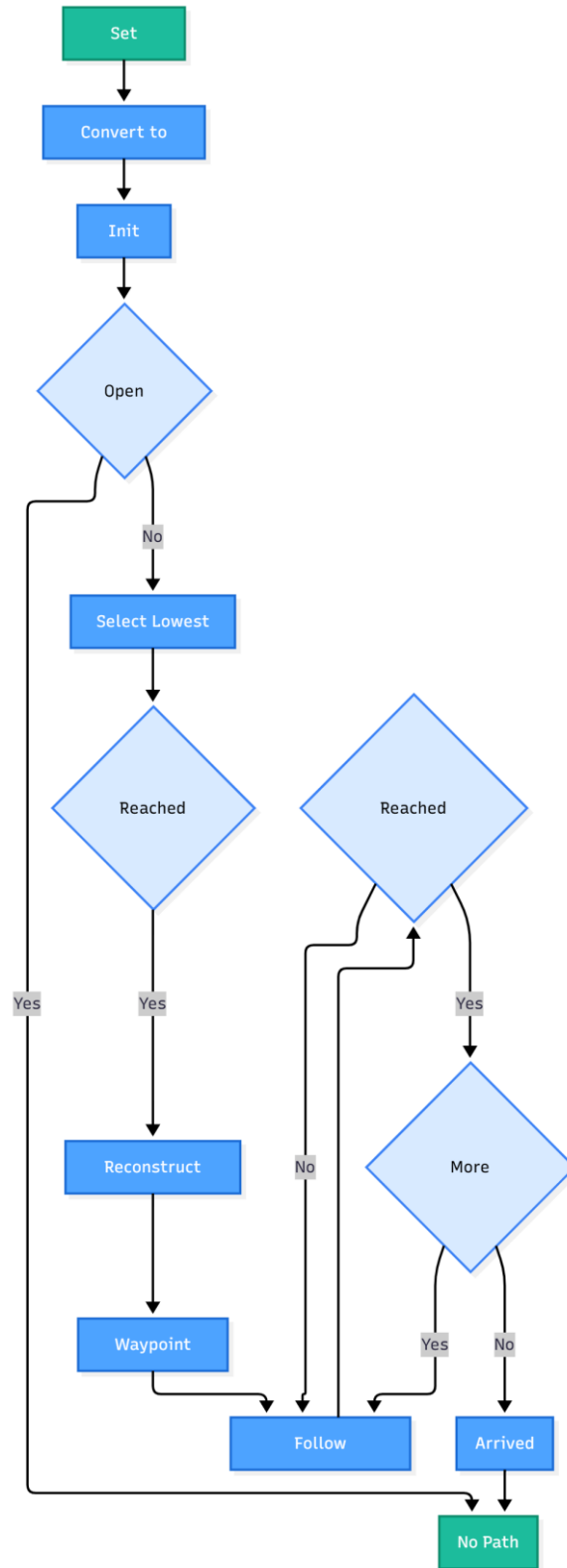
Quest progress is updated in real time by checking whether item pickups or enemy kills match any active quest objectives. When progress increases, the system updates the database, refreshes the UI, and broadcasts progress events. Once all objectives are complete, the quest becomes ready to turn in, signaled by a green icon. After the player reports back, the system validates completion, gives rewards, updates the quest status to “completed,” shows a notification, and removes the quest from the active list while triggering completion events.



Picture 2.66. Quest Progress Tracking & Completion

NPC Pathfinding (A* Algorithm):

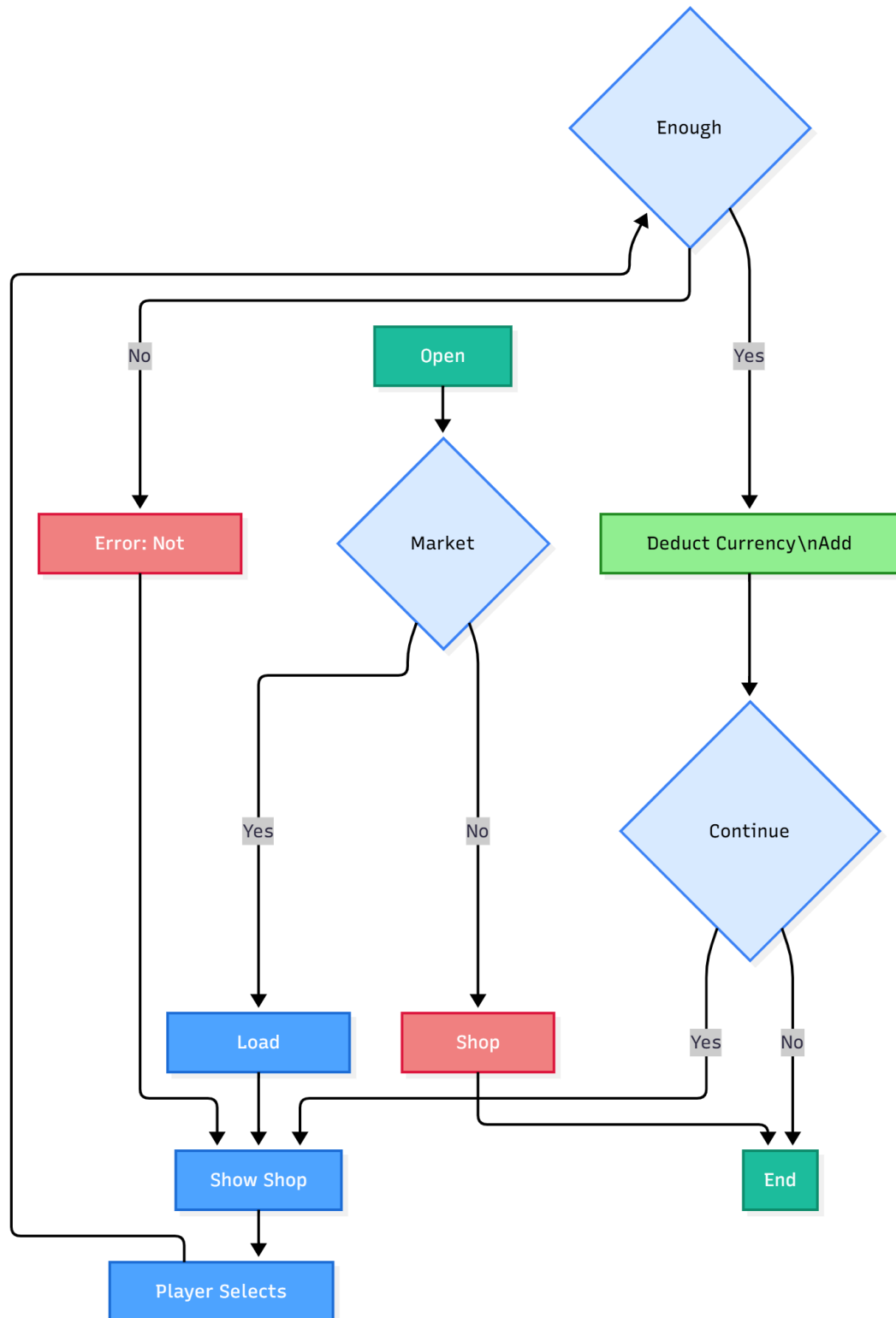
The A* system converts the target's world position into grid coordinates, then searches for the best path by expanding nodes with the lowest F-cost and evaluating walkable neighbors. When the target is found, it reconstructs a waypoint path and the NPC follows each waypoint, updating movement and animations until reaching the final destination.



Picture 2.67. NPC Pathfinding (A* Algorithm)

Shop Trading Transaction:

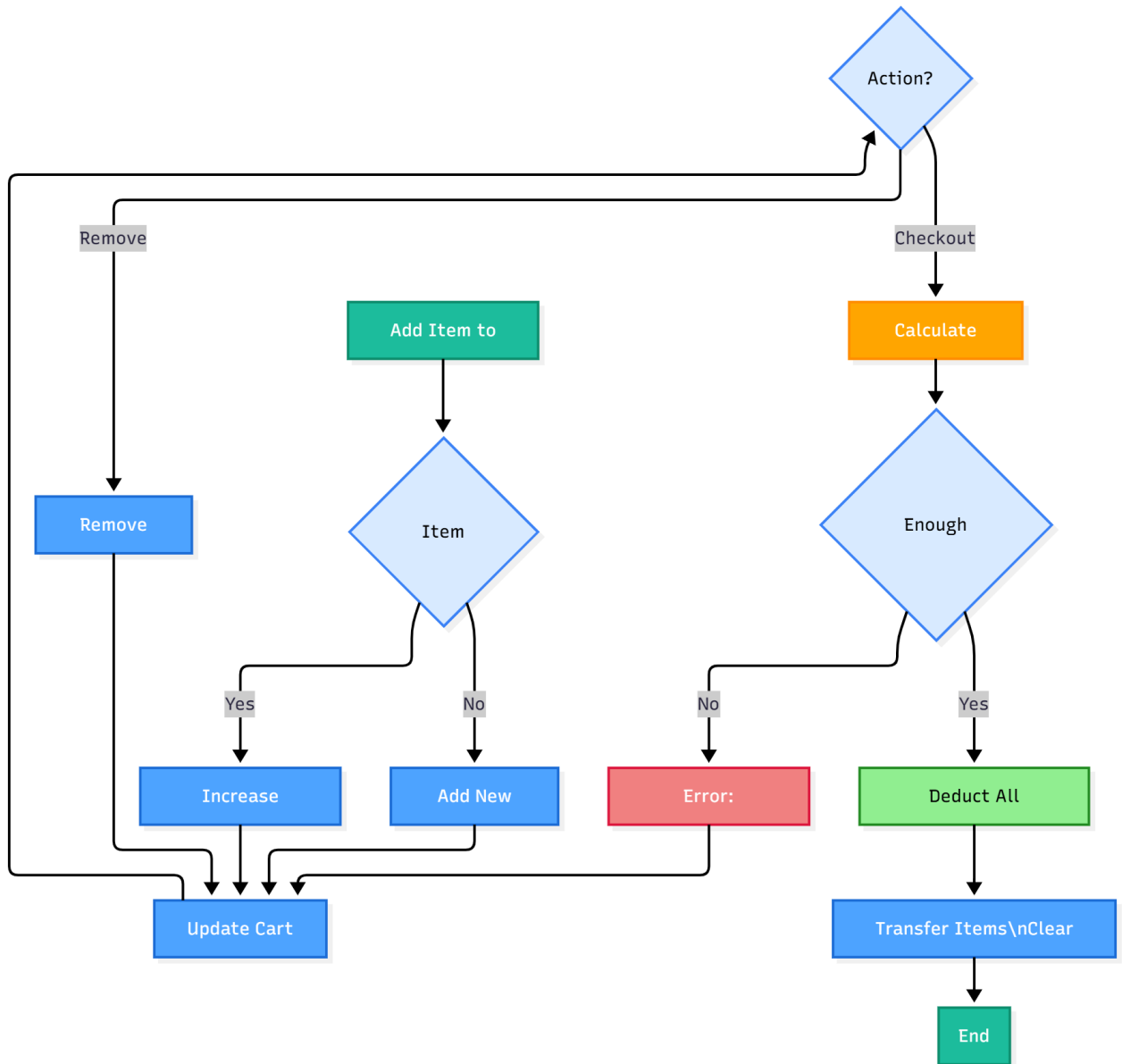
The shop system checks market hours, loads inventory, and displays the shop UI when player near. Players select an item, the system checks currency, performs payment, adds the item, and allows continued shopping or closing the shop.



Picture 2.68. Shop Trading Transaction

Shopping Cart System:

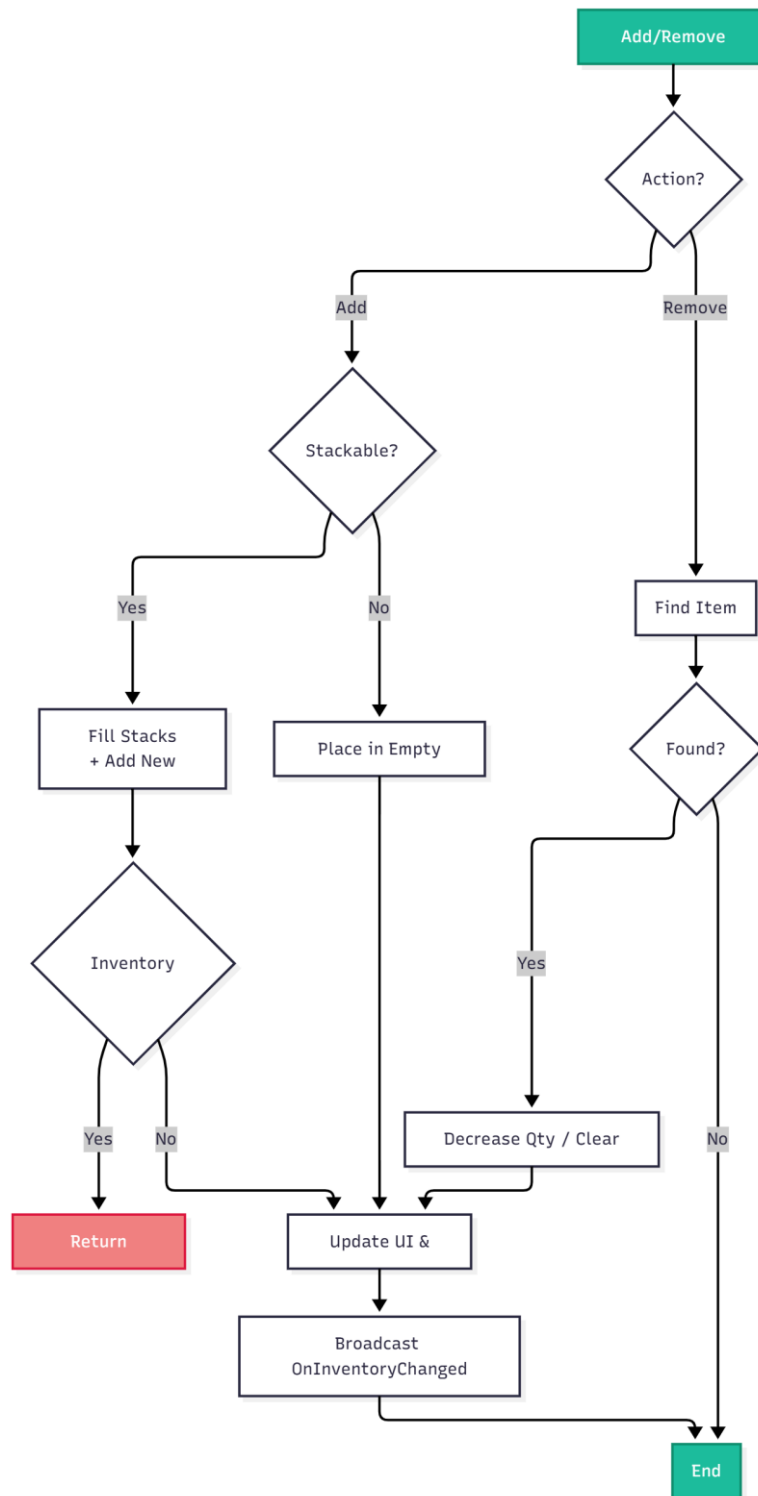
The shopping cart system stores selected items temporarily before purchase, allowing players to stack quantities, or remove items at any time. When the player proceeds to checkout, the system calculates the total cost across all selected items, verifies that the player has enough of each required currency type, and then processes the payment. After a successful purchase, the items are transferred to the player's inventory, the cart is cleared, and a success notification is shown.



Picture 2.69. Shopping Cart System

Inventory Add/Remove Item

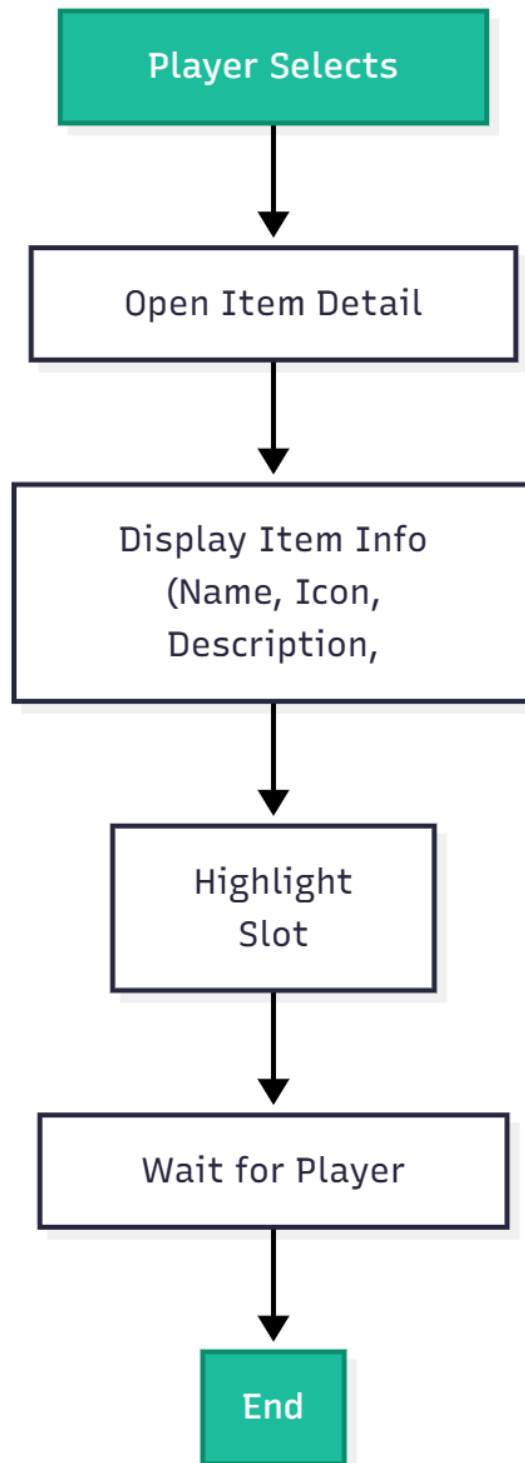
The inventory system manages item addition and removal depending on whether an item is stackable. Stackable items fill existing stacks before creating new ones, while non-stackable items use one slot each. If inventory becomes full, leftover items are returned. Removing items reduces stack quantities and clears slots when empty. After every change, the system updates the UI, saves to the database, and broadcasts an OnInventoryChanged event.



Picture 2.70. Inventory Add/Remove Item

Inventory – View Item Detail

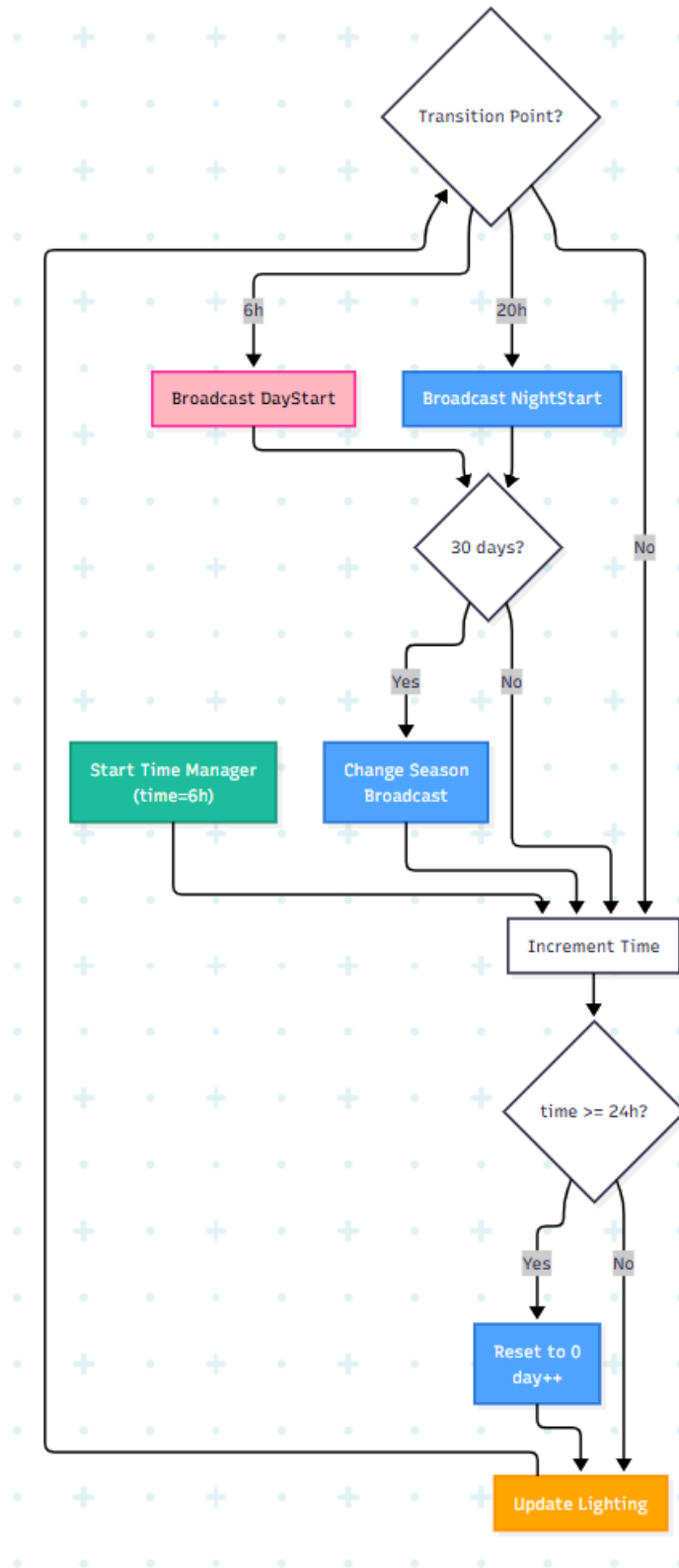
The item detail system activates when the player clicks on an item slot, opening the ItemDetailPanel to display the item's icon, name, description, stats, usable effects, and rarity. The selected slot is highlighted and a UI sound is played for feedback. The system then waits for the player to close the panel or perform another inventory action, without applying any item effects or modifying inventory data.



Picture 2.71. View Item Detail

Time Manager

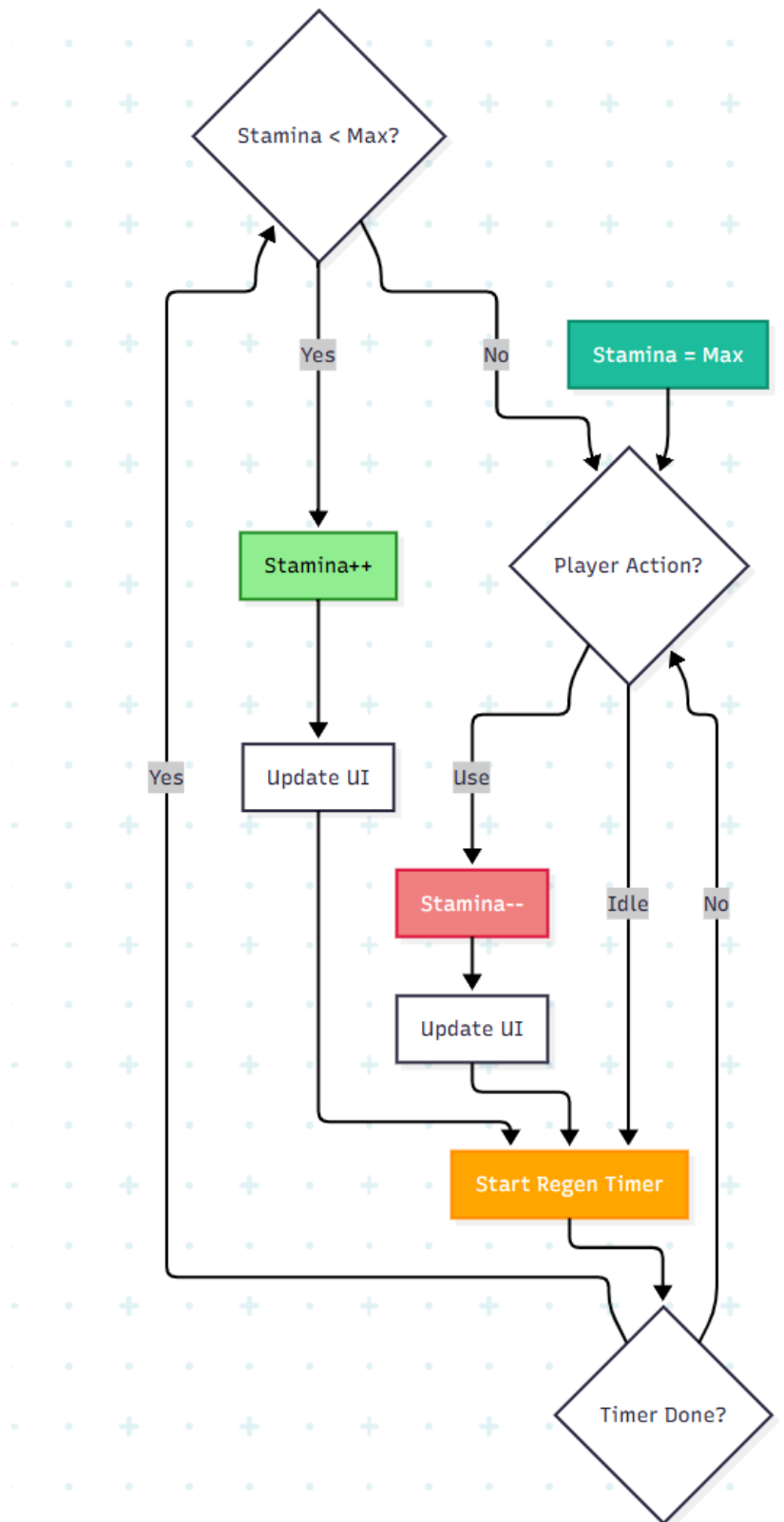
The time manager increases in-game time every frame, resets at 24 hours, updates global lighting based on hour, and triggers day/night events at 6h and 20h. Every 30 days, it switches seasons and broadcasts a season-change event so other systems can react.



Picture 2.72. Time Manager

Player Stamina

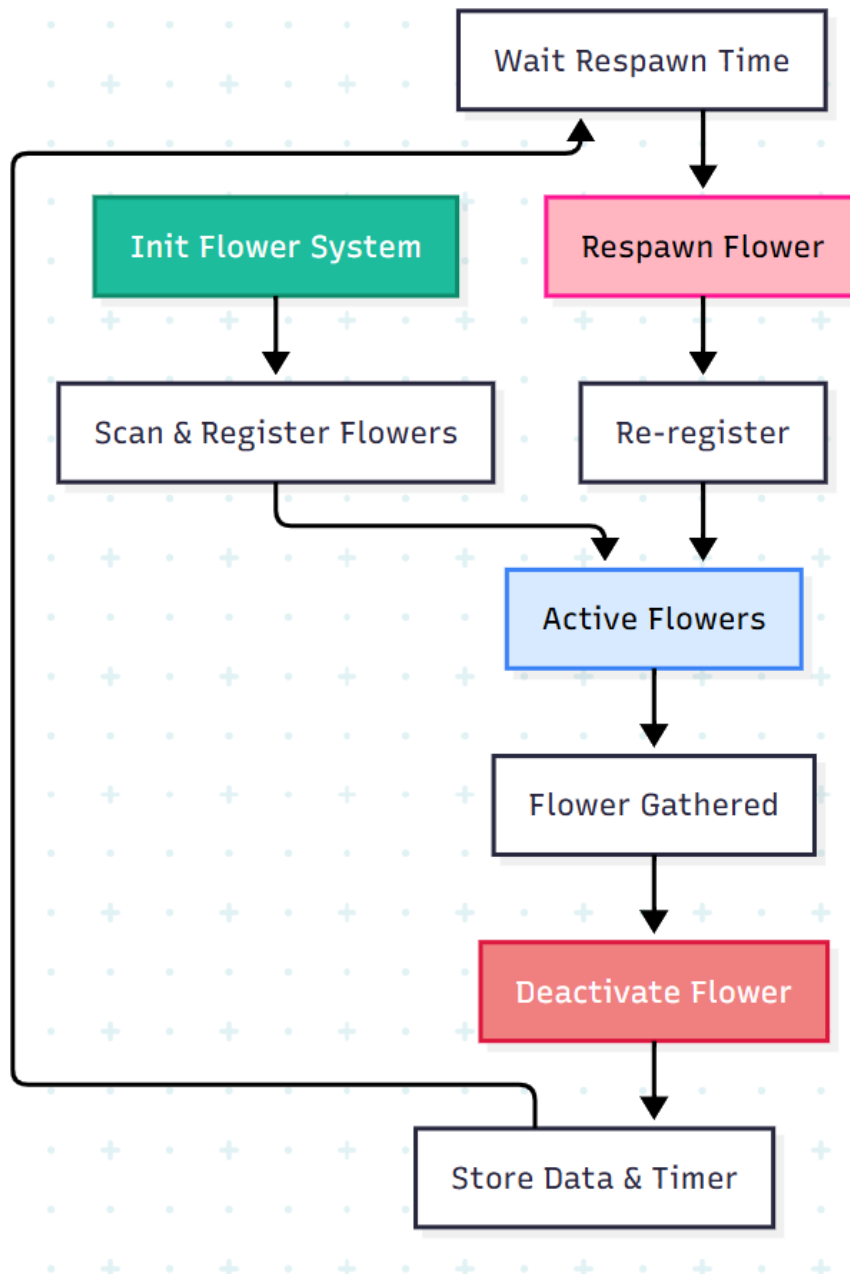
Stamina decreases when performing actions like sprinting or attacking, updates the UI instantly, and starts a regen delay. After 5 seconds of inactivity, stamina regenerates periodically until full, with UI icons updating in real-time to reflect recovery.



Picture 2.73. Player Stamina

Flower Respawn

All world flowers are registered at startup by type. When picked, the flower is deactivated, logged with timestamp, and queued for respawn. After a fixed delay, it respawns at the original position and is re-registered for NPC and player gathering



Picture 2.74. Flower Respawn

Other support features:

- Camera system that follows the main character in the game
 - Third-person perspective camera.
- Gold collection counter system.
- Health regeneration system when collecting items.
- Stamina regeneration system after delay.
- Slime system that attacks when close.
- GrapeShooter system that attacks with grape-shaped projectiles.
- Death effect for monsters.
- Graphic effect for wall collisions with destructible walls and arrows.
- Sound effect for monster, weapon, boss and environment
- Posting Processing: Set up color adjustments, metering, and effects for the camera.
- Background music for the game
- Menu Scene:
 - Play: Take player to the main game
 - Credit: A scene quoting the products along with the names of the contributors to the project.
 - Quit: Exit the game
- Pause Game Interface:
 - Press ESC to pause the game
 - Press ESC again to resume the game
 - Press F4 or the Quit Game button to exit the game
- Death effect for the character.
- Fade effect for scene transitions.
- Portal graphic effect for transitioning between scenes.
- AI system for monster and boss movement.
- Map Generation
- Data save and load
- Notification system

c. Instructions for using the functions of the components in the game

Movement Controls:

Use the arrow keys or [W], [S], [A], [D] to move the main character in the desired direction.



Picture 2.75. Player Move

The main character is equipped with a dodge function to help avoid attacks and chase enemies.

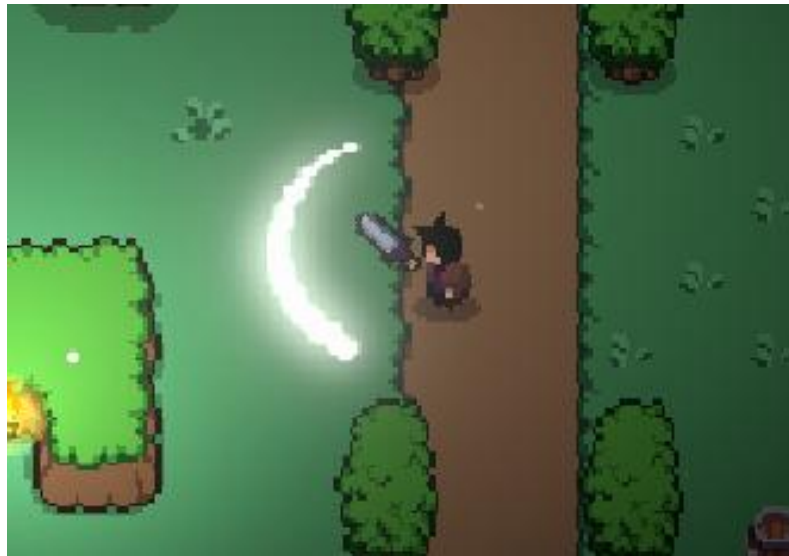
To dodge, simply press the [Space] key. However, this will consume energy, and you will need to defeat enemies, destroy structures, or wait for the cooldown to end before you can dodge again.



Picture 2.76. Player Dash

Weapon Controls: Sword, Magic Staff, and Bow:

Sword: Press the [1] key to select the sword and click the left mouse button while pointing to the target location to attack.



Picture 2.77. Player Sword Attack

Magic Staff: Press the [2] key to select the magic staff and click the left mouse button while pointing to the target location to shoot.



Picture 2.78. Player Staff Attack

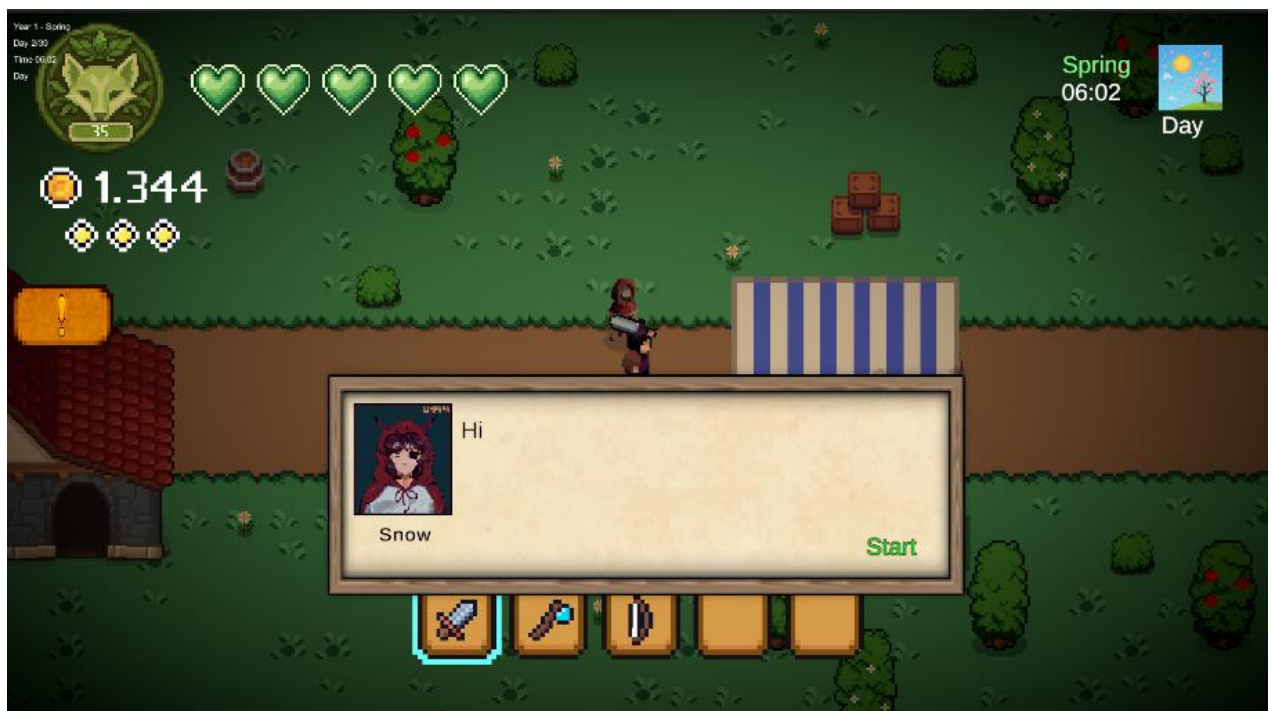
Bow: Press the [3] key to select the bow and click the left mouse button while pointing to the target location to shoot.



Picture 2.79. Player Bow Attack

NPC Interaction:

To interact with NPCs, approach the NPC, and the conversation will automatically display. Then press Start to open mic and start a conversation with NPCs with quest, ask for location,...

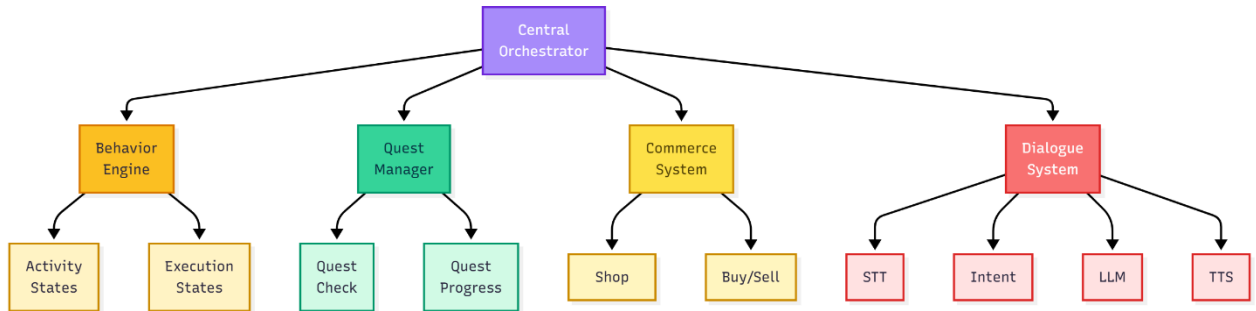


Picture 2.80. NPC interaction

CHAPTER 3: NPC AI SYSTEM

I. NPC AI Overview

The NPC Artificial Intelligence (AI) system is developed using a modular, component-based architecture that enables non-player characters to exhibit autonomous, believable behaviors while maintaining seamless integration with player-driven interaction systems. Each NPC is constructed from multiple specialized subsystems that cooperate to generate a coherent “living world” experience.



Picture 3.1. NPC AI Component Diagram

1.1. Core component:

The AI framework consists of four primary subsystems:

- **Central Orchestrator (NPC.cs)** – Coordinates high-level interactions, dialogue flow, and communication between all NPC sub-systems.
- **Autonomous Behavior Engine (NPCRoutineAI.cs)** – Controls daily routines, pathfinding, movement logic, and environmental interactions using coroutine-based scheduling.
- **Quest Lifecycle Manager (NPCQuestGiver.cs)** – Manages quest availability, quest progression, objective tracking, and player-specific branching outcomes.
- **Commerce System (NPCTrader.cs)** – Handles shop inventory, trading schedules, dynamic pricing logic, and in-shop interactions during market hours.
- **Dialogue System (NpcChatSpeaker.cs)** – Integrates the AI backend pipeline (STT, intent detection, LLM response, TTS) to process voice/text input and generate natural, context-aware replies.

1.2. Design Philosophy:

The system follows a “**Living World**” philosophy, in which NPCs behave as independent agents rather than static quest dispensers. This philosophy is carried out through four key principles:

- **Temporal Awareness** – NPCs follow a 24-hour in-game cycle, adjusting behaviors, dialogue, and locations based on the time of day and season.
- **Contextual Intelligence** – NPCs respond differently depending on their current activity, environmental context, relationship to the player, and ongoing quests.

- **Player-Centric Flexibility** – Although NPCs follow autonomous routines, players can interrupt or redirect them through natural interactions, ensuring a responsive and believable world.
- **Multi-Modal Interaction** – NPCs support voice interaction, text-based communication, and world/behavioral responses, enabling natural and immersive conversations.

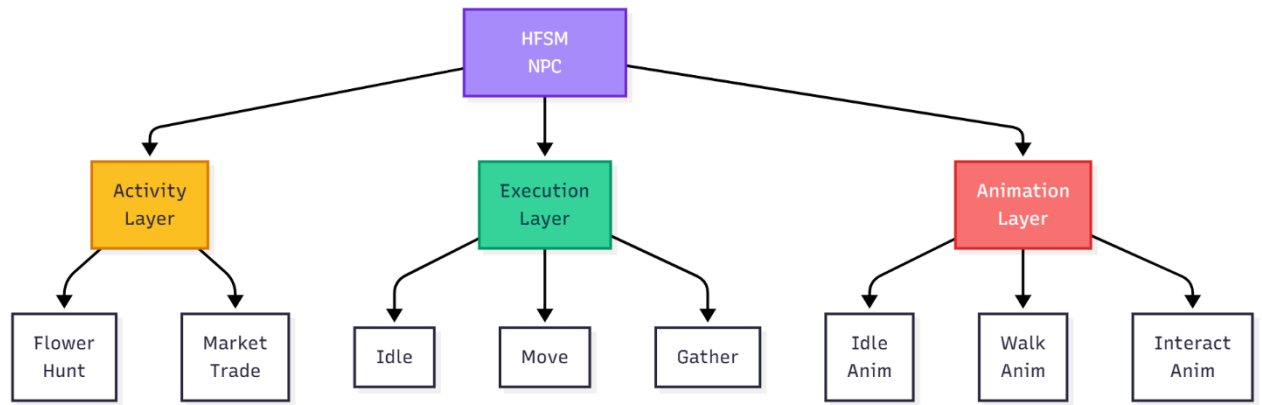
1.3. State Management Model

NPC behavior operates on a **three-layer hierarchical state machine (HFSSM)**:

- **Activity States – what the NPC is doing:**
These are high-level, schedule-driven behaviors that define the NPC's daily routine.

The system uses **nine activity states**:

- Sleep (23:00–06:00) (In-process)
- MorningRoutine (06:00–08:00) (In-process)
- MarketTrading (08:00–12:00)
- LunchBreak (12:00–13:00) (In-process)
- FlowerHunting (14:00–16:00)
- ExploreVillage (17:00–18:00) (In-process)
- EveningRoutine (18:00–20:00) (In-process)
- SocialTime (20:00–22:00) (In-process)
- NightRoutine (22:00–23:00) (In-process)
- **Execution States – how the NPC performs activities:**
These define moment-to-moment behaviors and low-level actions.
The system implements **six execution states**:
 - Idle
 - MovingToTarget
 - GatheringFlower
 - ReturningHome
 - Resting (In-process)
 - Socializing (In-process)
- **Animation States – visual representation:**
 - Idle animation
 - Walk animation
 - Gathering animation(In-process)
 - Talk animation(In-process)
 - Sleep animation(In-process)



Picture 3.2. HFSM Structure for NPC Behavior

II. Routine System Concept

The routine system provides each NPC with a coherent sense of autonomy—appearing to live independently from the player. This is achieved through time-based scheduling, dynamic priority management, and a robust override mechanism.

2.1. Time-Based Autonomy

NPCs follow structured daily routines governed by the global **TimeManager**, which controls:

- Day/night cycles (24-hour system compressed into real-time minutes)
- Seasonal transitions affecting dialogue and behavior
- Dynamic lighting synchronized with time progression
- Activity triggers based on the current in-game hour

The standard daily schedule is:

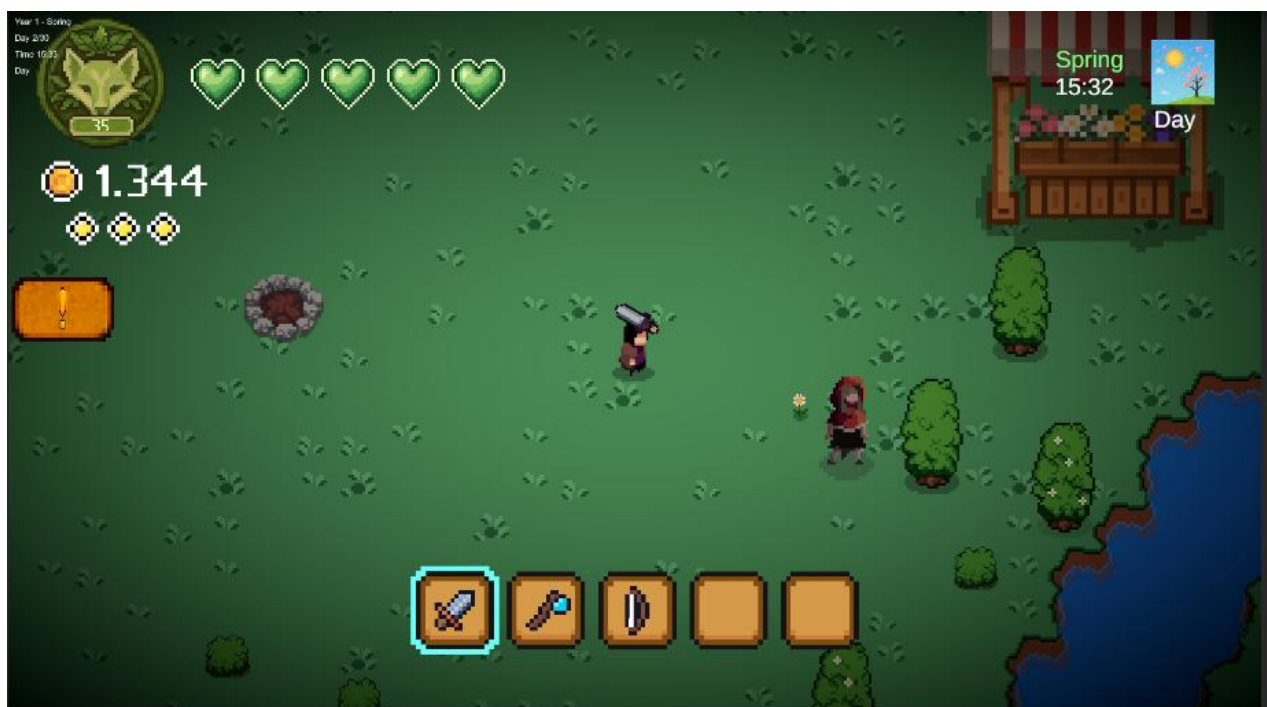
- 23:00–06:00 — Sleep (In-process)
- 06:00–08:00 — MorningRoutine (In-process)
- 08:00–12:00 — MarketTrading (merchant NPCs only)
- 12:00–13:00 — LunchBreak (In-process)
- 14:00–16:00 — FlowerHunting
- 17:00–18:00 — ExploreVillage (In-process)
- 18:00–20:00 — EveningRoutine (In-process)
- 20:00–22:00 — SocialTime (In-process)
- 22:00–23:00 — NightRoutine (In-process)

This schedule is dynamic and may be overridden by:

- Player requests
- Priority tasks (e.g., MarketTrading)
- Environmental events



Picture 3.3. Trading Time from 8:00 to 12:00



Picture 3.4. Flower Gathering Time from 14:00 to 16:00

2.2. Player Request Override System

One core innovation of the system is the **player request override**, which allows natural-language commands to temporarily override the NPC's daily routine.

Request Flow:

- The player asks a question or gives a command (e.g., “Can you gather some flowers for me?”).
- The Dialogue System interprets the request → outputs action (GATHER_FLOWER).
- Central Orchestrator sets **playerMadeRequest = true**.
- Behavior Engine prioritizes the request above scheduled activities.
- The NPC completes the request or stops on timeout.
- NPC returns to autonomous routine.

2.3. Activity Execution as Coroutines

All long-running behaviors (walking, gathering, trading, etc.) are implemented as coroutines. This allows:

- Non-blocking background execution
- Smooth transitions between activities
- Natural pause/resume handling
- Interruptions without destroying internal state

2.4. Pause / Resume Dialogue System

When the player approaches an NPC:

- Current activity is **paused non-destructively**
- NPC switches to **Idle execution state**
- Dialogue System takes control
- After conversation, routine resumes exactly where it stopped

This produces the impression that NPCs “pause their life” to speak with the player.

III. Routine System – Detailed Activity Breakdown

3.1. Flower Gathering Routine (14:00–16:00)

During flower-gathering hours—or when responding to a player request—the NPC executes a multi-stage pipeline:

- Detection Phase
 - Scans all objects tagged "Flower"
 - Queries FlowerManager for valid positions
 - Filters flowers outside bounds (0 → mapWidth/Height)
 - Filters flowers > 90 units from map center
 - Computes distance to each flower
 - Selects nearest available target

- Navigation Phase
 - Converts world → tile coordinates
 - Computes A* path using walkableGrid
 - Moves along path using Rigidbody2D.MovePosition
 - Avoids dynamic obstacles
 - Path is recalculated when obstruction occurs, limited to 4 retries.
 - Arrival is validated when the NPC enters a 0.45-unit radius around the target.
- Gathering Phase
 - Locks the flower target
 - Enters GatheringFlower execution state
 - Freezes NPC transform
 - Plays gathering animation for **3 seconds**
 - Ensures stable 2D plane position
 - Confirms distance remains < detectionRadius (3f)
- Completion Phase
 - Notifies FlowerManager to remove / respawn flower
 - Performs 1.5s post-action delay
 - Resets to Idle
 - Clears player request flag (if triggered by player)
 - Returns to routine loop

3.2. Market Trading Routine (08:00–12:00)

For NPCs marked as isTrader, MarketTrading involves:

- Pre-Market Setup
 - Validates marketStallLocation
 - Auto-discovers market if missing: tag = "Camp" → Market_0 → StandPoint
 - Generates stand point if absent
- Movement to Market
 - At 08:00, MarketTrading overrides all activities
 - Computes A* path → stall
 - Moves to position (distance < 0.45f)
 - Enters Idle execution state
- Market Operation (08:00–12:00)
 - Prevents displacement by players
 - Enables NPCTrader for shop UI
 - Trader remains stationary
 - Plays idle animation
- Player Interaction
 - Player enters NPC trigger collider
 - Dialogue System activates
 - Intent "trade" → Action: OPEN_SHOP
 - Loads item list from database
 - Purchases update:

- Player currency
- Trader stock
- Player inventory
- Market Close (12:00)
 - TimeManager triggers exit
 - Switches to next activity
 - Unlocks physics
 - NPC resumes autonomous routine



Picture 3.5. Market Trading Interaction

IV. AI-Driven Dialogue System

4.1. Multi-Modal Input Processing

The system supports both voice and text input via a hybrid processing pipeline.

- Speech-to-Text (STT)
 - Captures audio via Microphone API (16kHz mono)
 - Sends to STT service (Whisper / Cloud)
 - Returns text transcript to NPC.cs
 - Triggers dialogue flow: OnPlayerSpeechReceived(text)
- Text Input (Fallback)
 - Uses same processing pipeline
 - Useful for testing and accessibility

4.2. Intent Detection System

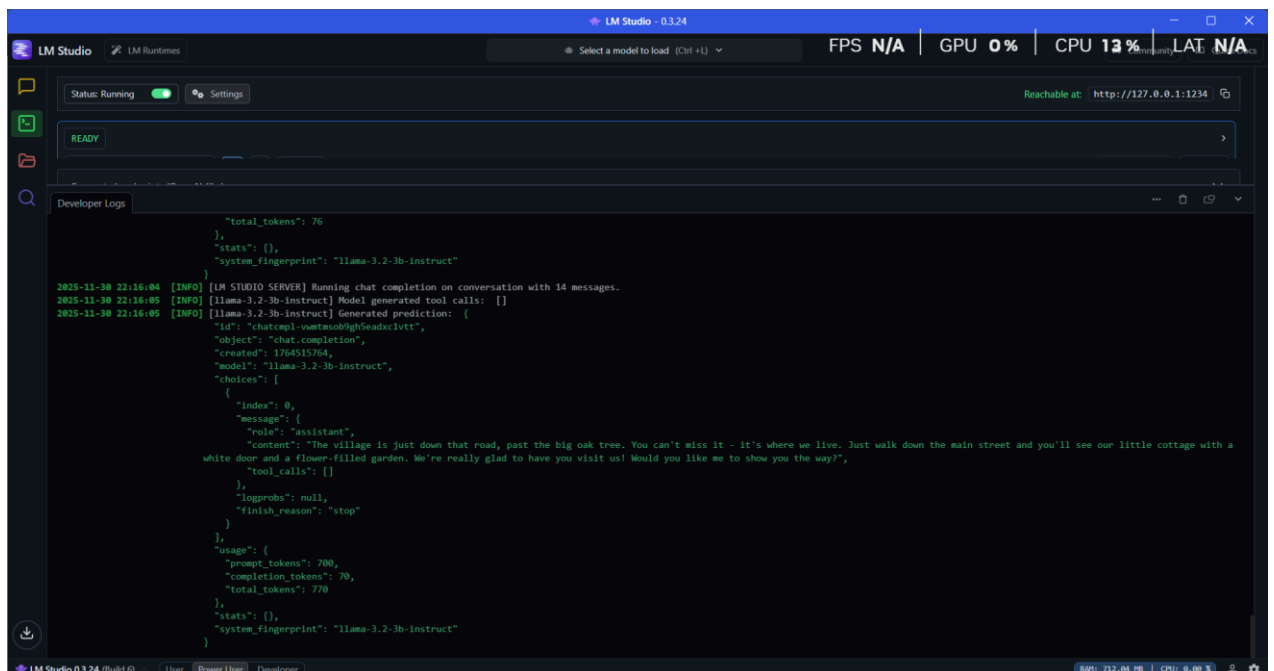
The system uses a **two-stage hybrid classifier**:

- Stage 1 — Semantic Similarity (Fast)
 - Encodes text via SentenceTransformer all-MiniLM-L6-v2
 - Computes cosine similarity
 - Accepts intent if similarity ≥ 0.55
- Stage 2 — LLM Fallback (Accurate)
 - Text passed to Llama 3.2 3B (local)
 - Prompt restricts output to known intent labels
 - Return intents: greeting, trade, quest_confirmation, gather_flower, etc.
- Benefits:
 - Fast GPU-free first pass (<10ms)
 - High-accuracy fallback.

4.3. LLM Processing

During the response generation stage, the system constructs a full conversational context for the language model. It first injects a system prompt that defines the NPC's personality, tone, and speaking style.

The prompt is then enriched with quest-related context, allowing the NPC to reference active objectives, requirements, or rewards. The NPC's current activity status is also included to ensure replies remain consistent with what the character is doing in the game world. Additionally, the system provides up to twenty turns of session history so the NPC can maintain continuity across the conversation. Using all of this information, the model generates a natural-language reply and then extracts emotion markers to guide the NPC's expressive behavior during the response.



Picture 3.6. LLM Response

4.4. Text-to-Speech and Action Mapping

The Text-to-Speech module generates spoken dialogue for NPC responses using Microsoft Edge-TTS with the JennyNeural voice, configured with a slightly increased pitch (+4 Hz) and a slower speech rate to enhance clarity and emotional expression.

The system produces an MP3 file and returns its URL to the Unity client. In parallel, the Action Mapping subsystem converts the detected intent into the appropriate in-game action, such as NAVIGATE, GATHER_FLOWER, OPEN_SHOP, or ACCEPT_QUEST_CONFIRM. These actions are then dispatched either to NPC-specific handlers or to global world controllers depending on their scope.

On the Unity side, the MP3 file is downloaded and played through an AudioSource, while the dialogue panel animates text in sync with the audio duration to maintain conversational naturalness. A turn-taking system ensures that the player and NPC do not speak simultaneously, preventing overlapping speech and preserving a smooth interaction flow.

```
[TTS] ✓ Saved MP3: tmp\tmp_89e4cbaaddea460bb4382d1546800685.mp3
127.0.0.1 - - [30/Nov/2025 22:05:52] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [30/Nov/2025 22:05:52] "GET /audio/tmp_89e4cbaaddea460bb4382d1546800685.mp3?t=639001119525265948 HTTP/1.1" 200 -
[DEBUG] Input: 'Where can I find the village' | Has quest context: False
[DEBUG] Sending to LM Studio: http://127.0.0.1:1234/v1/chat/completions
[DEBUG] Payload: {'model': 'Llama-3.2-3B-Instruct-GGUF', 'messages': [{'role': 'system', 'content': 'You are Snow, a gentle young girl in the country side. You are picking wildflowers in a sunny meadow, wearing a white dress. You are kind, soft-spoken, sometimes shy, but warm-hearted. Always reply as Snow, briefly and naturally. \nIMPORTANT: You MUST respond ONLY in English. Never use Vietnamese or any other language. All your responses must be in clear. \nNever include code blocks, JSON, or technical details. Speak like a person. \n\n[YOUR CURRENT STATUS] \nHiện tại tôi đang FlowerHunting. Giờ là 6.'}, {'role': 'user', 'content': '[intent=greeting] Hello hello hello how are you'}, {'role': 'assistant', 'content': 'I\'m so happy to meet you! *smiles shyly* I\'m doing very well, thank you for asking! The sun is shining and the wildflowers are blooming, it\'s a beautiful day in the meadow. *gently laughs* Would you like to help me pick some flowers?'}, {'role': 'user', 'content': '[intent=greeting] Hello how are you'}, {'role': 'assistant', 'content': 'I\'m good, thank you for asking! It\'s such a lovely day outside, isn\'t it? The flowers are so colorful and pretty. I was just picking some for my mom\'s birthday.'}, {'role': 'user', 'content': '[intent=ask_for_quest] Can I help you'}, {'role': 'assistant', 'content': 'That\'s so kind of you to offer... I really need your help, actually. My mom has been wanting me to decorate our little cottage with some fresh flowers, and I just can\'t seem to find enough. We only have a few already in the house. If I could collect 10 more from this lovely meadow... would you be willing to help me?'}, {'role': 'user', 'content': '[intent=ask_for_quest] Can I help you'}, {'role': 'assistant', 'content': 'That\'s so sweet of you! *smiles* I really do need your help with something. You see, my mom has been wanting me to decorate our little cottage with some fresh flowers, and I just can\'t seem to find enough. We only have a few already in the house. There are so many beautiful wildflowers here in the meadow that would be perfect for it. If you could help me collect 10 of them... I\'d be ever so grateful!'}, {'role': 'user', 'content': '[intent=quest_confirmation] OK sure'}, {'role': 'assistant', 'content': '*beams with a warm smile* Thank you so much! You\'re really too kind. I\'ll show you where the best wildflowers are. We can tie them together in my basket... *ties the flowers* There we are! All 10 of them. Thank you again for helping me out. I really appreciate it. Would you like to have a cup of tea with me and my mom? We\'d love to have some company.'}, {'role': 'user', 'content': '[intent=ask_direction] Where can I find the village'}}]
[DEBUG] LM Studio response status: 200
[TTS] ✓ Saved MP3: tmp\tmp_3778893160fc438b984ff0ebc86e3d19.mp3
127.0.0.1 - - [30/Nov/2025 22:16:08] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [30/Nov/2025 22:16:08] "GET /audio/tmp_3778893160fc438b984ff0ebc86e3d19.mp3?t=639001125688054600 HTTP/1.1" 200 -
```

Picture 3.7. NPC Dialog Conversation and Intent

CHAPTER 4: TESTING AND EVALUATION

I. TESTING OBJECTIVES

To ensure the game functions correctly and runs smoothly, comprehensive testing will be conducted across all implemented systems including NPC AI behaviors, dialogue interactions, quest mechanics, inventory management, combat systems, and environmental features. Performance testing will verify that the game maintains stable frame rates and efficient memory usage during gameplay, while functional testing will confirm that each system operates according to its designed workflow without errors. Integration testing will validate that different game components interact properly, such as NPCs communicating with the quest system or inventory updates syncing with the database.

The game will be considered ready when all critical bugs are resolved, performance targets are met, core workflows function without issues, and the overall gameplay experience is smooth and intuitive across different scenarios including edge cases and stress conditions..

II. TESTING PLAN

Scenario 1: Launch the game, access the Main Menu, and select Play:

The game launches correctly, the Main Menu loads without issues, and selecting “Play” transitions smoothly into the main scene with all objects rendered properly. Sword: Slash nearby slimes, bushes, crates and wooden barrels on the map. Player movement, dashing, and stamina behavior work as intended, including depletion, regeneration, and dash blocking when stamina is empty.

NPCs follow their full schedules: traders operate from 08:00–12:00, gatherers collect flowers from 14:00–16:00, and all NPCs transition between daily activities using A* pathfinding. Time progresses accurately with proper lighting transitions, and NPC activities update according to the current hour and season.

Dialogue triggers correctly when approaching NPCs, routines pause during conversation, and both voice and text input generate context-aware AI responses with synchronized audio. Quest-giver NPCs display correct markers, provide quest details through dialogue, track progress in real time, and distribute rewards upon completion.

Flowers can be collected by both players and NPCs, NPCs autonomously search and gather during scheduled hours, and flowers respawn after the configured delay. Trader NPCs appear at stalls during market hours, the shop UI opens via dialogue, purchases deduct the correct currency, and items are added to player inventory.

The cart displays selected items, updates quantities and totals correctly, validates currency, and completes checkout with proper inventory transfer. Items display with correct icons and stacking behavior, update instantly on add/remove, and respect inventory space limits.

All currency types are collected and converted correctly, displayed in the UI, and deducted properly during transactions. Dropped items such as coins, health, stamina, and XP are

collectable, magnetically attracted, and update player stats immediately.

Experience gains update accurately, level-ups trigger expected visual effects, and new stats and levels save correctly. All key data—including inventory, quests, currency, XP, and player state—saves and reloads correctly, with auto-save functioning during long sessions.

Map elements load properly, collision boundaries work, and NPC pathfinding avoids obstacles and respects map limits. Gameplay maintains stable framerate, UI transitions are smooth, and no crashes or memory issues occur during testing.

Scenario 2: Play and Pause:

During gameplay, verify that the player can pause the game. Confirm that the **PAUSE GAME** screen appears when the game is paused. Ensure that the game halts all actions while paused. Test if the **Quit Game** option on the pause screen functions correctly, allowing the player to exit the game.

Testing results according to the scenarios:

	Scenario 1	Scenario 2
Level of completion	Good	Good

Table 3.1. Table of the testing results according to the scenarios

Testing Environment:

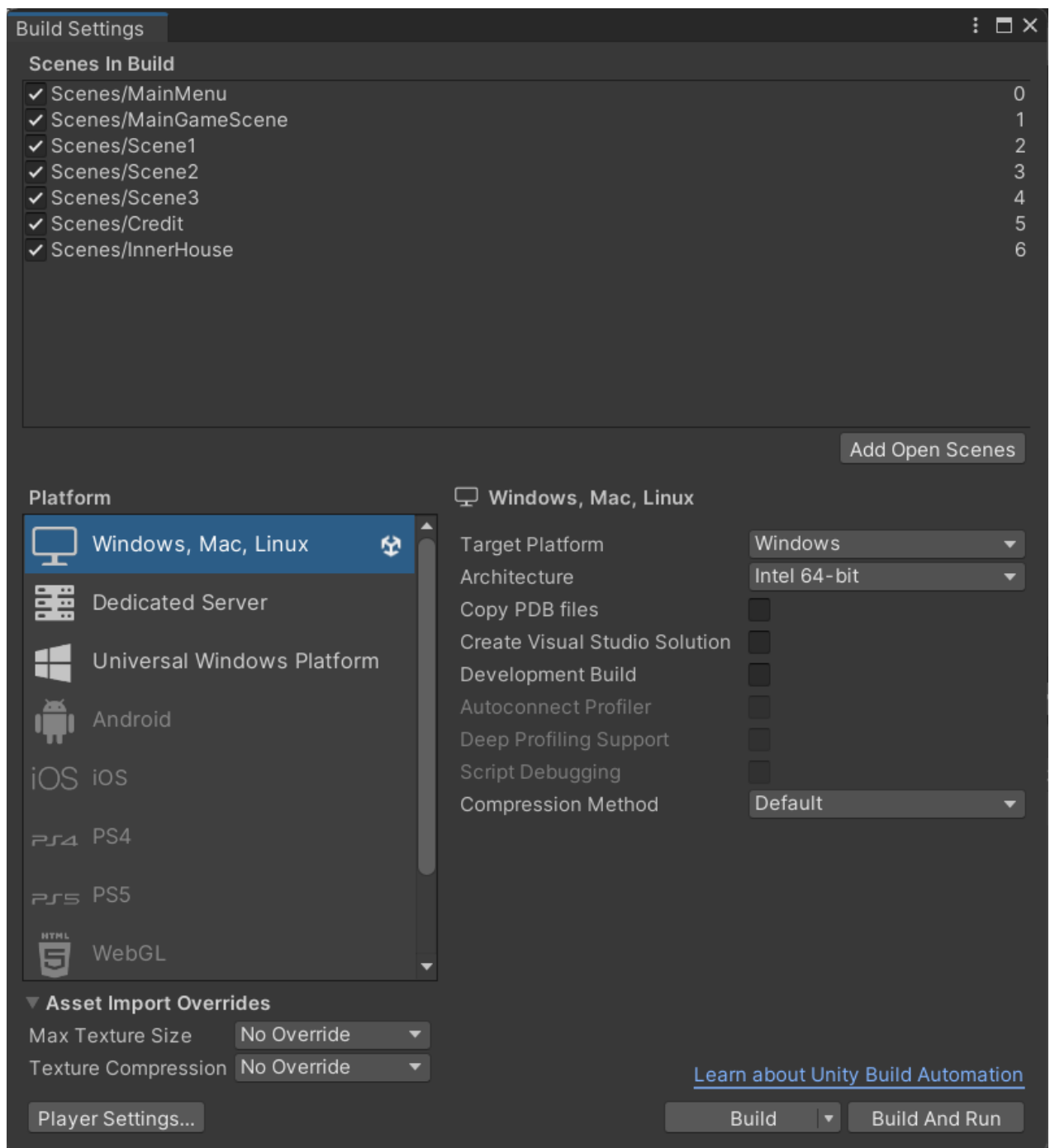
Device Name: Workstation Laptop - Acer Nitro 5

Specifications:

- **Processor:** Intel Core i5 (11th Gen Intel® Core™ i5-11400H @ 2.70GHz, up to 2.69 GHz)
- **RAM:** 16GB
- **Graphics Chip:** NVIDIA GeForce RTX 3050 Laptop GPU
- **Game Storage Requirement:** ~500MB
- **Operating System:** Windows 11
- **Game Performance Capability:** Excellent

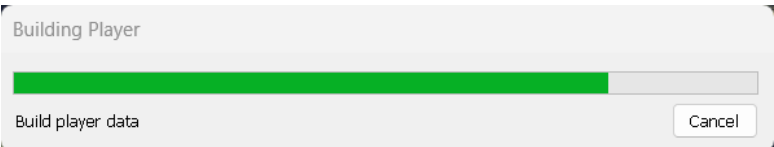
III. TESTING RESULT EVALUATION

a. Game Packaging Process:



Picture 4.1. Build Game window and setting of Unity

The game was packaged in Unity with default settings, and packaged to run on the Windows platform.



Picture 4.2. Begin to build game

The packaging process may take between 4 to 8 minutes depending on the computer's configuration and processing speed.

The game, after being packaged into the folder, will include the following files and directories:

Name	Date modified	Type	Size
AlexTheWanderer_BurstDebugInformatio...	11/29/2024 11:22 AM	File folder	
AlexTheWanderer_Data	11/29/2024 10:47 AM	File folder	
MonoBleedingEdge	11/28/2024 8:29 PM	File folder	
AlexTheWanderer.exe	11/28/2024 8:29 PM	Application	651 KB
UnityCrashHandler64.exe	11/28/2024 8:29 PM	Application	1,158 KB
UnityPlayer.dll	11/28/2024 8:29 PM	DLL File	30,092 KB

Picture 4.3. The files and directories of the game after packaged

b. Testing Results:

Game Menu Interface:



Picture 4.4. Game Menu Interface

Pause interface when the player presses the [ESC] key:



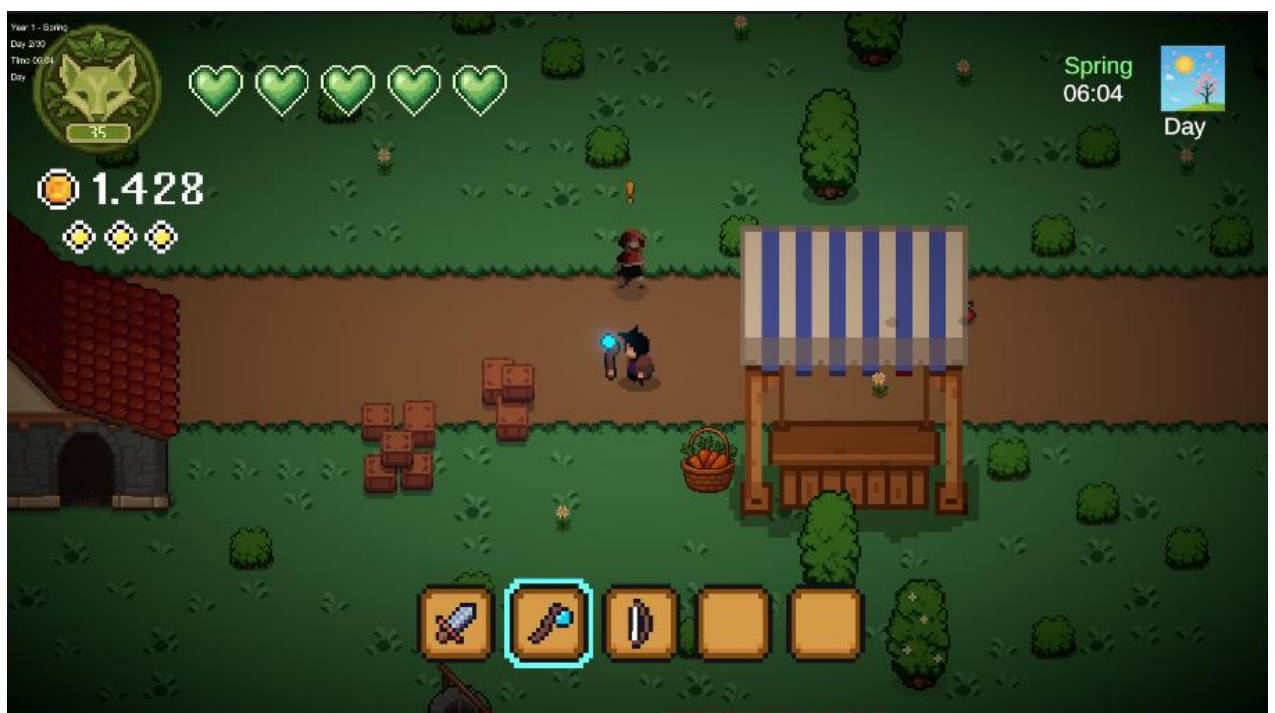
Picture 4.5. Pause Game Interface

Starting the game:



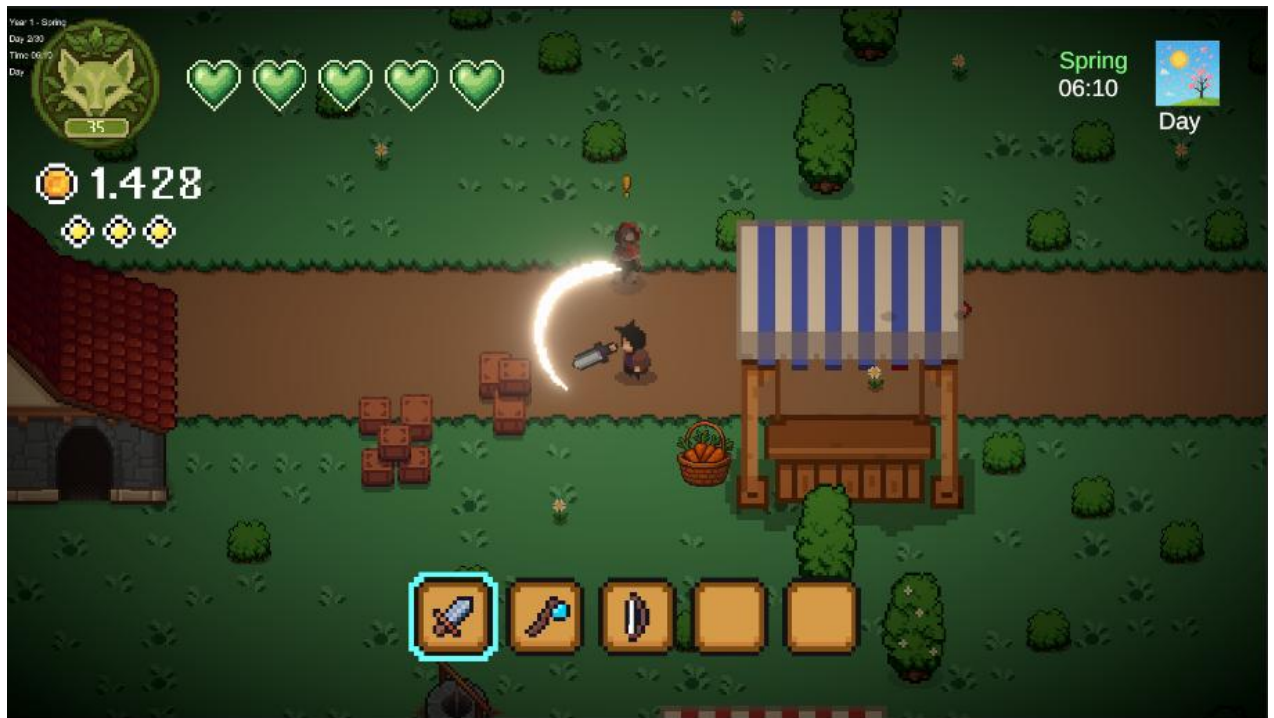
Picture 4.6. Game Scene starting the game

Weapon Change:



Picture 4.7. Change Weapon

Sword Slash Effect:



Picture 4.8. Sword Slash Effect

Magic Laser Blast Effect:



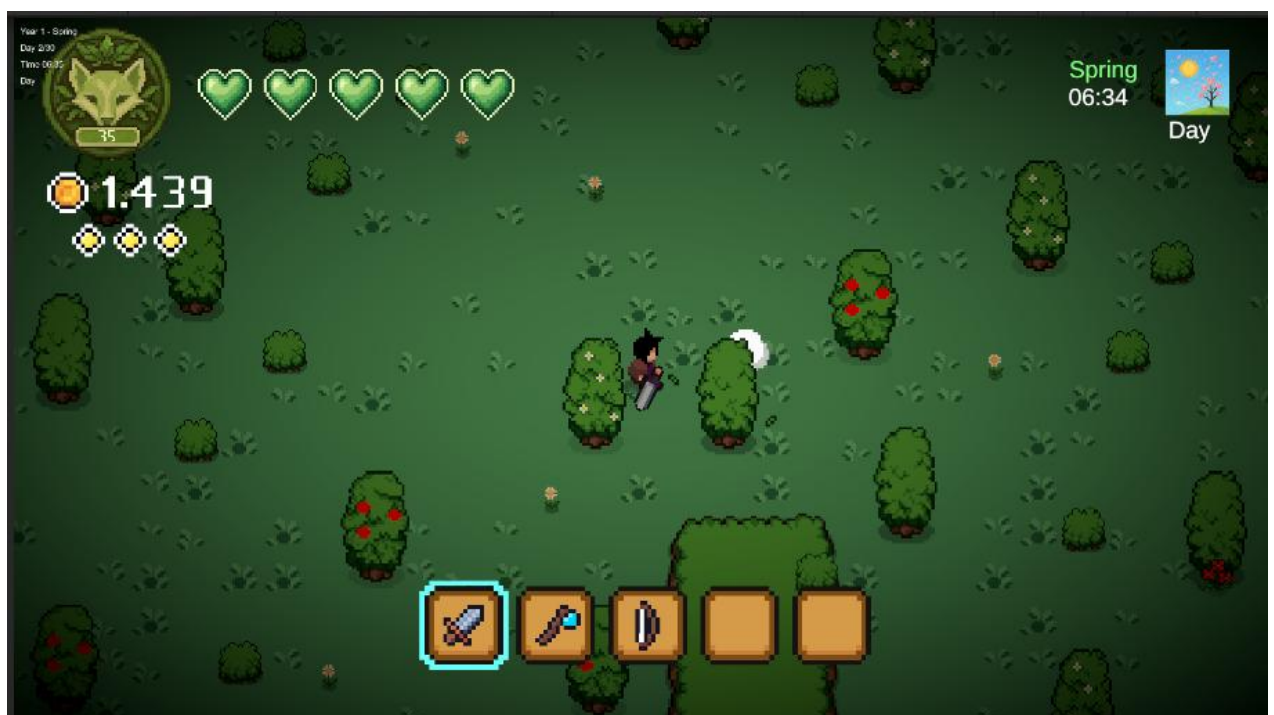
Picture 4.9. Staff Laser Effect

Bow Shooting Effect:



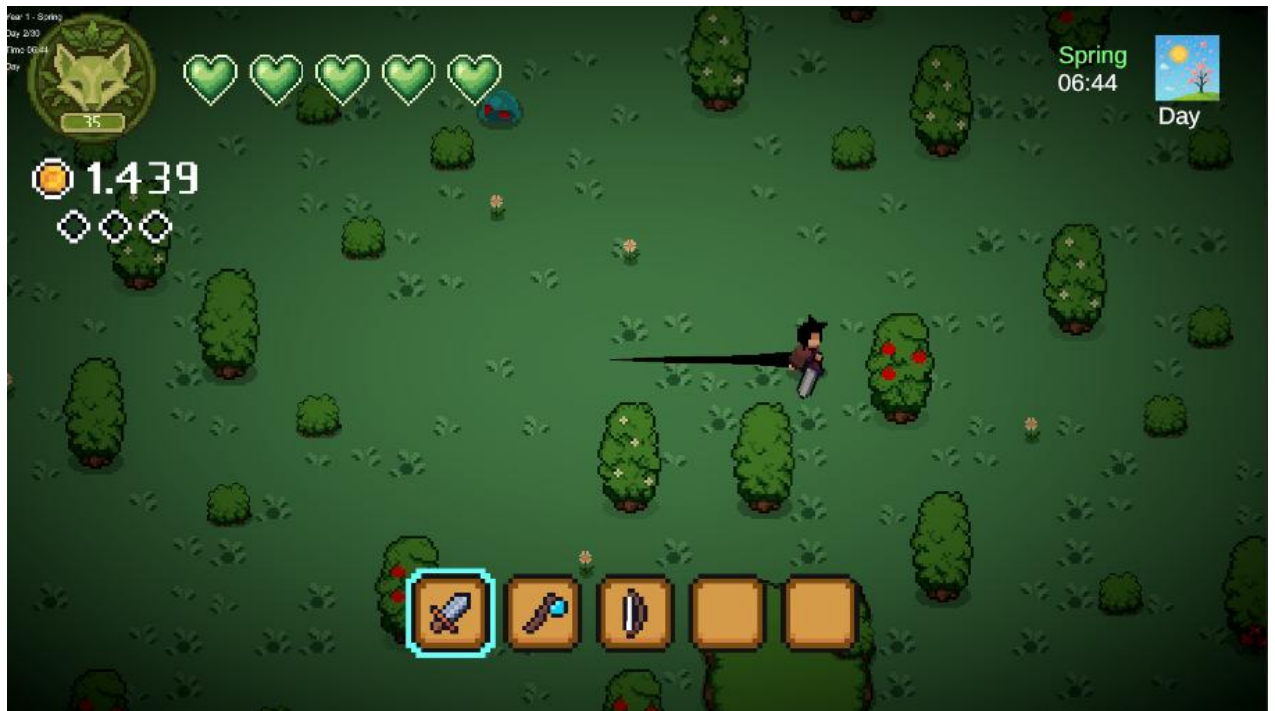
Picture 4.10. Arrow Effect

Knockback Effect:



Picture 4.11. Knockback and Flash Effect

Dash Effect and Stamina:



Picture 4.12. Dash Effect and Lose Stamina

Structure Destruction Effect:



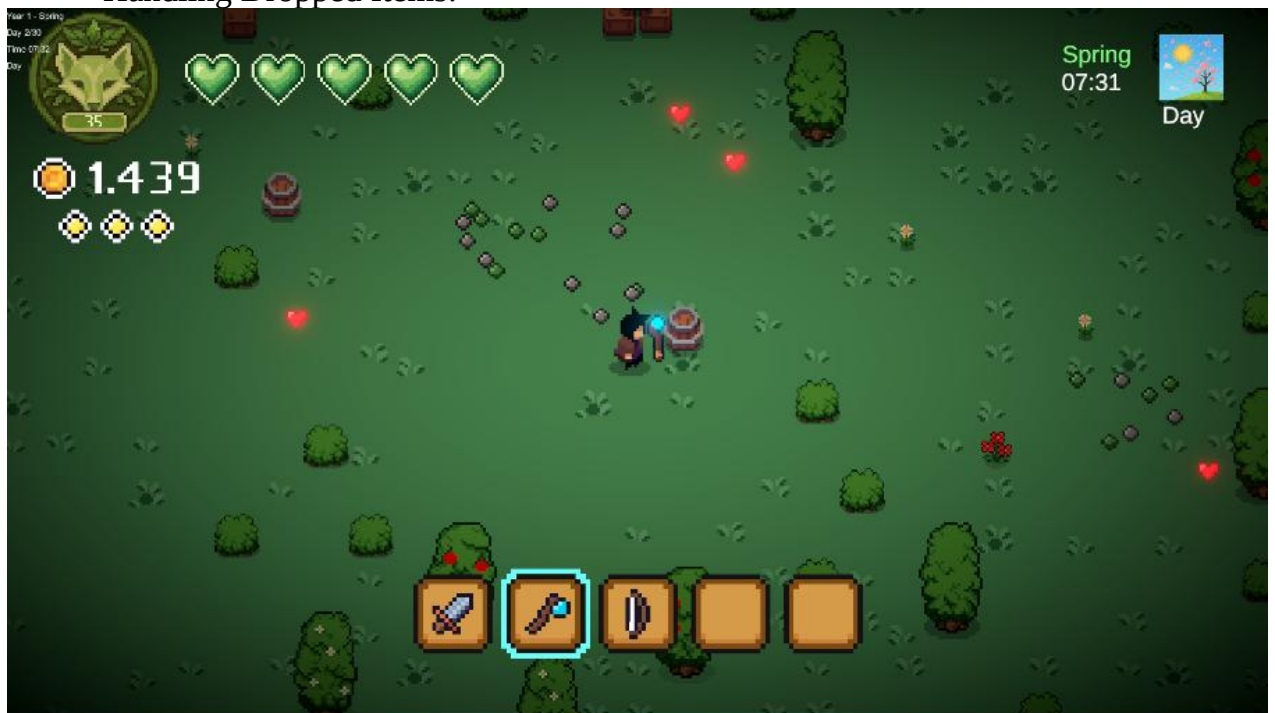
Picture 4.13. Structure Destruction

Handling Enemy Death:



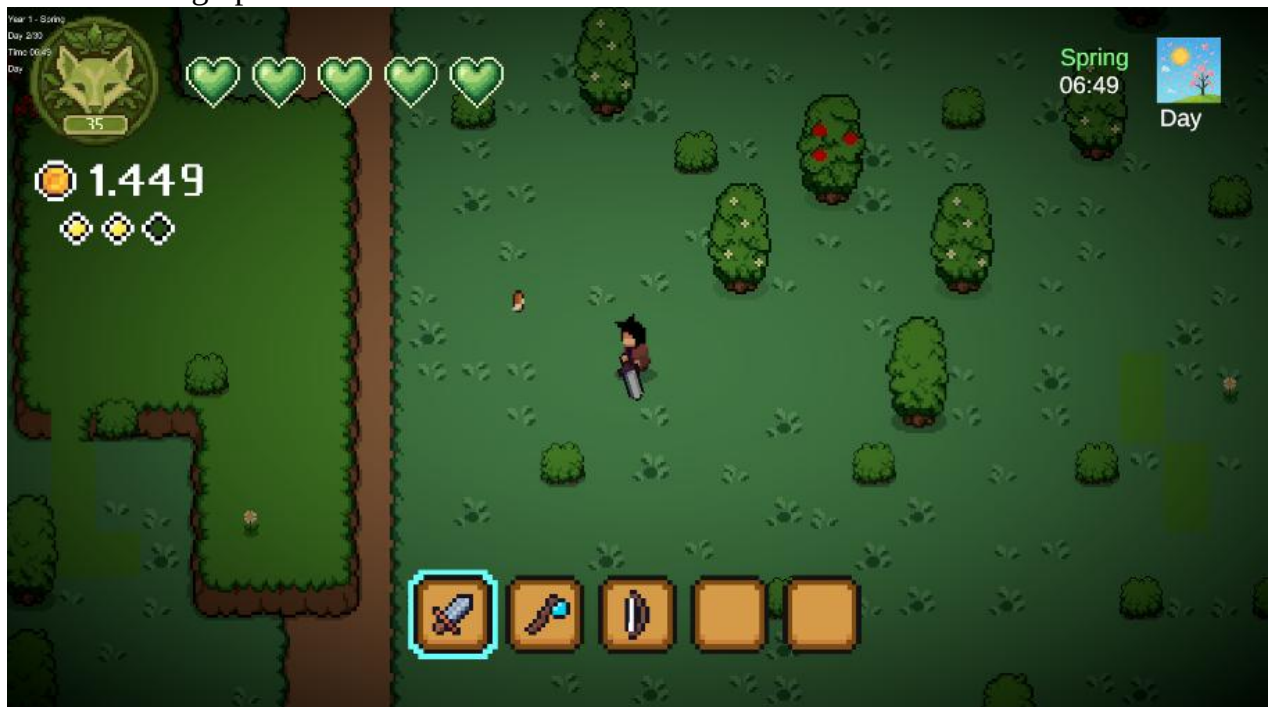
Picture 4.14. Enemy Death

Handling Dropped Items:



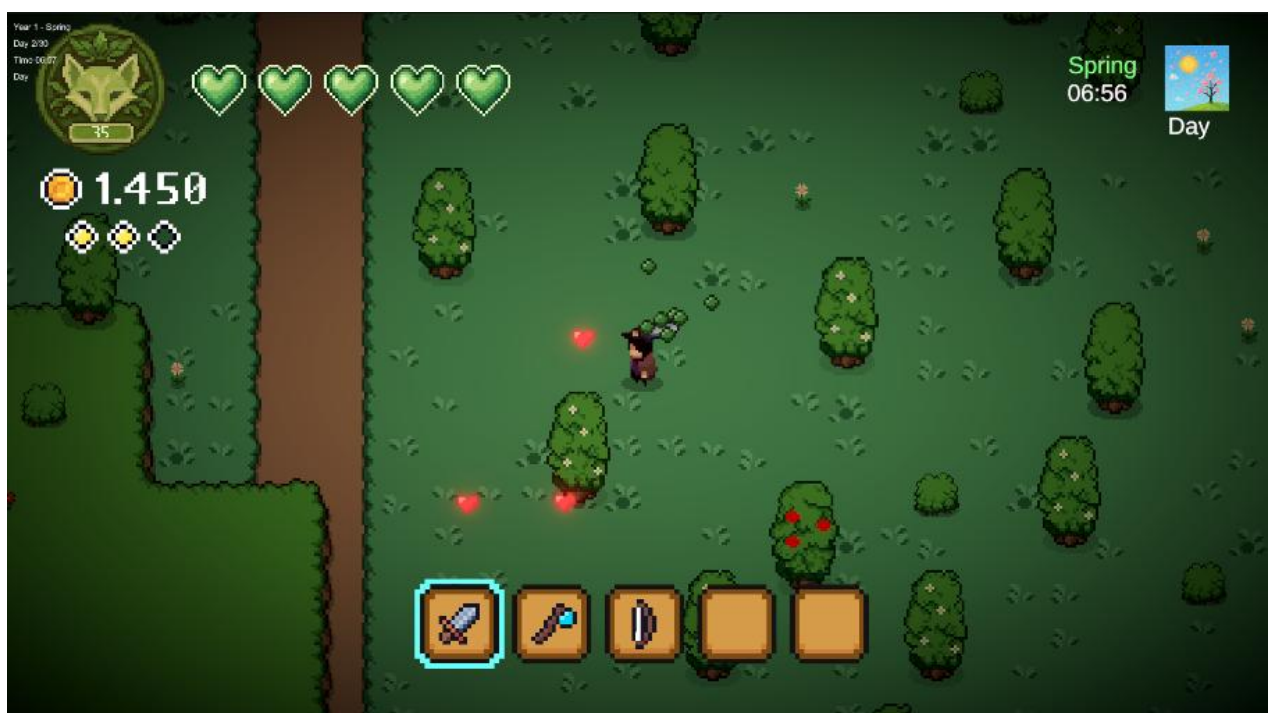
Picture 4.15. Dropped Items

Picking up Coins:



Picture 4.16. Picking up Coins

Picking up EXP and Health:



Picture 4.17. Picking up EXP and Health

Attacked Effects:



Picture 4.18. Attacked Effect

Death Handling when Health Depletes:



Picture 4.19. Death Effects

Player Inventory:



Picture 4.20. Player Inventory

Items Detail:



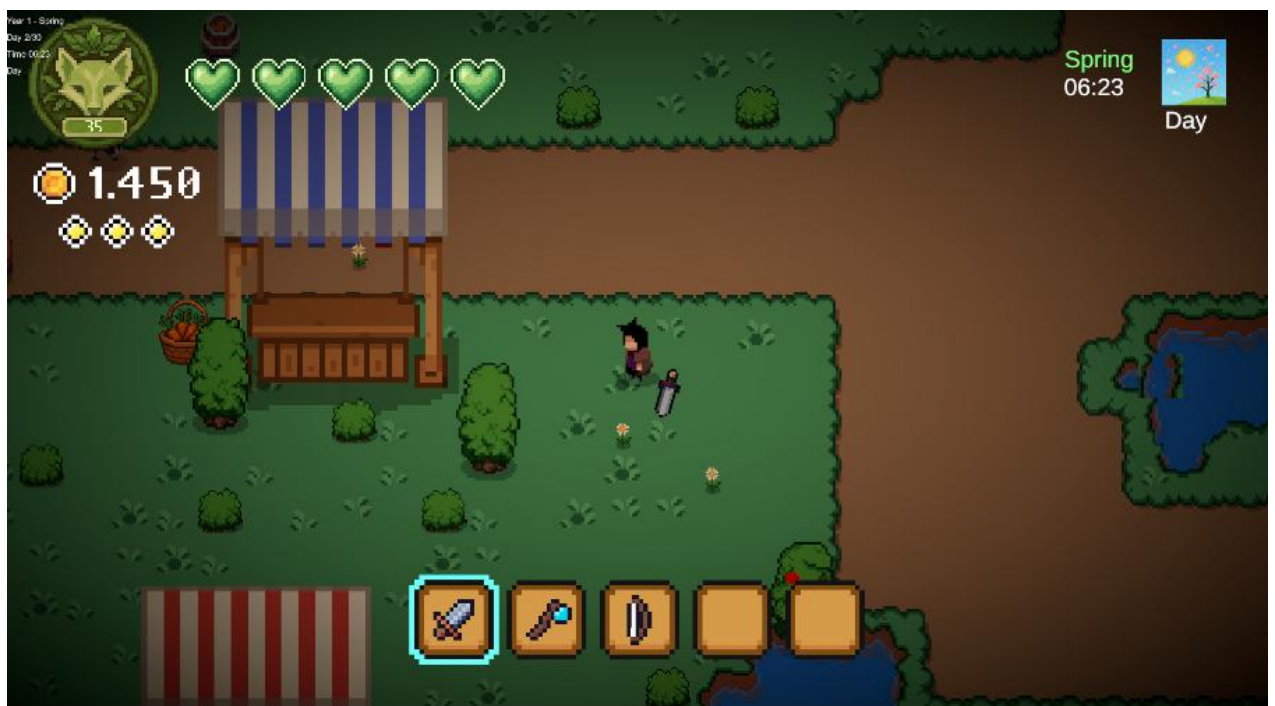
Picture 4.21. Items Detail

Players Coins Bag:



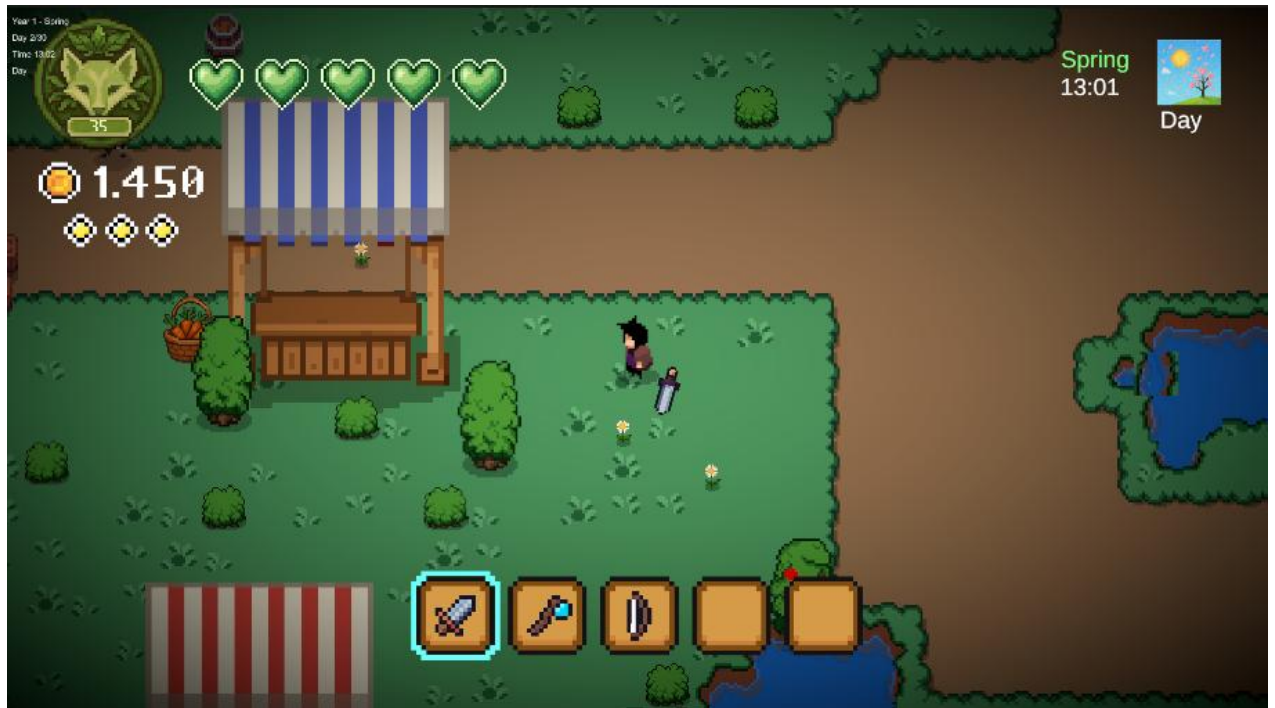
Picture 4.22. Coins Bag

Time Morning Light:



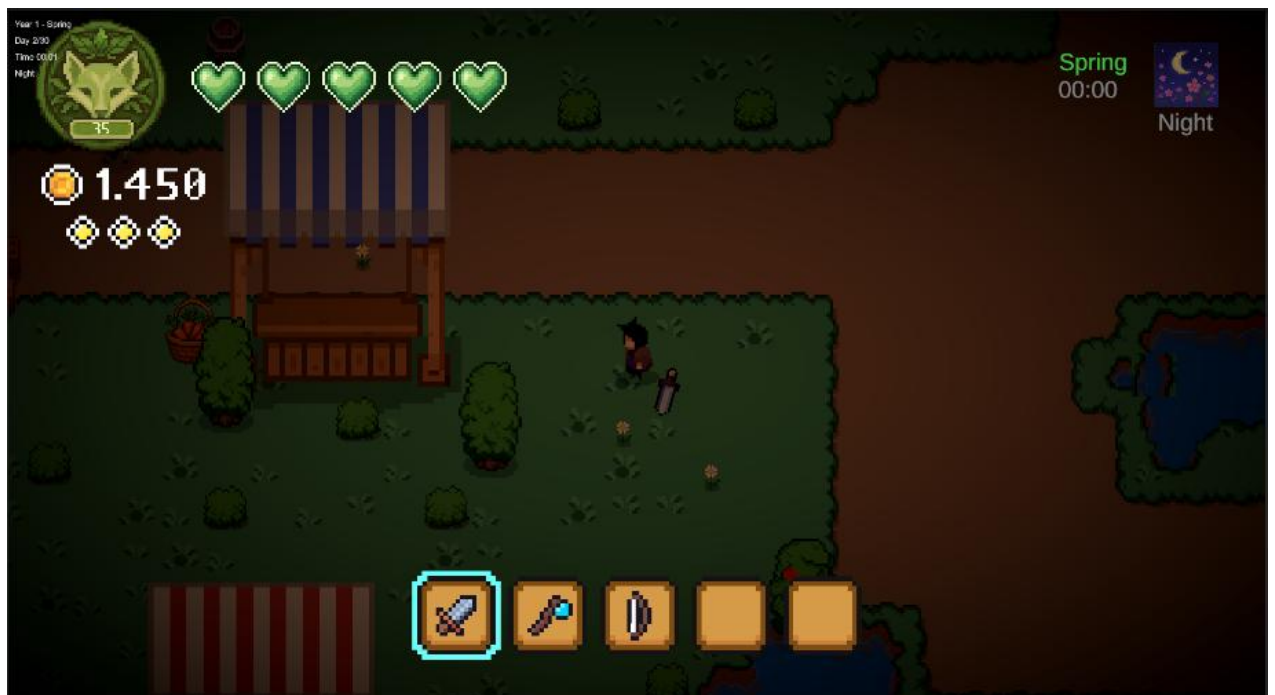
Picture 4.23. Time Moring Light

Time Midday Light:



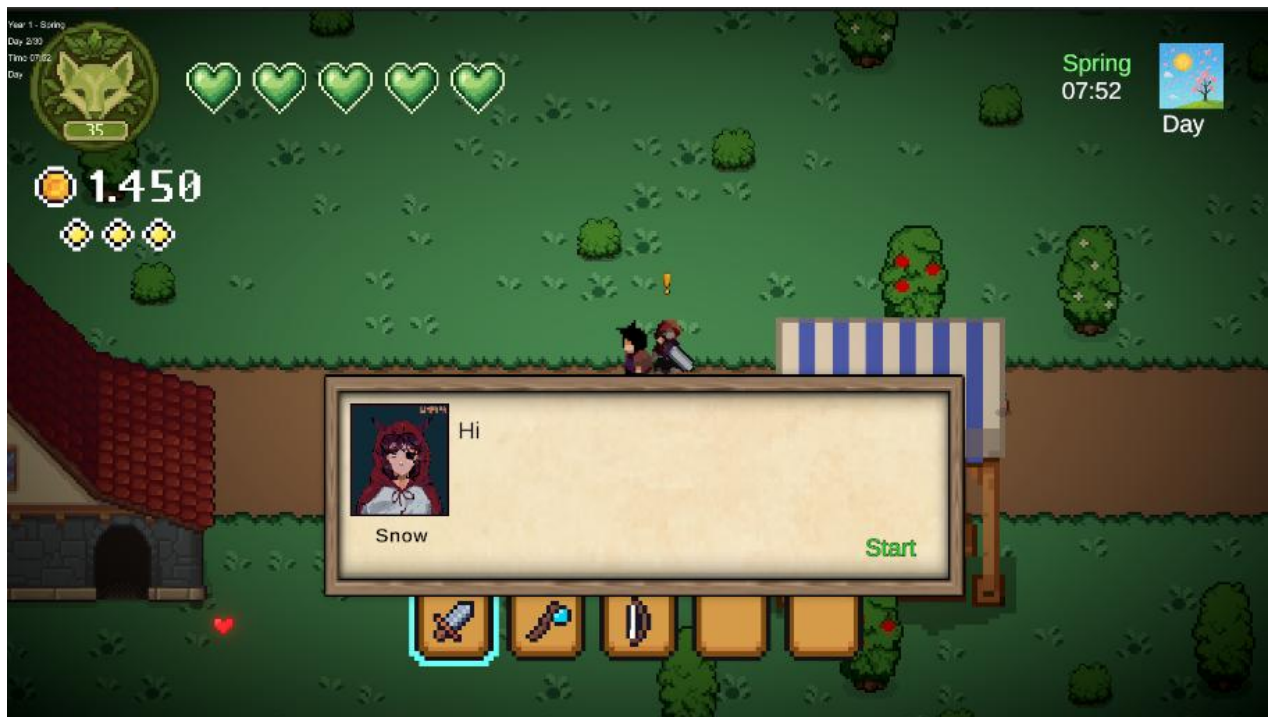
Picture 4.24. Time Midday Light

Time Night Light:



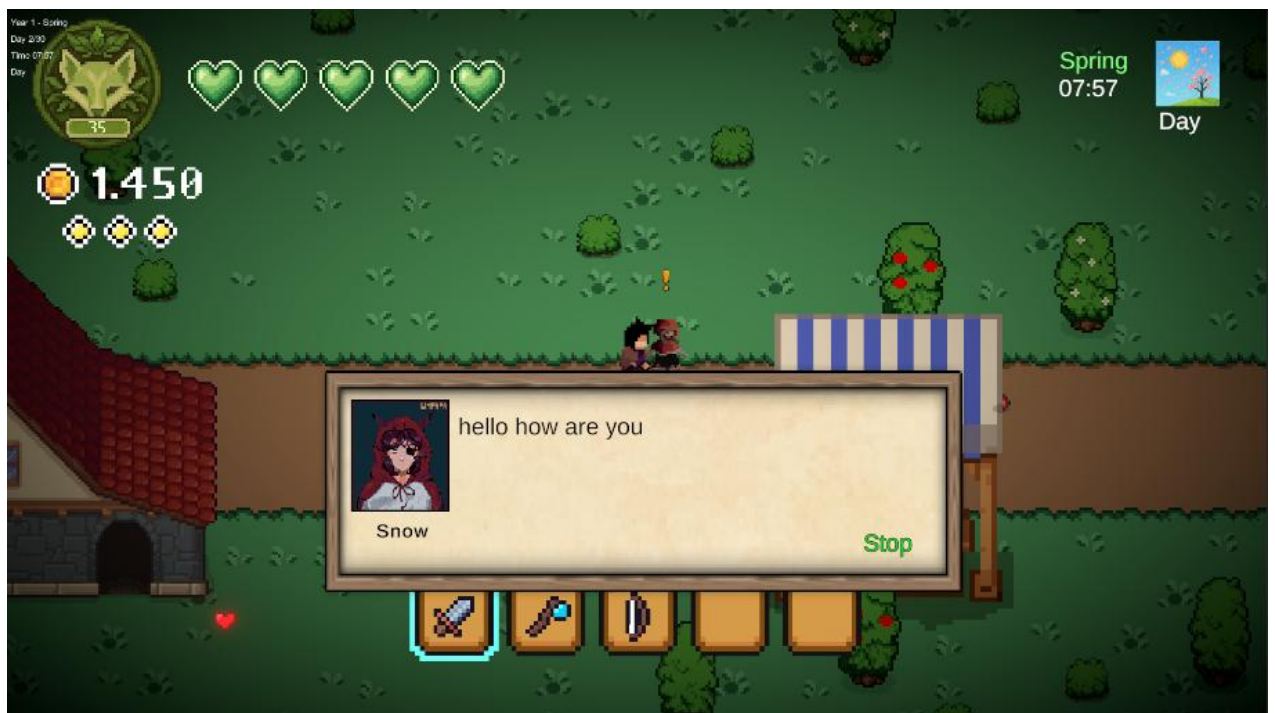
Picture 4.25. Time Night Light

NPC Interaction Effects:



Picture 4.26. NPC Dialogue

Players speak to NPC:



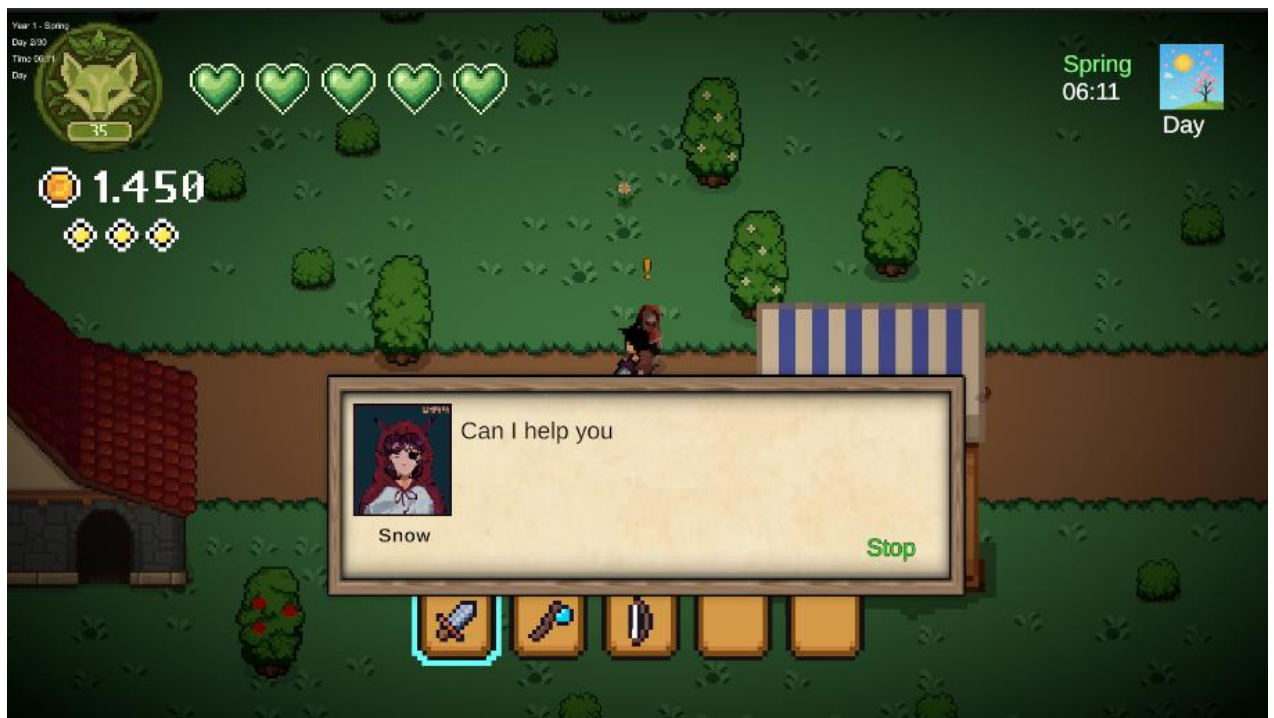
Picture 4.27. Players Speak

NPC response to Players:



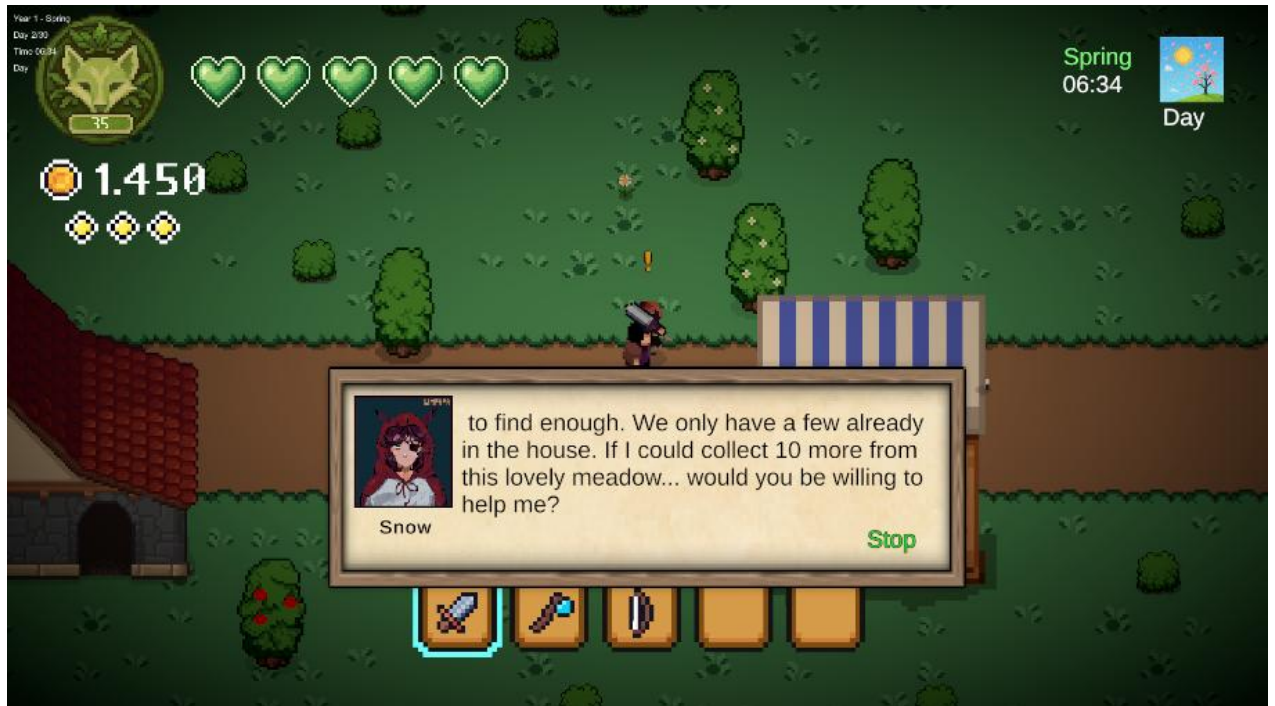
Picture 4.28. NPC Reply

Ask NPC for Quest:



Picture 4.29. Ask NPC for Quest

NPC reply with Quest content:



Picture 4.30. Reply with Quest content

Player accept the Quest:



Picture 4.31. Player accept the Quest

Trigger the Quest Panel and Give the quest detail to Player:



Picture 4.31. Quest Panel List



Picture 4.33. Quest Detail Panel

Quest update progress when player collects the right Items



Picture 4.34. Quest Progress Updated

Quest Complete:



Picture 4.35. Quest Complete

Player report the Quest to NPC



Picture 4.36. Quest Report

Reward Updated and Remove Quest Pending List



Picture 4.37. Quest Reward

NPC open the Market shop at 8:00



Picture 4.38. Shop Opened

Shop Items, Inventory and Price:



Picture 4.39. Shop UI

Update Shopping Cart and Payment Panel:



Picture 4.40. Buy some Flowers

Pay by Player Coins:



Picture 4.41. Pay Coins

Pay not enough money:



Picture 4.42. Not Enough Money Error

Pay Enough Money Valid Payment and change the extra coin:



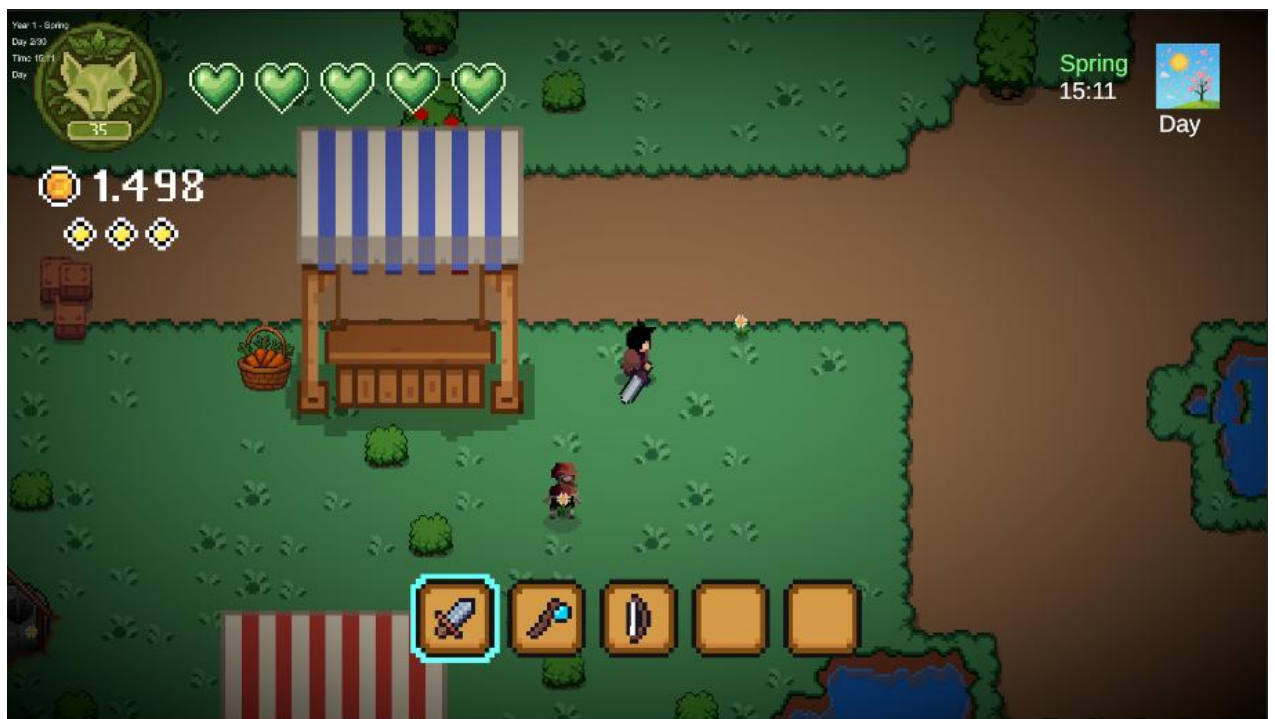
Picture 4.43 Valid Payment

Items Updated in Player Inventory:



Picture 4.44. Items Updated

NPC Flower Gathering at 15:00:



Picture 4.45. NPC Flower Gathering Routine

Player ask NPC for the Village Position:



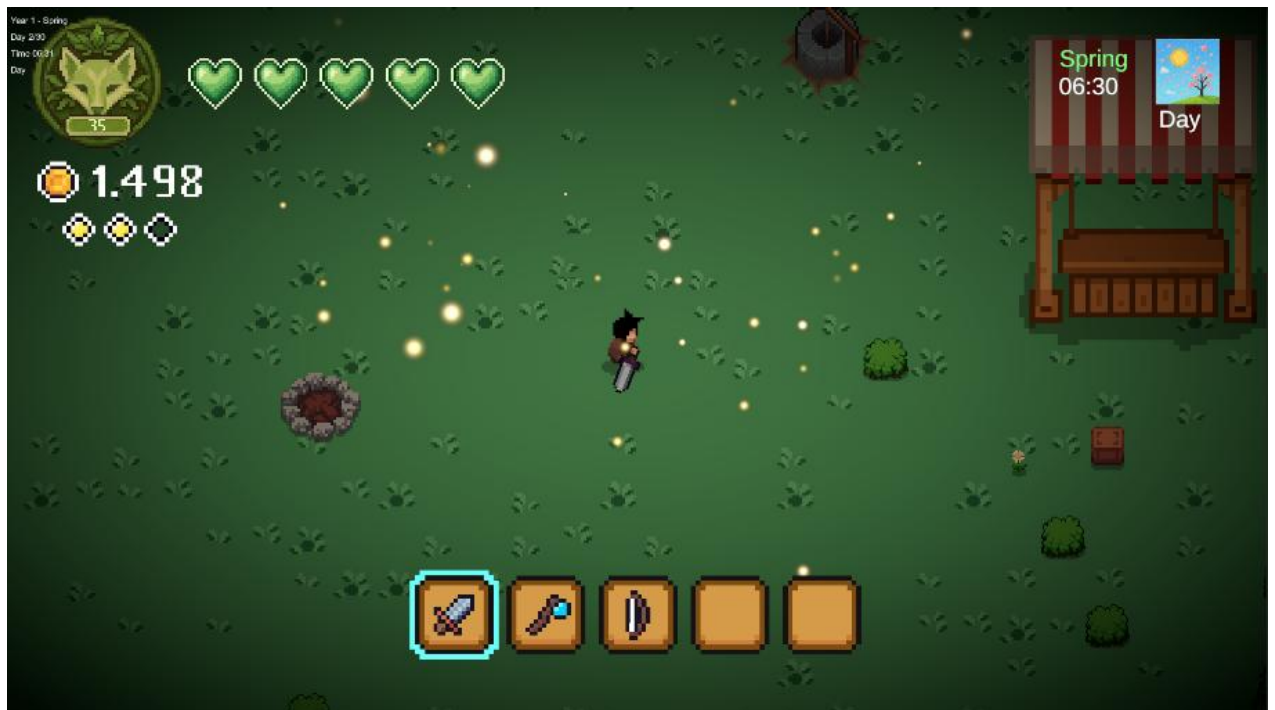
Picture 4.46. Asking for Village Position

NPC show the way and trigger firefly path:



Picture 4.47. NPC show the way

Firefly path lead to the Village:



Picture 4.48. Firefly pathfinding

PART 3: CONCLUSION

I. CONCLUSION

a. Achieved Results

The game design and development have successfully achieved the initial objectives, creating an immersive NPC-driven experience with intelligent AI behaviors, natural dialogue interactions, and dynamic economy systems. Special attention was given to creating a living world where NPCs follow daily schedules, respond to player requests, and interact naturally through voice and text input. The tasks of the project have been completed with the following specific outcomes:

- **NPC AI System:**
 - Autonomous daily routines with time-based activities (market trading, flower gathering).
 - A* pathfinding for intelligent navigation around obstacles.
 - Player request override system allowing dynamic task assignment.
- **Dialogue and Interaction:**
 - Voice-to-AI dialogue system with speech-to-text integration
 - Multi-turn conversation support with context awareness
- **Quest System:**
 - Dynamic quest assignment through natural dialogue
 - Real-time progress tracking with objective updates
 - Quest completion with reward distribution (gold, experience, items)
- **Market Trading System:**
 - Time-based shop availability (NPCs open shops during market hours)
 - Shop inventory management with database integration
 - Shopping cart functionality with multi-currency support
- **Currency System:**
 - Six-tier currency economy (Obal, Varos, Sylv, Feron, Astryl, Aurum)
 - Automatic currency conversion and display
 - Secure transaction validation
- **Time Management:**
 - 24-hour day/night cycle with dynamic lighting transitions
 - Trigger NPCs action at several time in a days.
- **Leveling System:**
 - Experience-based progression with exponential scaling
 - Stat increases upon leveling up
 - Update Level UI
- **Inventory Management:**
 - Stackable item system with capacity limits
 - Database persistence for saved progress
- **In-game Interface:**
 - Main Menu interface when starting the game.
 - Pause Menu interface when pressing the [ESC] key.
- **Programming Functions for the Game:**
 - Player character movement controlled by predefined keys.
 - Dash system.
 - Weapon switching system.

- Weapon-specific functionalities: Sword, Magic Staff, Bow.
- Collision detection and event handling between objects and attributes in the game.
- Item drop handling after defeating enemies or destroying structures.
- Particle System for In-game Effects
- Sound effects and background music for the game.

b. Existing Issues

As the game is still under development, minor issues are expected. For example, NPCs occasionally misinterpret player speech and generate incorrect responses, and some NPC routines remain incomplete. In certain cases, pathfinding errors may cause NPCs to walk through cliffs or rivers due to unfinished navigation logic. Additionally, because of limited experience with animation, most NPCs currently only have basic idle animations, which reduces the overall visual quality of the game. More animations—such as gathering, trading, moving, and interacting—need to be added for both NPCs and the player to improve realism and immersion.

The small monsters and bosses programmed are relatively simple and do not offer much excitement for players. The game environment is still somewhat limited, lacking many new terrains to explore, which affects the overall gameplay experience. Additionally, important features such as Settings and Save Game functionality are missing, which diminishes the player's overall experience.

II. DEVELOPMENT DIRECTION

In the upcoming period, *Shadow of Eclycia* will receive new features to diversify and improve the game, including:

- Fixing existing issues in the previous version.
- Adding a Settings feature to allow players to customize their preferences.
- Implementing a Save Game functionality to let players save their progress.
- Introducing new monsters, bosses, and NPCs.
- Expanding the game world with new terrains.
- Introducing new weapons.
- Adding new spells that players can learn and use with the Magic Staff..
- Adding a Rune System with four slots for buffs, which appear randomly during gameplay in each chapter.
- Introducing a survival system that includes hunger, thirst, fatigue, sleepiness, and temperature. (Player will respawn at the bed where they sleep if they die)
- Randomly dropping items such as spell books, different quality weapons, runes, food, drinks, armor, and shields.
- Adding player upgrade attributes.
- Adding new animations such as rolling, climbing, diving, swimming, digging, and jumping.
- Adding roles for player to choose their roles, each role have a different ability
- Enhancing the boss's abilities for more complexity and variety.
- Expanding the game's online multiplayer capabilities to allow interaction with other players.

REFERENCES

1. **Unity Technologies.** *Unity Engine Documentation*. Retrieved from: <https://docs.unity.com>
2. **Brackeys.** (n.d.). *Unity 2D & 3D Game Development Tutorials* [YouTube Channel]. Retrieved from: <https://www.youtube.com/@Brackeys>
3. **Abdelmoumene, B.** (n.d.). *Unity Programming Tutorials* [YouTube Channel]. Retrieved from: <https://www.youtube.com/@AbdelmoumeneUnity>
4. **Pandemonium.** (n.d.). *Unity 2D RPG Tutorials* [YouTube Channel]. Retrieved from: <https://www.youtube.com/@PandemoniumGameDev>
5. **BMo.** (2020). *Unity Menu UI Tutorial – Create a Main Menu* [Video]. Retrieved from: <https://www.youtube.com/watch?v=-GWjA6dixV4>
6. **Rehope Games.** (2021). *Scene Management in Unity* [Video]. Retrieved from: <https://www.youtube.com/watch?v=E25JWfeCFPA>
7. **Brackeys.** (2017). *Unity Animation Tutorial* [Video]. Retrieved from: <https://www.youtube.com/watch?v=hkaysu1Z-N8>
8. **diving_squid.** (2021). *NPC Dialogue System in Unity* [Video]. Retrieved from: <https://www.youtube.com/watch?v=1nFNOyCalzo>
9. **Jimmy Vegas.** (n.d.). *Creating Credits Scene in Unity* [Video]. Retrieved from: <https://www.youtube.com/watch?v=cj6hwCjiVZE>
10. **itch.io.** (n.d.). *Free Game Assets, Sprites, and Pixel Art Resources*. Retrieved from: <https://itch.io>
11. **Unity Asset Store.** (n.d.). *2D Sprite Packs, Prefabs, and Tools*. Retrieved from: <https://assetstore.unity.com>
12. **OpenAI Whisper.** (2022). *Speech-to-Text Model*. Retrieved from: <https://github.com/openai/whisper>
13. **HuggingFace.** *Transformers & Sentence-Transformers Library*. Retrieved from: <https://huggingface.co>
14. **Reimers, N., & Gurevych, I.** (2019). *Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks*. Retrieved from: <https://www.sbert.net>
15. **Microsoft Edge TTS.** (2023). *Text-to-Speech API Documentation*. Retrieved from: <https://learn.microsoft.com/en-us/azure/cognitive-services/speech-service>
16. **LM Studio.** (2024). *Local LLM Inference Platform*. Retrieved from: <https://lmstudio.ai>
17. **MySQL Documentation.** (n.d.). *MySQL 8.0 Reference Manual*. Retrieved from: <https://dev.mysql.com/doc>
18. **Flask Documentation.** (n.d.). *Python Flask Web Framework*. Retrieved from: <https://flask.palletsprojects.com>
19. **Latitude.** (2019). *AI Dungeon – AI-Powered Text Adventure Game*. Retrieved from: <https://aidungeon.io>
20. **Bethesda Game Studios.** (2011). *The Elder Scrolls V: Skyrim*. NPC routine system reference.
21. **Google Assistant / Alexa Documentation.** (n.d.). Voice interaction design guidelines.
22. **A Pathfinding Algorithm – Red Blob Games.*** (n.d.). *Introduction to A Pathfinding.** Retrieved from: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>
23. **OpenAI.** (2024). *GPT Models & API Documentation*. Retrieved from: <https://platform.openai.com/docs>
24. **Free Game Assets.** (n.d.). *Home Top-Down Pixel Art Pack*. Retrieved from: <https://free-game-assets.itch.io>